

INDEX

INDEX	1
VISIÓ PER COMPUTADOR	2
INTRODUCCIÓ A MATLAB	3
HOMEWORK 0: INTRODUCCIÓ A L'ENTORN DE TREBALL MATLAB	3
ENTREGA 1: TRACTAMENT D'IMATGES AMB MATLAB	6
INTRODUCCIÓ ALS FORMATS I OPERACIONS D'IMATGE	10
ENTREGA 2: FORMATS D'IMATGE RGB I HSV	10
ENTREGA 3: OPERACIONS BÀSIQUES AMB IMATGES.....	15
HOMEWORK 1	21
PREPROCESSAT D'IMATGE: PRODUCTES CONVOLUCIONALS	23
ENTREGA 4: OPERADORS DE SUAVITZAT	24
ENTREGA 5: OPERADORS DE REALÇAT	28
ENTREGA 7: OPERADORS MIXTES.....	35
EXERCICI INDIVIDUAL 1.....	37
PREPROCESSAT D'IMATGE: OPERACIONS MORFOLÒGIQUES	41
ENTREGA 8: DILATACIONS, EROSIONS I TRANSFORMADES DE DISTÀNCIA	42
ENTREGA 9: OPEN/CLOSE, RECONSTRUCCIONS I DILATACIONS CONDICIONALS.	48
ENTREGA 10: ESQUELETS.....	55
ENTREGA 11: OPERACIONS MORFOLÒGIQUES MULTINIVELL	61
ENTREGA 12: BINARITZACIÓ I LABELLING.....	67
SEGMENTACIÓ DE LA IMATGE	75
ENTREGA 13: ALGORISME DE CANNY.....	76
ENTREGA 14: WATERSHED.....	77
ENTREGA 15: K-MEANS CLUSTERING	85
EXTRACCIÓ DE DESCRIPTORS	93
ENTREGA 16: DESCRIPTORS I.....	94
ENTREGA 17: DESCRIPTORS II.....	100
INTRODUCCIÓ A LA CLASSIFICACIÓ	104
MATCHING I PATTERN RECOGNITION	104
LOCAL FEATURES	112
ENTREGA 18: HOGS I HISTOGRAMES DE COLOR	113
ENTREGA 19: TRANSFORMADA DE HOUGH	121
EXERCICI INDIVIDUAL 2.....	125
ENTREGA 20: KEYPOINTS I.....	127
ENTREGA 21: KEYPOINTS II.....	137
ENTREGA 22: DESCRIPTORS DE HAAR I L'ALGORISME DE VIOLA-JONES.....	140
ÚS DE XARXES NEURONALS A MATLAB	143

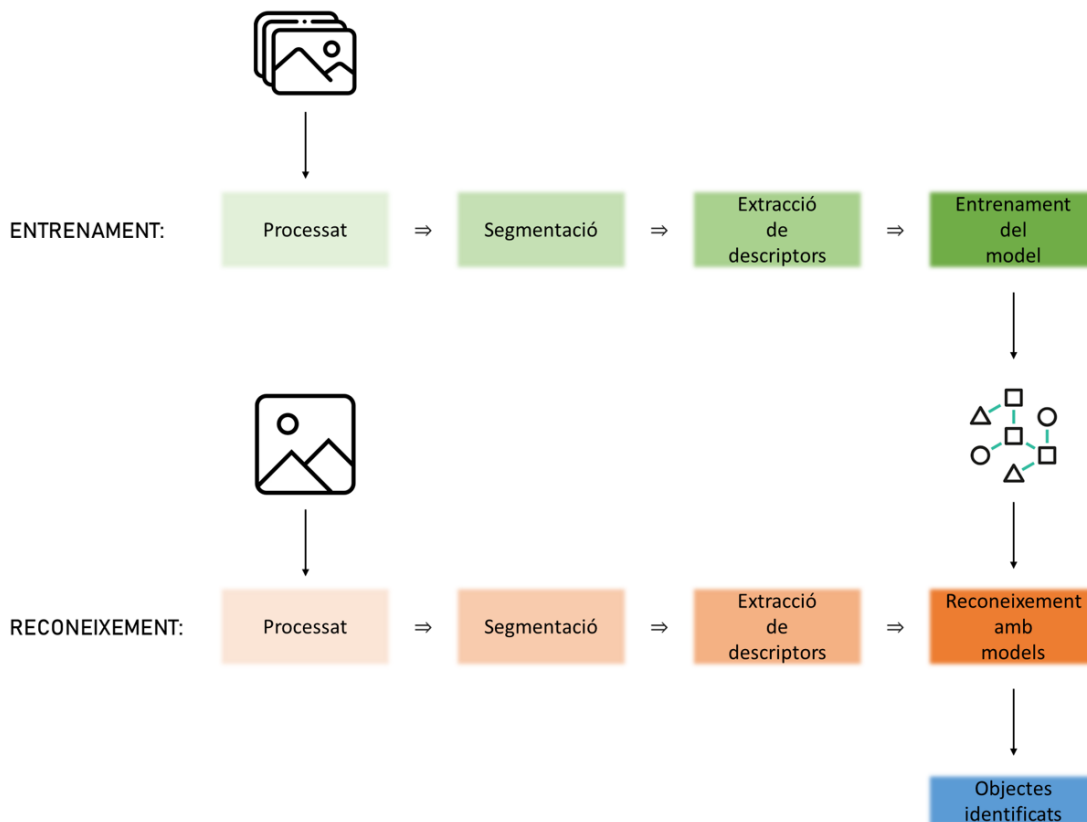
VISIÓ PER COMPUTADOR

La **Visió per Computador** és un camp de la informàtica que permet als ordinadors interpretar i analitzar imatges o vídeos de manera similar a com ho faria un ésser humà. És una disciplina que combina matemàtiques, programació i teoria de la percepció per a extreure informació útil d'imatges digitals.

El procés de visió i reconeixement de l'ésser humà és molt avançat i identifica sense esforç el que percep a través de la vista. Però quan tractem amb ordinadors, cada pas que nosaltres duem a terme sense esforç ni ser conscients suposa un gran repte, ja sigui tractar una imatge per a identificar les diferents parts o identificar quines són les qualitats que descriuen un objecte i ens permeten saber què és.

L'objectiu d'aquest curs és el reconeixement d'imatges, partint dels estadis més primaris de tractament d'imatge fins a arribar a la identificació dels seus elements mitjançant models entrenats.

El *pipeline* de treball de la Visió per Computador és el següent:



Com es pot observar, els primers passos de l'entrenament de models i del reconeixement d'imatges són els mateixos, això es deu a que els ordinadors no treballen directament amb les imatges, sinó que n'extreuen diferents característiques que els permeten trobar patrons i similituds.

Les fases de processat, segmentació i extracció de característiques són automàtiques, però un programador les ha d'haver dissenyat i implementat prèviament. Avui dia, amb els avenços en xarxes neuronals i el *Deep Learning*, existeixen sistemes capaços de trobar de manera autònoma algorismes que duguin a terme les fases de segmentació i extracció de descriptors, però això queda fora de l'abast del curs i per tant, tot i que s'esmentarà en algun moment, no ho tractarem en profunditat.

INTRODUCCIÓ A MATLAB

Homework 0: Introducció a l'entorn de treball MATLAB

1. Genera una matriu A de 10x10 amb valors aleatoris entre 0 i 255 de tipus enter

```
A = randi([0, 255], 10, 10)
```

A = 10x10

181	149	38	157	97	3	176	20	66	140
193	57	65	121	145	86	191	113	204	37
70	192	215	90	19	41	115	27	110	218
174	65	65	212	13	203	21	246	233	159
167	129	208	149	135	79	58	1	46	89
41	178	62	140	199	135	233	198	67	131
30	228	237	234	239	42	39	209	37	102
127	245	89	73	33	154	211	222	34	19
245	140	50	193	145	67	137	21	222	61
87	35	64	192	120	167	255	102	148	31

2. Obté un vector amb la 4ª fila de A

```
v1 = A(4, :)
```

v1 = 1x10

174	65	65	212	13	203	21	246	233	159
-----	----	----	-----	----	-----	----	-----	-----	-----

3. Obté un vector amb la 4ª columna de A

```
v2 = A(:, 4)
```

v2 = 10x1

157
121
90
212
149
140
234
73
193
192

4. Obté una matriu on s'hagi suprimit la 4ª columna de A

```
B = A(:, [1:3, 5:end])
```

B = 10x9

181	149	38	97	3	176	20	66	140
193	57	65	145	86	191	113	204	37
70	192	215	19	41	115	27	110	218
174	65	65	13	203	21	246	233	159
167	129	208	135	79	58	1	46	89
41	178	62	199	135	233	198	67	131
30	228	237	239	42	39	209	37	102
127	245	89	33	154	211	222	34	19
245	140	50	145	67	137	21	222	61
87	35	64	120	167	255	102	148	31

5. Obté un vector amb el valor màxim de cada columna de A

```
vmax = max(A(:,1:end))

vmax = 1x10
    245    245    237    234    239    203    255    246    233    218
```

6. Obté el valor màxim de la matriu A

```
maxA = max(A(:))

maxA = 255
```

7. Obté una matriu amb només les files parells de A

```
C = A(2:2:end, :)

C = 5x10
    193     57     65    121    145     86    191    113    204     37
    174     65     65    212     13    203     21    246    233    159
     41    178     62    140    199    135    233    198     67    131
    127    245     89     73     33    154    211    222     34     19
     87     35     64    192    120    167    255    102    148     31
```

8. Obté la fila i columna on es troba el valor mínim de A

```
[~, imin] = min(min(A))

imin = 8
```

9. Genera la matriu B trasposant la matriu A

```
B = A'

B = 10x10
    181    193     70    174    167     41     30    127    245     87
    149     57    192     65    129    178    228    245    140     35
     38     65    215     65    208     62    237     89     50     64
    157    121     90    212    149    140    234     73    193    192
     97    145     19     13    135    199    239     33    145    120
     3     86     41    203     79    135     42    154     67    167
    176    191    115     21     58    233     39    211    137    255
     20    113     27    246     1    198    209    222     21    102
     66    204    110    233     46     67     37     34    222    148
    140     37    218    159     89    131    102     19     61     31
```

10. Obté el producte de les matrius A i B

```
prod = A*B

prod = 10x10
    145805    133736    124104    125057    119801    145717 ...
    133736    180020    111122    171731    121370    164886
    124104    111122    171709    135634    131551    140361
    125057    171731    135634    265575    126676    172447
    119801    121370    131551    126676    147863    129548
    145717    164886    140361    172447    129548    229778
    136221    143160    160588    173758    167418    208809
    124478    144149    126717    160961    114539    189798
```

159396	184740	131692	180850	146608	161929
126707	169510	108094	167598	114816	180658

11. Obté el producte element a element de A i B

```
prod2 = A.*B
```

```
prod2 = 10x10
    32761    28757    2660    27318    16199    123 ...
    28757     3249    12480     7865    18705    15308
     2660    12480    46225     5850     3952     2542
    27318     7865     5850    44944     1937    28420
    16199    18705     3952     1937    18225    15721
     123     15308     2542    28420    15721    18225
     5280    43548    27255     4914    13862     9786
     2540    27685     2403    17958         33    30492
    16170    28560     5500    44969     6670     4489
    12180     1295    13952    30528    10680    21877
```

12. Genera una matriu booleana on cada element (i,j) valgui 1 si A(i,j) > B(i,j), i 0 en cas contrari

```
D = A > B
```

```
D = 10x10 logical array
    0     0     0     0     0     0     1     0     0     1
    1     0     0     1     1     0     0     0     1     1
    1     1     0     1     0     0     0     0     1     1
    1     0     0     0     0     1     0     1     1     0
    1     0     1     1     0     0     0     0     0     0
    1     1     1     0     1     0     1     1     0     0
    0     1     1     1     1     0     0     0     0     0
    1     1     1     0     1     0     1     0     1     0
    1     0     0     0     1     0     1     0     0     0
    0     0     0     1     1     1     1     1     1     0
```

13. Genera un vector amb tots els elements A(i,j) més grans que B(i,j)

```
vgreater = A(A > B)' % el trasposem perquè no ocupi una pàgina sencera a l'informe
```

```
vgreater = 1x44
    193     70    174    167     41    127    245    192    178    228    245    208 ...
```

14. Genera una matriu on cada element (i,j) valgui A(i,j) si A(i,j)>B(i,j) , i 0 en cas contrari

```
E = A;
E(A <= B) = 0
```

```
E = 10x10
     0     0     0     0     0     0    176     0     0    140
    193     0     0    121    145     0     0     0    204     37
     70    192     0     90     0     0     0     0    110    218
    174     0     0     0     0    203     0    246    233     0
    167     0    208    149     0     0     0     0     0     0
     41    178     62     0    199     0    233    198     0     0
     0    228    237    234    239     0     0     0     0     0
    127    245     89     0     33     0    211     0     34     0
    245     0     0     0    145     0    137     0     0     0
     0     0     0    192    120    167    255    102    148     0
```

Entrega 1: Tractament d'imatges amb MATLAB

Com les imatges són una sèrie de píxels disposats en un ordre determinat, podem tractar-les amb matrius on cada cel·la conté el valor d'un píxel.

Funcions bàsiques per treballar amb imatges a MATLAB

```
im=imread('Floppy.bmp'); % Llegeix la imatge del directori de treball  
[files, cols] = size(im);
```

```
im(121,54) % Seleccióem un valor (píxel) de la imatge
```

```
ans = uint8  
77
```

```
im(121:125,54:60) % Seleccióem un rang de valors (píxels) de la imatge
```

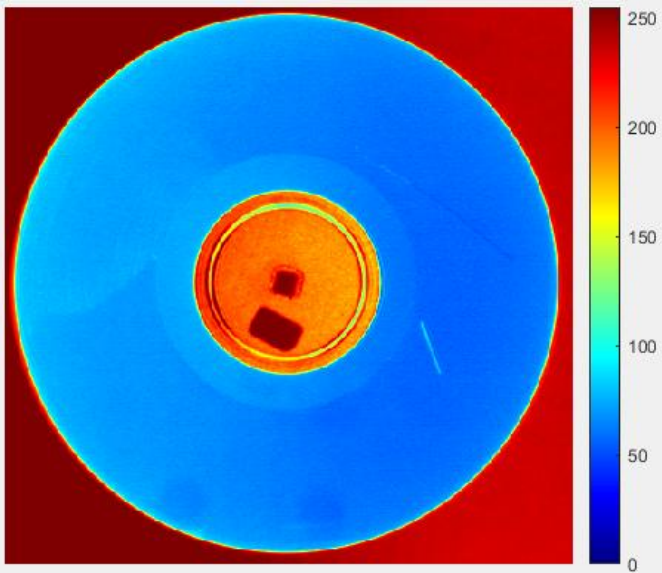
```
ans = 5x7 uint8 matrix  
77    77    76    77    75    74    74  
75    75    75    73    73    73    73  
77    76    76    76    75    75    74  
75    75    75    76    76    75    74  
76    77    76    75    76    76    74
```

```
figure, imshow(im) % Mostra la Imatge a una nova figura/finestra  
impixelinfo % Mostra informació del píxel sobre el que es troba el ratolí  
%improfile % Permet seleccionar un rang de píxels i genera un gràfic  
%segons els seus valors
```

```
% Mapa de colors  
figure, imshow(im)
```

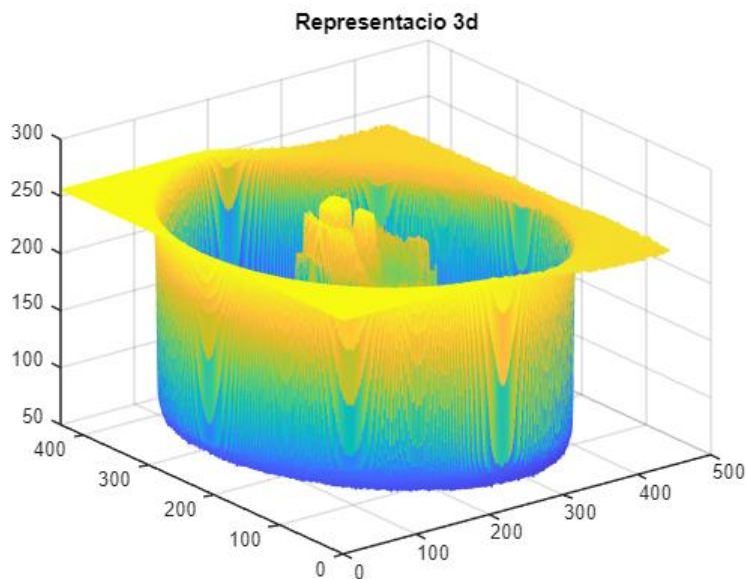


```
colorbar, colormap jet % Mostra una llegenda i coloreja la imatge segons els seus valors
```

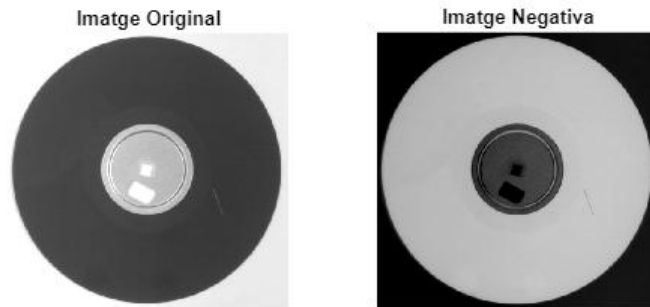


Pixel info: (X, Y) Pixel Value

```
figure, mesh(im), title('Representacio 3d')
```



```
% Subplot ens permet representar diverses imatges en una sola figura  
% subplot(files, cols, index)  
subplot(1,2,1); imshow(im), title('Imatge Original')  
subplot(1,2,2); imshow(255-im), title('Imatge Negativa')
```



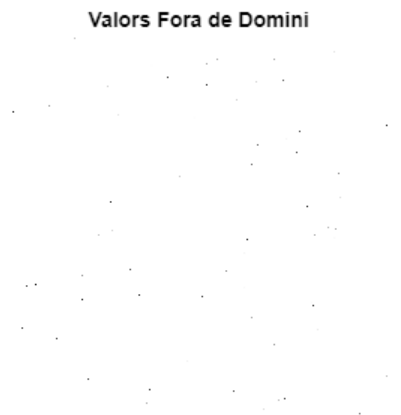
```
imwrite(255-im,"negatiu.jpg") % Guarda la imatge al directori de treball

% close all; clear all
% Tanca totes les figures obertes i neteja el workspace
```

Problemes amb el domini

La funció `imshow()` per defecte representa valors de tipus Byte (dins del domini `[0, 255]`). Però què passa si hem de visualitzar valors fora d'aquest rang?

```
% Generem una imatge amb valors aleatòris i observem que imshow() la
% representa pràcticament sencera de color blanc perquè la majoria dels
% seus píxels se'ns surten de rang (superen el 255) .
B = rand(256).*1000;
figure, imshow(B), title('Valors Fora de Domini')
```

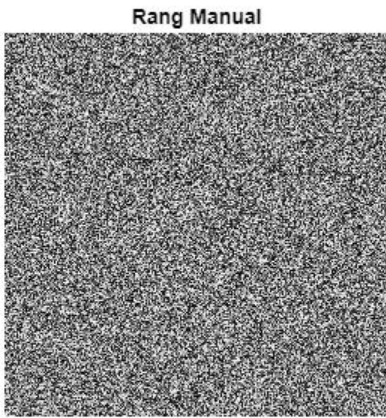


```
min(B(:)), max(B(:)) % Observem quins són el valor màxim i mínim de la imatge
```

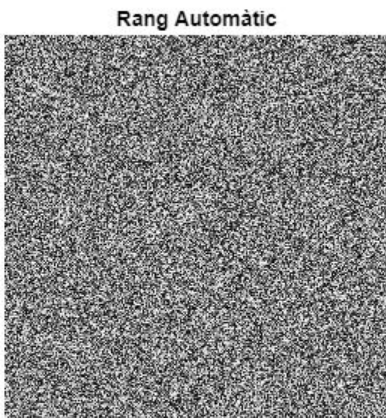
```
ans = 0.0357
ans = 999.9930
```



```
% D'aquesta manera li indiquem que el valor més alt és 1000 i el més baix  
% és 0 És a dir que el blanc més blanc és 1000 i el negre més negre és 0  
figure,imshow(B,[0,1000]), title('Rang Manual')
```



```
% D'aquesta altra manera no li indiquem nosaltres quin valor ha  
% d'interpretar com el més alt ni el més baix així que pren el rang  
% dinàmicament. Agafa els valor mínim i màxim de la imatge.  
figure,imshow(B,[]),title('Rang Automàtic')
```



Introducció als Formats i Operacions d'Imatge

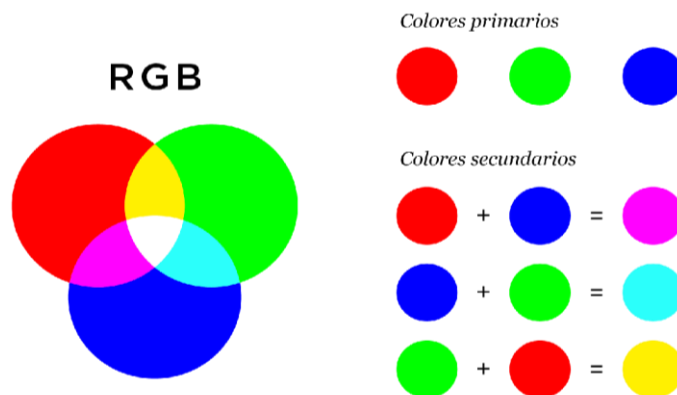
Entrega 2: Formats d'Imatge RGB i HSV

```
% Obrim la imatge
im=imread('flowers.tif');
imshow(im), title('Imatge Original')
impixelinfo
```



Format RGB

```
% Descomposem les 3 components de la imatge (la 3a Dimensió de l'arxiu)
% Cadascun representa la quantitat de llum [Red, Green i Blue] respectivament
R=im(:, :, 1);
G=im(:, :, 2);
B=im(:, :, 3);
figure, imshow(R), title('Component R')
```



```
% Mostrem una mapa de colors amb tonalitats de Vermell
% Generem una matriu nul·la 256x3 i omplim la 1a columna (component Red)
% amb números random del rang [0, 255]
lut=zeros(256,3);
lut(:,1)=0:255;
lut=lut/255; % normalitzem
colormap(lut),colorbar
```



```
% Comprobem que concatenant les 3 componets obtenim la imatge original  
rgb = cat(3,R,G,B);  
figure, imshow(rgb), title('Imatge Original ajuntant Canals RGB')
```

imatge Original ajuntant Canals RGB



```
% Per representar la quantitat de llum representem la imatge en Gray  
mono = rgb2gray(rgb);  
figure, imshow(mono), title('Imatge en Escala de Grisos')
```

imatge en Escala de Grisos



```
% Normalitzem la llum a les 3 components sabent que:  
%   r = R/(R+G+B)  
%   g = G/(R+G+B)  
%   b = B/(R+G+B)  
%   r+g+b = 1  
sum=double(R)+double(G)+double(B);  
r = double(R)./sum;  
g = double(G)./sum;  
b = double(B)./sum;  
rgb_norm = cat(3,r,g,b);  
  
figure,imshow(rgb_norm), title('RGB amb la Il·luminació Normalitzada')
```

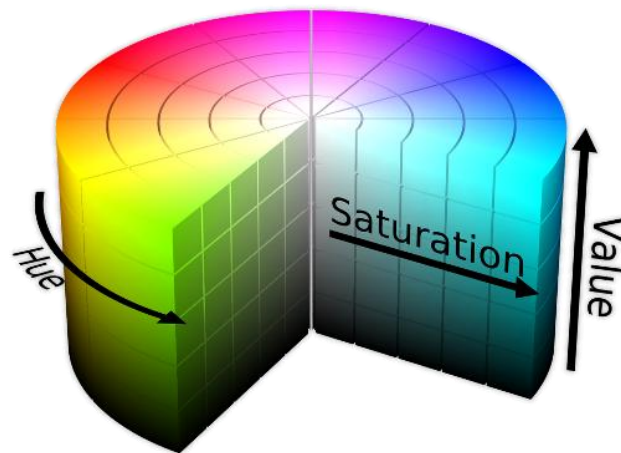
RGB amb la ll·luminació Normalitzada



Format HSV

```
hsv = rgb2hsv(im); % Transformem a HSV

% Descomposem les seves característiques
h = hsv(:, :, 1);
s = hsv(:, :, 2);
v = hsv(:, :, 3);
```



% Visualitzem les diferents components. Observem que imshow() no mostra
% correctament l'HSV, haurem de transformar-la abans a RGB.

```
subplot(2,2,1), imshow(h), title('Imatge Hue')
subplot(2,2,2), imshow(s), title('Imatge Saturació')
subplot(2,2,3), imshow(v), title('Imatge Value')
subplot(2,2,4), imshow(hsv), title('Imatge HSV')
```

Imatge Hue



Imatge Saturació



Imatge Value



Imatge HSV



```
% Normalitzar una imatge HSV: assignem un valor arbitrari al Value
v_norm = v.*0 + 0.5;
hsv_norm = cat(3,h,s,v_norm);
hsv_out = hsv2rgb(hsv_norm);
figure, imshow(hsv_out), title('HSV normalitzada')
```

HSV normalitzada



Exercici HSV

Control de qualitat de pastilles: a l'empresa se li han colat 3 pastilles de laxant a la producció d'un blister d'antitussigens, identifica les pastilles infiltrades i genera una imatge binària que les ressalti.

```
im2 = imread('Pills.tif');
figure, imshow(im2), title('Imatge Original')
```

imatge Original



```
hsv = rgb2hsv(im2);
h = hsv(:,:,1);
s = hsv(:,:,2);
v = hsv(:,:,3);
```

```
% Visualitzem les 3 components amb impixelinfo per analitzar els valors
subplot(3,1,1), imshow(h), title('Hue'), impixelinfo
subplot(3,1,2), imshow(s), title('Saturació'), impixelinfo
subplot(3,1,3), imshow(v), title('Value'), impixelinfo
```

Hue



Saturació



Value



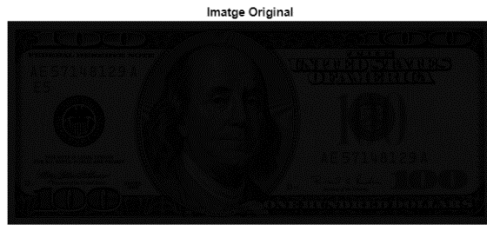
```
out = h > 0.3 & h < 0.45 & s > 0.2;  
figure, imshow(out), title('Imatge Binària Resultant')
```



Entrega 3: Operacions bàsiques amb Imatges

Tractar els nivells de gris d'una imatge

```
im=imread('Que_es.png');  
figure, imshow(im), title('Imatge Original')
```



```
% Fem la imatge més clara  
im2 = im+100;  
figure, imshow(im2), title('Offset')
```



```
% Separem els valors per a què el contingut de la imatge sigui apreciable  
im3 = im*10;  
figure, imshow(im3), title('Contrast')
```



```
% Invertim les tonalitats de gris (fem negatiu)  
im4 = 255-im;  
figure, imshow(im4), title('Negatiu de la Imatge Original')
```




```
im5 = 255-im3;
figure,imshow(im5),title('Negatiu del Contrast')
```



```
% Fem negatiu usant una funció ja implementada
im6 = imcomplement(im3);
figure,imshow(im6),title('Negatiu amb imcomplement')
```

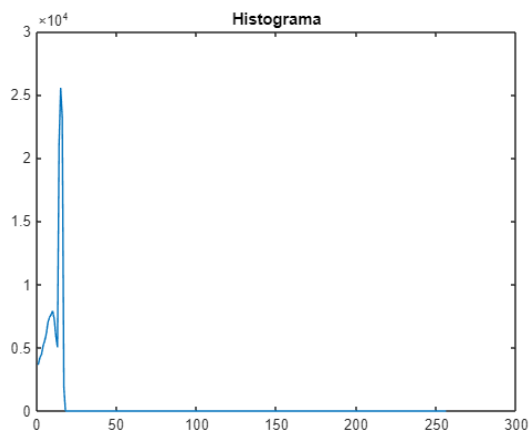


Equilibrar un histograma de grisos

```
im = imread("Que_es.png");
hist = zeros(256,1);
[f, c] = size(im);

for i = 1 : f
    for j = 1 : c
        % Llegim la imatge incrementant el valor pertinent a l'histograma
        hist(im(i,j)+1) = hist(im(i,j)+1) + 1;
    end
end

% Observem que el 17 és el màxim valor diferent de 0
figure, plot(hist), title('Histograma')
```




```

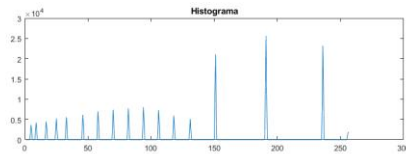
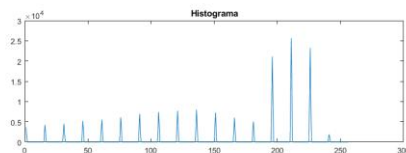
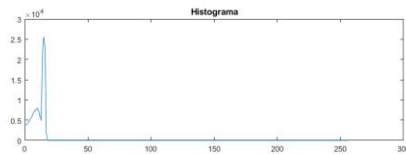
% Equalitzem la imatge i generem l'histograma
im2 = round(double(im) * 255 / 17);

hist2 = zeros(256,1);
for i = 1 : f
    for j = 1 : c
        hist2(im2(i,j)+1) = hist2(im2(i,j)+1) + 1;
    end
end

% Ara equalitzem l'histograma amb una funció ja implementada
hist3 = imhist(im); % Funció per a generar un histograma
im3=histeq(im);      % Funció per a equalitzar l'histograma
hist3 = imhist(im3);

subplot(3,2,1), imshow(im), title('Imatge Original')
subplot(3,2,2), plot(hist), title('Histograma')
subplot(3,2,3), imshow(im2), title('Imatge Equalitzada (per nosaltres)')
subplot(3,2,4), plot(hist2), title('Histograma')
subplot(3,2,5), imshow(im3), title('Imatge Equalitzada amb histeq')
subplot(3,2,6), plot(hist3), title('Histograma')

```



Transformacions d'Imatges

```

im = imread('lenna.tif');
figure, imshow(im), title('Imatge Original')

```

Imatge Original



```
im2 = imresize(im, 0.25);  
figure, imshow(im2), title('Zoom x0.25')
```

Zoom x0.25



```
im3 = imresize(im, 2);  
figure, imshow(im3), title('Zoom x2')
```

Zoom x2



```
% En intentar restaurar la imatge original amb una reduïda, perdem qualitat  
im4 = imresize(im2, 4);  
figure, imshow(im4), title('Imatge x0.25 x4')
```

imatge x0.25 x4



```
% En rotar la imatge, per tal de no perdre informació truncant la imatge,  
% s'afegeixen píxels a 0 (negre) automàticament  
im5 = imrotate(im, 45);  
figure, imshow(im5), title('Imatge rotada')
```

imatge rotada



```
% Li indiquem la Matriu de Transformació TG  
T = affine2d([1, 0, 0; 0.5, 1, 0; 0, 0, 1]);  
im6 = imwarp(im, T);  
figure, imshow(im6), title('Transformació afi')
```

Transformació afi



```
im1=imread('toycars1.png');  
im2=imread('toycars2.png');  
subplot(1,2,1), imshow(im1)  
subplot(1,2,2), imshow(im2)
```



```
% En restar imatges, és important obtenir també el valor absolut ja que en  
% cas contrari podem perdre informació  
im3 = im1-im2;  
figure, imshow(im3), title("Resta d'Imatges")
```

Resta d'Imatges



```
im4 = imabsdiff(im1, im2);  
figure, imshow(im4), title("Resta d'Imatges amb Valor Absolut")
```

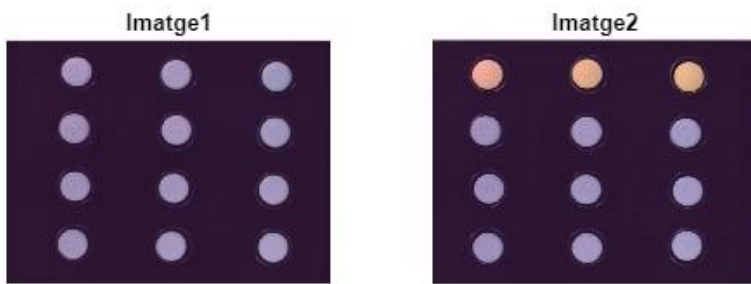
Resta d'Imatges amb Valor Absolut



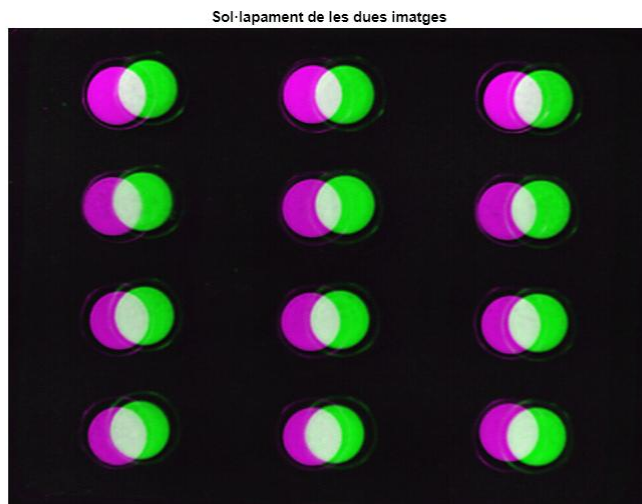
Homework 1

Control de qualitat de pastilles: Una companyia farmacèutica ens demana que desenvolupem un programa que identifiqui pastilles que s'empaqueten a blisters errònis. Ens proporcionen imatges de la seva producció i ens asseguren que totes tenen la mateixa mida, però no ens poden garantir que totes tinguin la exactament mateixa orientació.

```
im1 = imread('Blispac1.tif');  
im2 = imread('Blispac2.tif');  
  
subplot(1,2,1), imshow(im1), title('Imatge1');  
subplot(1,2,2), imshow(im2), title('Imatge2');
```



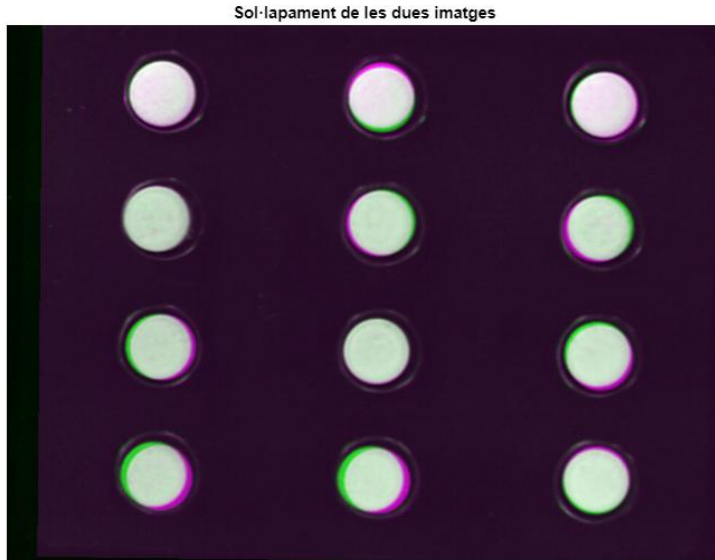
```
% Comprovem que les dues imatges no estan alineades  
figure, imshow(imfuse(im1, im2)), title('Sol·lapament de les dues imatges')
```



```
% Seleccionem manualment els punts de control equivalents de les dues imatges  
[punts1, punts2] = cpselect(im1, im2, 'Wait', true);  
  
% Ajustem la matriu de transformació afí per alinear im2 amb im1  
TG = fitgeotrans(punts2, punts1, 'affine');  
new_im2 = imwarp(im2, TG, 'OutputView', imref2d(size(im1)));
```

```
% Comprovem que les dues imatges estiguin ben alineades
```

```
figure, imshow(imfuse(im1, new_im2)), title('Sol·lapament de les dues imatges')
```



```
% Convertim a HSV per a extreure el Color
```

```
hsv1 = rgb2hsv(im1);
```

```
hsv2 = rgb2hsv(new_im2);
```

```
hue1 = hsv1(:, :, 1);
```

```
hue2 = hsv2(:, :, 1);
```

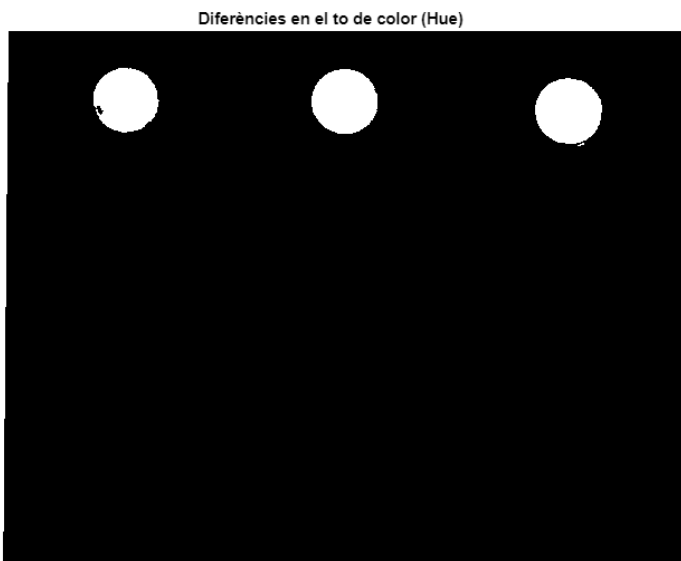
```
% Trobem la diferencia de hue entre les dues imatges
```

```
dif_hue = abs(hue1 - hue2);
```

```
% Creem una imatge binària que indiqui si la diferència de Hue és gran o petita
```

```
res = dif_hue > 0.3;
```

```
figure, imshow(res), title('Diferències en el to de color (Hue)');
```



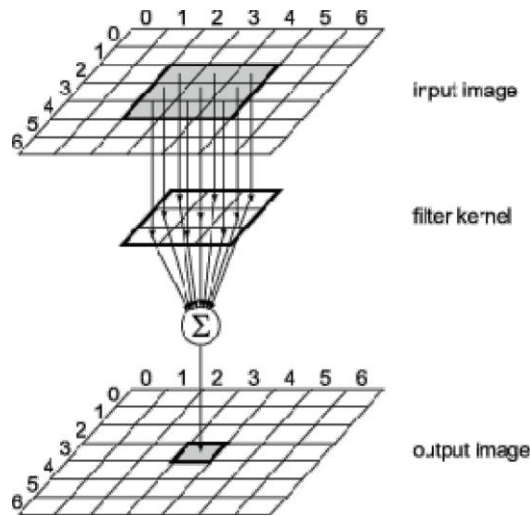
Preprocessat d'Imatge: Productes Convolucionals

Un producte de convolució és una operació molt utilitzada que serveix per a aplicar filtres que ens permetran suavitzar imatges, realçar-les, detectar contorns, etc. Consisteix a passar una "finestra" (kernel) per tots els píxels de la imatge i assignar a cada píxel resultant la suma dels productes Kernel*Imatge.

Matemàticament equival a combinar dues funcions diferents per a obtenir una 3a: $S(x, y)$ és una combinació de $I(x, y)$ i $K(x, y)$

$$S(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k I(x+i, y+j) * K(i, j)$$

Observem que l'expressió matemàtica es pot implementar amb dos bucles enniestrats que multipliquin els valors pertinents de la imatge amb els respectius del Kernel i després assignin el resultat al píxel que estiguem tractant en aquell moment.



Tipus d'Operadors:

- **Operadors de Suavitzat:** filtren el soroll però la imatge perd detalls, actuen com a filtres passa baix. També se'ls coneix com a "filtres integratius".

- **Operadors de Realçat:** ressalten els detalls però amplifiquen el soroll, actuen com a filtres passa alt. També se'ls coneix com a "filtres derivatius".

Els primers tenen utilitats obvies, ja que eliminar soroll resulta clau a l'hora de tractar amb imatges. Però per a què voldríem amplificar soroll? La resposta és que serveix per a la detecció de contorns, que ens ajuda a identificar figures.

Entrega 4: Operadors de suavitzat

```
kernel = [0,1,0; 1,2,1; 0,1,0] % Creem un Kernel
```

```
kernel = 3x3
```

```
0     1     0
1     2     1
0     1     0
```

```
% Creem la imatge original
```

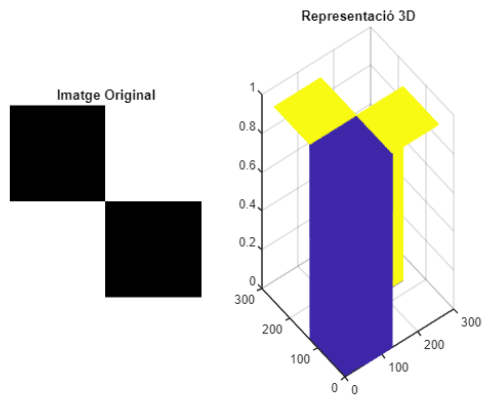
```
im = ones(256);
```

```
im(1:128, 1:128) = 0;
```

```
im(129:256,129:256) = 0;
```

```
subplot(1,2,1), imshow(im), title('Imatge Original')
```

```
subplot(1,2,2), mesh(im), title('Representació 3D')
```

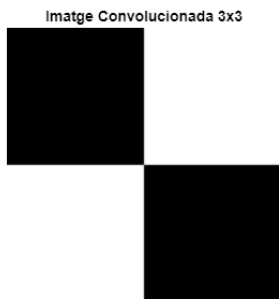


```
% Convolucionem la imatge
```

```
kernel = kernel/6;
```

```
res = imfilter(im,kernel,"replicate",'conv');
```

```
figure, imshow(res), title('Imatge Convolucionada 3x3')
```



```
% Convolucionem la imatge amb un Kernel més gran
```

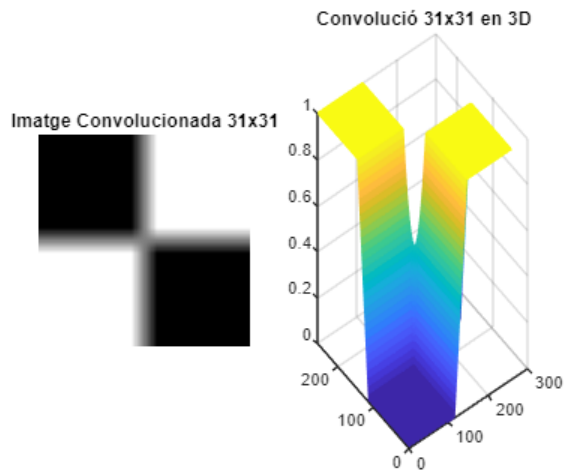
```
h = ones(31);
```

```
h = h/31/31;
```

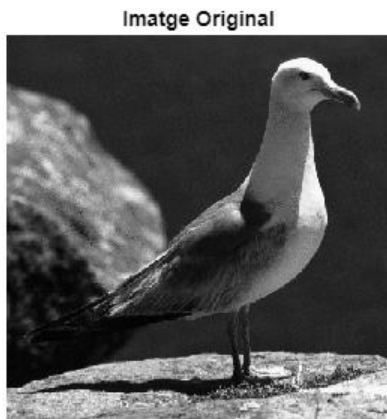
```
res2 = imfilter(im, h,"replicate", 'conv');
```

```
subplot(1,2,1), imshow(res2), title('Imatge Convolucionada 31x31')
```

```
subplot(1,2,2), mesh(res2), title('Convolució 31x31 en 3D')
```

```
im = imread('gull.tif');
figure, imshow(im), title('Imatge Original')
```



```
% Afegim soroll gaussià per provar els diferents operadors
img = imnoise(im, 'gaussian');
imsp = imnoise(im, 'salt & pepper', 0.2);

kg = fspecial("gaussian",7,2); % Creem un Kernel seguint una distribució Gaussiana
resg = imfilter(img,kg);      % Apliquem el Filtre Gaussià
resp = imfilter(imsp,kg,'conv'); % Apliquem el Filtre Gaussià
resmed = medfilt2(imsp,[5,5]); % Apliquem un Filtre de Mediana

subplot(3,2,1), imshow(img), title('Soroll Gaussià')
subplot(3,2,2), imshow(resg), title('Filtratge Gaussià (Soroll Gaussià)')
subplot(3,2,3), imshow(imsp), title('Soroll Impulsional')
subplot(3,2,4), imshow(resp), title('Filtratge Gaussià (Soroll S&P)')
subplot(3,2,5), imshow(imsp), title('Soroll Impulsional')
subplot(3,2,6), imshow(resmed), title('Filtratge Mediana (Soroll S&P)')
```

Soroll Gaussià



Filtratge Gaussià (Soroll Gaussià)



Soroll Impulsional



Filtratge Gaussià (Soroll S&P)



Soroll Impulsional



Filtratge Mediana (Soroll S&P)



La nostra convolució

Creem la nostra pròpia funció de convolució per tal de comprovar que hem assimilat bé el concepte, normalment utilitzarem les que ja venen implementades amb MATLAB.

```
im=imread('lenna.tif');  
figure, imshow(im), title('Imatge Original')
```

Imatge Original



```

[files cols] = size(im);

im2 = zeros(files, cols);
kernel = [0,1,0; 1,2,1; 0,1,0];
kernel = kernel/6; % Tots els seus elements han de sumar 1

for i = 1:files
    for j = 1:cols
        suma = 0;
        for k = -1 : 1
            for l = -1 : 1
                if i+k > 1 && i+k < files && j+l < cols && j+l > 1
                    suma = suma + im(i+k, j+l) * kernel(k+2, l+2);
                end
            end
        end
        im2(i,j) = suma;
    end
end

% Observem que hem minimitzat el soroll però hem perdut nitidessa
subplot(1, 2, 1), imshow(im), title('Imatge Original')
subplot(1, 2, 2), imshow(im2, []), title('Imatge Convolucionada')

```



Entrega 5: Operadors de realçat

És important diferenciar entre vores i contorns:

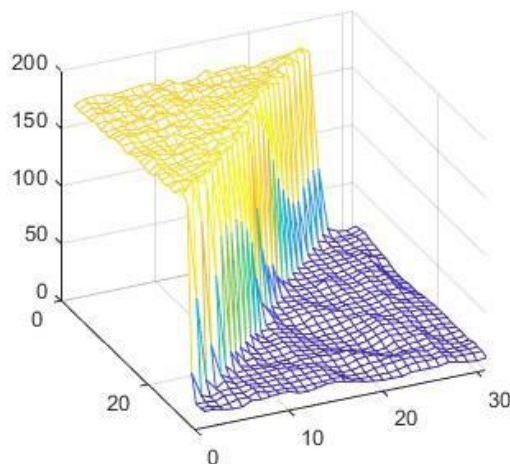
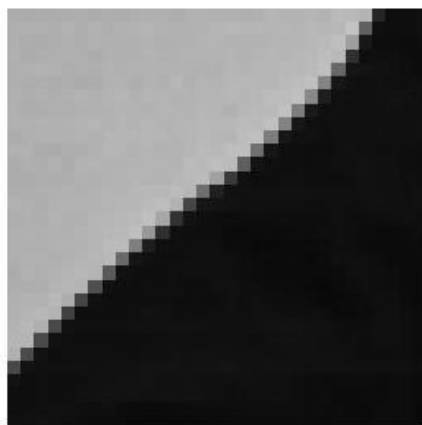
- **Contorn / edge**: lloc on hi ha una ràpida transició en la intensitat de la imatge.

- **Vora / contour**: els píxels límit de la imatge.

Podem interpretar una imatge en escala de grisos com una figura tridimensional, on la 3a dimensió és la profunditat segons el color de cada píxel. Fent la derivada d'aquesta funció podem trobar els 'edges' de les figures representades a la imatge, ja que allà on hi hagi canvis de color significatius, el canvi de la pendent serà notable.

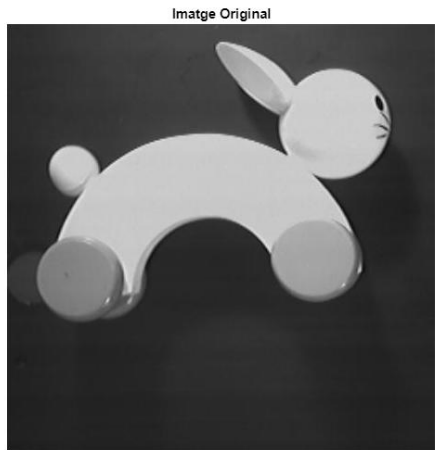
Gradient: el gradient és un concepte que apareix en tractar amb derivades multivariables i ens indica la pendent de la figura tridimensional a cada punt de la funció. Pel que respecta a Visió per Computador, el gradient acaba sent una finestra de convolució de 3 elements: $[-1, 0, 1]$.

Per a veure gràficament els paràgrafs anteriors tenim l'exemple següent:



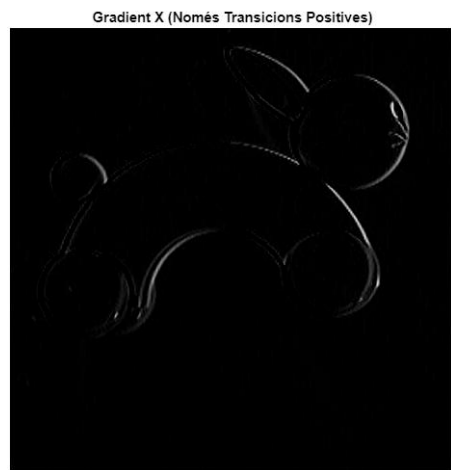
Podem observar que allà on hi ha el canvi sobtat de tonalitat, la pendent de la funció es dispara i conseqüentment el gradient també. Per tant, el què ens interessarà de trobar el gradient serà conèixer el seu mòdul i saber-ne la direcció. La magnitud del mòdul ens permetrà discernir si ens trobem davant d'un contorn o un canvi d'intensitat sense importància i la direcció ens ajudarà a saber l'orientació del contorn (quina banda és més fosca i quina és més clara).

```
im = imread('rabbit.jpg'); % Observem que la imatge es uint
figure, imshow(im), title('Imatge Original')
```



```
k = [-1,0,1];
Gx = imfilter(im,k,'conv');

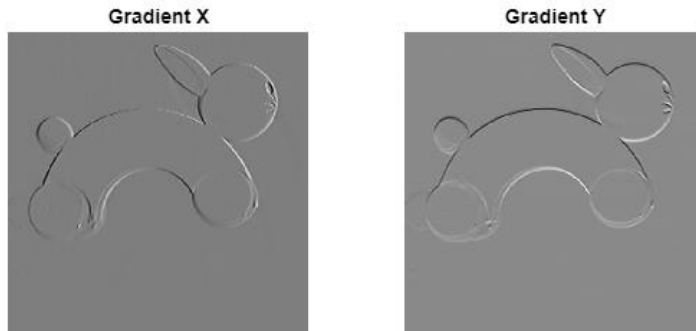
% Observem que només apareixen les positives (blanc-negre), ja que la
% imatge està emmagatzemada com a 'unsigned int'
figure, imshow(Gx,[]), title('Gradient X (Només Transicions Positives)')
```



```
im = double(im);

Gx = imfilter(im, k, 'conv'); % Transicions horitzontals
Gy = imfilter(im, k, 'conv'); % Transicions verticals

% Transicions horitzontals i verticals
subplot(1,2,1),imshow(Gx, []), title('Gradient X')
subplot(1,2,2),imshow(Gy, []), title('Gradient Y')
```



Operadors Tipus Gradient

Lògicament hi ha diferents maneres de discretitzar la operació matemàtica de la derivada. Així que existeixen diversos operadors de tipus gradient, tots ells amb similituds però cadascun amb les seves característiques. Aquí destaquem i utilitzem el de Sobel, però també cal esmentar el de Prewitt.

- **Operador de Prewitt:** redueix la sensibilitat al soroll respecte els 2x2.

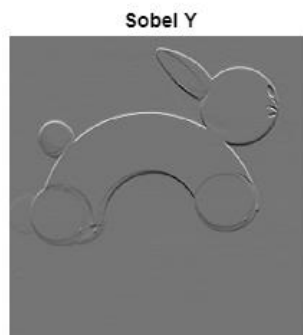
$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Operador de Sobel:** a més de reduir la sensibilitat al soroll dona més pes als píxels directament adjacents (pondera).

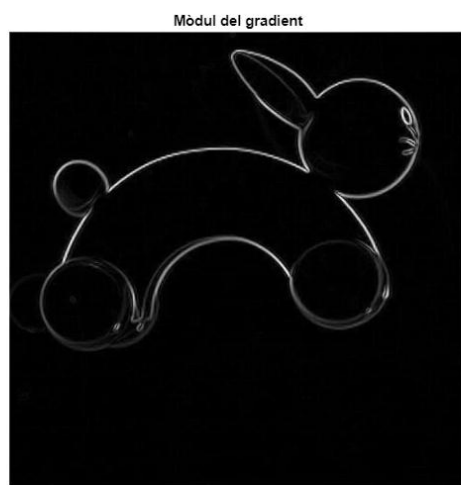
$$G_x = \begin{bmatrix} -1 & 0 & -1 \\ -2 & 0 & -2 \\ -1 & 0 & -1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

```
Sy = fspecial("sobel")/4;
Gy = imfilter(im,Sy,'conv');
Gx = imfilter(im,Sy','conv');

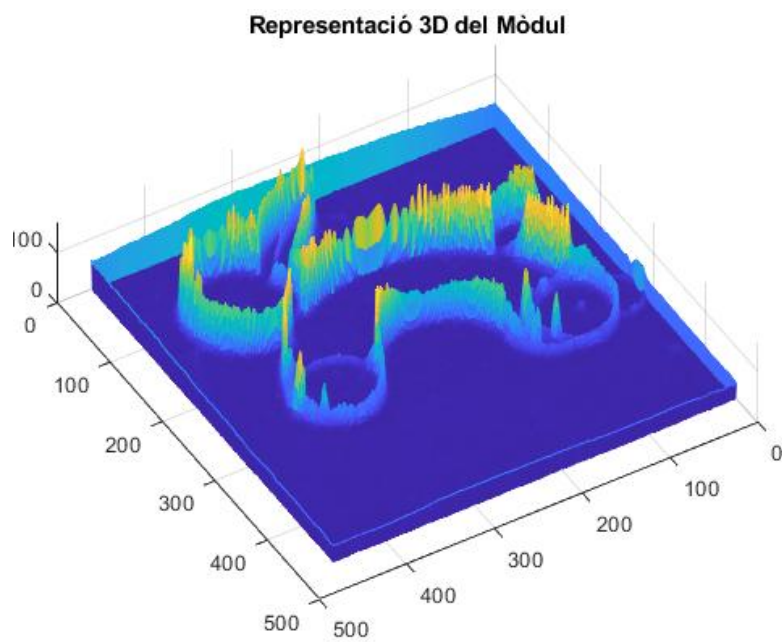
subplot(1,2,1), imshow(Gx,[]),title('Sobel X')
subplot(1,2,2), imshow(Gy,[]),title('Sobel Y')
```



```
mod = sqrt(Gx.^2+Gy.^2);  
figure, imshow(mod,[]),title('Mòdul del gradient')
```

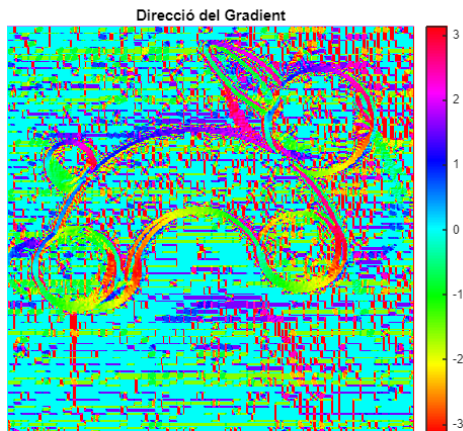


```
figure, mesh(mod), title('Representació 3D del Mòdul')
```



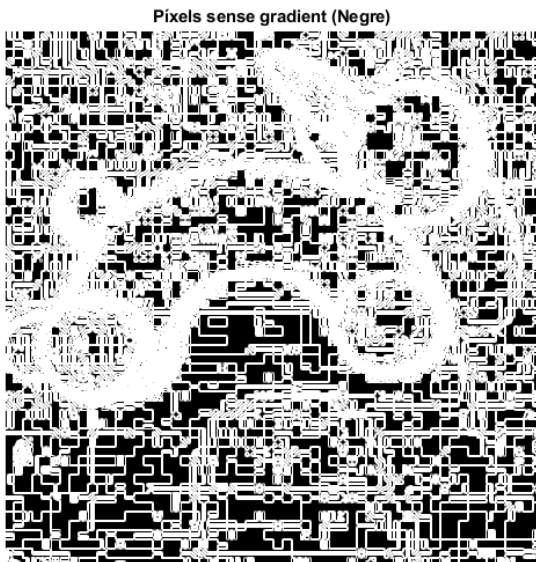

```
dir = atan2(Gy, Gx);
```

```
% En representar la direcció dels Gradients observem que hi ha molt de soroll
figure,imshow(dir,[]),title('Direcció del Gradient')
colormap hsv, colorbar
```



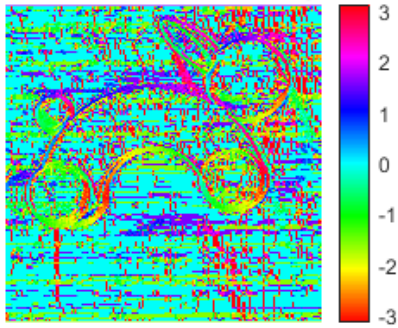
```
mask = mod<4;
```

```
% Hi ha molts valors de la imatge que tenen gradient tot i que haurien de ser plans
figure, imshow(mod), title('Píxels sense gradient (Negre)')
```

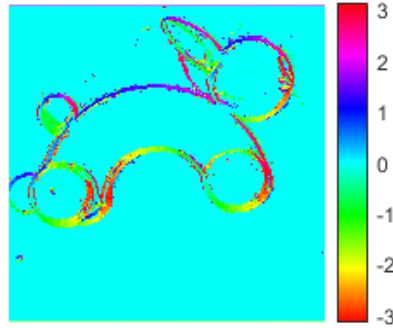


```
dir2 = dir;
dir2(mask) = 0; % Equilibrem la direcció del Gradient
subplot(1,2,1), imshow(dir, []), title('Direcció del Gradient Verge'), colormap hsv,
colorbar
subplot(1,2,2), imshow(dir2,[]), title('Direcció del Gradient Equilibrada'), colormap hsv,
colorbar
```


Direcció del Gradient Verge



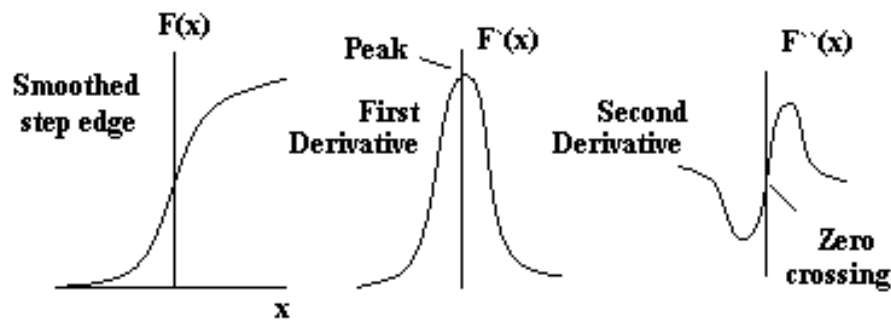
Direcció del Gradient Equilibrada



Operadors de 2a derivada

Fins ara hem aconseguit trobar el gradient de la imatge i veure'n el mòdul i direcció. Però tot i haver-lo trobat i representat gràficament, encara hem de trobar els extrems locals de la funció matemàticament. La representació gràfica que hem vist ara representa tots els extrems locals però també els punts propers al contorn que tot i no ser el màxim/mínim local tenen un gradient considerable. Això es deu a la dificultat d'identificar el punt exacte de la funció que representa el màxim/mínim (el contorn de la figura).

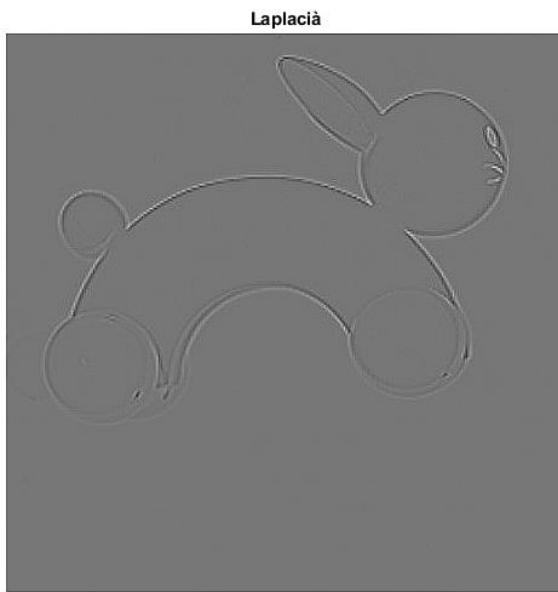
La forma de solucionar-ho és fent la segona derivada de la funció i buscar els passos per zero, ja que els extrems relatius de la funció del Gradient prendran valor 0 quan tornem a derivar.



- **Operador Laplaciana:** podem definir-ne un per treballar amb veïnatge 4 i un altre per veïnatge 8.

$$L4 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad L8 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

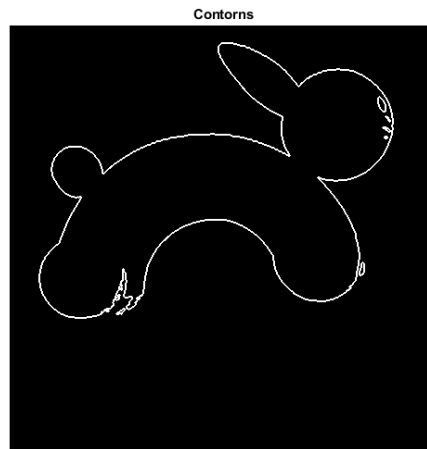
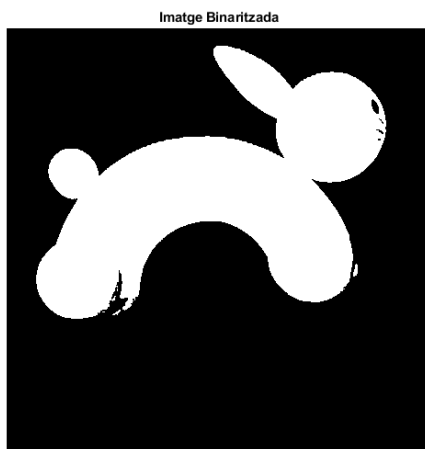
```
kernel = fspecial("laplacian");
lap = imfilter(im,kernel,'conv');
figure,imshow(lap,[]),title('Laplacià')
```



Exercici

Busqueu els passos per 0 de la figura (els contorns). Proveu a binaritzant la imatge.

```
bin = im > 95;
figure, imshow(bin), title('Imatge Binaritzada')
kernel = fspecial("laplacian");
lap = imfilter(bin,kernel,'conv');
figure, imshow(lap,[]), title('Contorns')
```



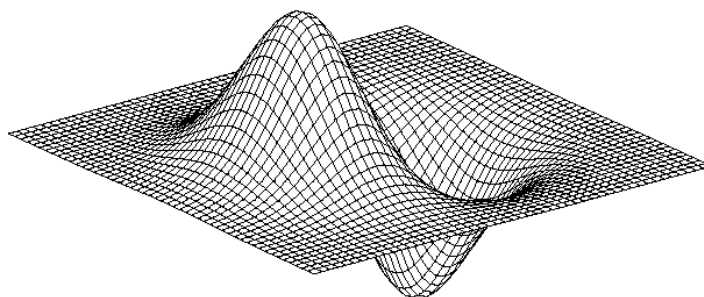
Entrega 7: Operadors mixtes

DoG: Derivative of Gaussian

En moltes ocasions, quan intentem detectar els contorns d'una imatge, veurem que el soroll ens destorba. Això és degut a que les tècniques de detecció de contorns es basen en el gradient, que detecta pujades i baixades sobtades d'intensitat i el soroll és justament això, així que ens detectarà com a contorns molts dels píxels sorollosos. Per aconseguir evitar això, existeixen kernels que combinen operadors de suavitzat i de realçat.

El procés lògic és passar primer l'operador de suavitzat per a reduir el soroll i després el de realçat per a detectar els contorns, però aquesta operació seria molt costosa computacionalment. Així doncs el que es fa és passar una sola finestra de convolució que faci totes dues funcions. Matemàticament aquesta funció correspon a la derivada de la funció Imatge per la funció de suavitzat:

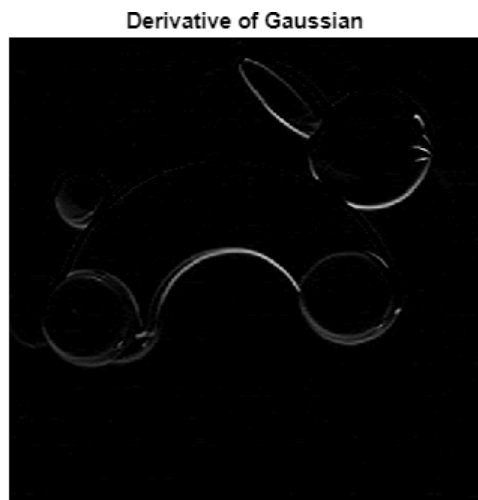
$$\text{DoG} = \frac{\delta}{\delta x} (S * I) \quad \text{on} \quad S = \text{Op. Suavitzat i } I = \text{Imatge}$$



```
im = imread("rabbit.jpg");  
k_gauss = fspecial('gaussian',5,1); % Paràmetres: tipus, tamany, desviació  
DoG = k_gauss(1:3,:)-k_gauss(3:5,:) % Podem observar que s'assembla a Sobel
```

```
DoG = 3x5  
-0.0190    -0.0850    -0.1402    -0.0850    -0.0190  
         0         0         0         0         0  
 0.0190     0.0850     0.1402     0.0850     0.0190
```

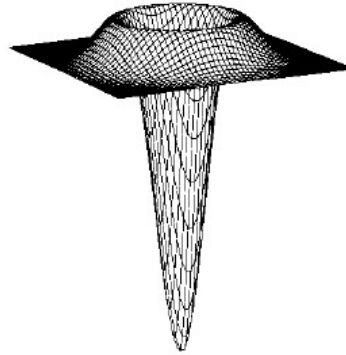
```
res = imfilter(im,DoG,'conv');  
figure, imshow(res,[]), title('Derivative of Gaussian')
```



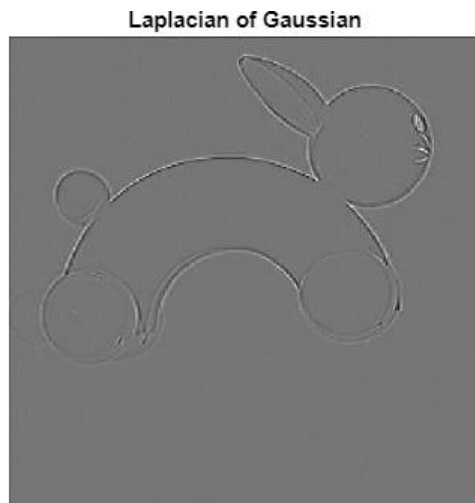
LoG: Laplacian of Gaussian

Anàlogament a la 1a derivada, també hi ha operadors mixtes amb la segona derivada, que filtraran el soroll i buscaran els passos per 0. El raonament matemàtic és el mateix:

$$\text{LoG} = \frac{\delta^2}{\delta x^2}(S * I) \quad \text{on} \quad S = \text{Op. Suavitzat} \text{ i } I = \text{Imatge}$$



```
k_log = fspecial('log');  
res2 = imfilter(double(im),k_log,'conv');  
figure, imshow(res2,[]),title('Laplacian of Gaussian')
```



Exercici Individual 1

Crear Imatge Gaussiana

Crea una imatge sintètica de mida 257x257 que segueixi la següent distribució Gaussiana: $f = \exp\left(\frac{-x^2 + y^2}{2\sigma^2}\right)$ i representeu-la en 3 dimensions.

```
size = 257;

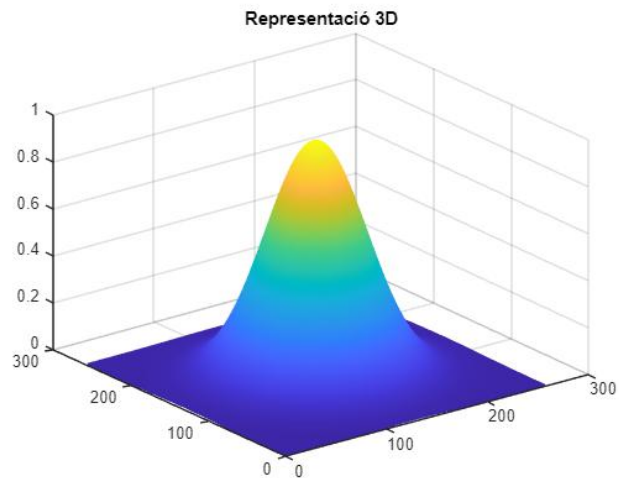
x = zeros(257);
y = zeros(257);

for i = 1:size
    x(:,i) = i-floor(size/2);
    y(i,:) = i-floor(size/2);
end

sigma = 40; % Prenem un valor aribtràri
f = exp(-(x.^2 + y.^2) / (2 * sigma^2)); % Apliquem la funció demanada

im = f / max(f(:)); % Normalitzem

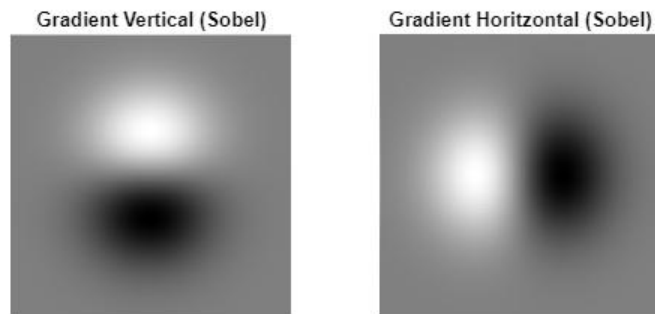
figure, imshow(im), title('Sintetic Image 257x257');
figure, mesh(im), title('Representació 3D')
```



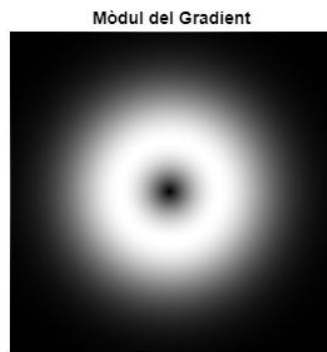
Calcular els Gradients amb Sobel

Aplica un filtre per trobar els gradients de la imatge. Un cop els hakis trobat, troba el seu mòdul i grafica la seva direcció.

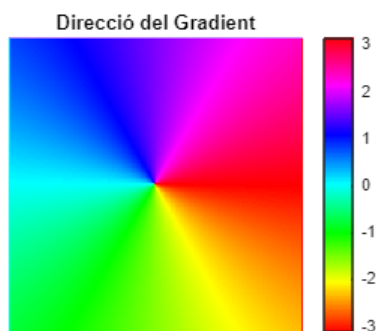
```
sobel_y = fspecial("sobel");  
sobel_x = sobel_y';  
grad_y = imfilter(im,sobel_y,'conv');  
subplot(1,2,1), imshow(grad_y,[]),title('Gradient Vertical (Sobel)')  
  
grad_x = imfilter(im,sobel_x,'conv');  
subplot(1,2,2), imshow(grad_x,[]),title('Gradient Horitzontal (Sobel)')
```



```
modul = sqrt(grad_x.^2 + grad_y.^2);  
figure, imshow(modul, []), title('Mòdul del Gradient')
```



```
direccio = atan2(grad_y, grad_x);  
figure, imshow(direccio, []), title('Direcció del Gradient')  
colorbar, colormap hsv
```

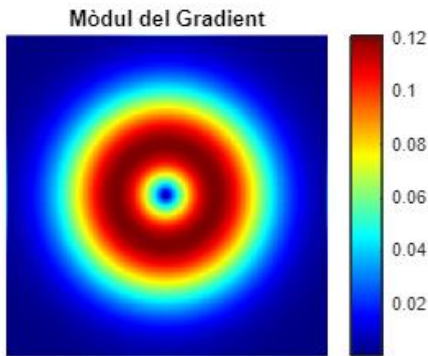


Representar el Mòdul del Gradient amb un Mapa de Color

Representa el mòdul del Gradient amb un mapa de color (utilitza el colormap jet) i analitza el resultat traient conclusions.

```
% Observem que podem trobar el gradient més elevat en un cercle proper al centre  
% però sense arribar a tocar-lo. Això es deu a que a la vora la funció encara és  
% molt plana i conforme ens apropem al centre va agafant inclinació. Però un cop  
% ens hem apropat prou al centre torna a ser pla, ja que equival al pic de la imatge.
```

```
figure, imshow(modul, []), title('Mòdul del Gradient')  
colorbar, colormap jet
```



Calcular Histograma de la imatge d'Orientacions

Genera i representa un histograma amb les direccions dels gradients d la imatge i analitza els resultats. En cas de no obtenir els resultats esperats, analitza la imatge i raona perquè has obtingut uns resultats equívocs.

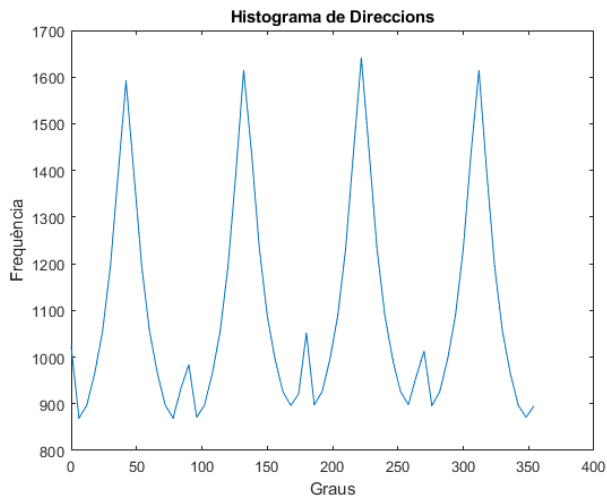
```
% Escalem valors: [-pi, pi] -> [0, 360]  
dir_grads = rad2deg(direccio); % [-pi, pi] -> [-180, 180]  
dir_grads(dir_grads < 0) = dir_grads(dir_grads < 0) + 360; % [-180,180] -> [0,360]  
  
figure, imshow(dir_grads), title('Direcció del Gradient traslladada a 360 valors')
```

Direcció del Gradient traslladada a 360 valors



```
% Discretitzarem la informació: 6 graus/bin  
bins = 60;  
edges = linspace(0, 360, bins+1);  
histo = histcounts(direccio_grads, edges);
```

```
figure, plot(edges(1:end-1), histo), title('Histograma de Direccions'), xlabel('Graus'),  
ylabel('Frequència');
```



% Tot i que en treballar amb una campana gaussiana cabria esperar un histograma totalment
% uniforme, podem observar 4 pics que sobresurten. Això es deu a que, tot i que a l'interior
% de la imatge estiguem tractant la campana, la imatge segueix sent quadrada, per això,
% aquells píxels que queden dins de la imatge però fora de la campana ens esbiaixen les dades,
% provocant aquells pics en els graus 45, 135, 225 i 315.

Preprocessat d'Imatge: Operacions Morfològiques

Fins ara hem estat treballant amb processat lineal d'imatges, que donen un enfoc molt proper al càlcul matemàtic. Ara introduïrem noves eines de processat no lineal, que en comptes de tenir com a element principal les 'finestres de convolució', hi jugaran aquest paper els 'elements estructurants'. D'igual manera que els productes de convolució tenien un suport i raonament matemàtic al darrere, aquestes operacions atenen a l'àlgebra no lineal i a les teories de conjunts (treballarem molt amb conjunts de punts i les seves formes i connectivitat).

Aquestes tècniques de processat tenen moltes aplicacions directes a indústries com la geologia, la inspecció industrial o les anàlisis mèdiques microscòpiques.

Un exemple molt bàsic per a comprendre com intervé la Teoria de Conjunts és el filtre de mediana que ja vam utilitzar a l'Entrega 4. El filtre de mediana agafa una sèrie de píxels de la imatge determinats per l'element estructurant i els endreça de menor a major, després, com és lògic, n'agafa la mediana.



Un element estructural és un conjunt de píxels amb valor 1 als que podem assignar qualsevol forma i mida segons ens convingui. Aquests elements ens ajudaran a fer moltes operacions de les que s'expliquen a continuació.

Anàlogament a les finestres de convolució, els "desplaçarem" per tota la imatge i segons la operació que estiguem fent modificarem els píxels adients d'una manera o altra. La seva forma i mida són clau a l'hora de modificar la imatge, per això és important escollir-los bé.

Més endavant s'expliquen totes les operacions més detalladament, però en termes generals:

Dilatació/Erosió:

- **Dilatació:** engruixa tots els elements blanc de la imatge.
- **Erosió:** engruixa tots els elements negres de la imatge.

Open/Close:

- **Open:** elimina petites estructures blanques de la imatge i manté les demés sense modificar-les.
- **Close:** elimina petites estructures negres de la imatge i manté les demés sense modificar-les.

Reconstrucció: manté els elements que ens interessen sense modificar-los i elimina els que no.

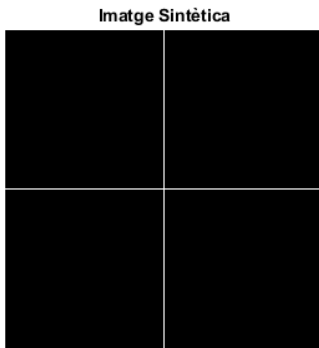


Entrega 8: Dilatacions, erosions i transformades de distància

Dilatació

En lliscar l'element estructurant per sobre de la imatge, sempre que algun píxel d'aquest es solapi amb un píxel en blanc, el píxel sobre el estigui situat el centre de l'EE en aquell moment es posarà a 1 (blanc).

```
im = zeros(256);  
im(128,:) = 1;  
im(:,128) = 1;  
figure, imshow(im), title("Imatge Sintètica")
```



```
% L'element estructurant serveix per a determinar el gruix de dilatació  
ee = [1,1,1]; % mida senar
```

```
% Implementem la dilatació manualment
```

```
[fils, cols] = size(im);  
[n, m] = size(ee);
```

```
im_res = false(fils, cols);
```

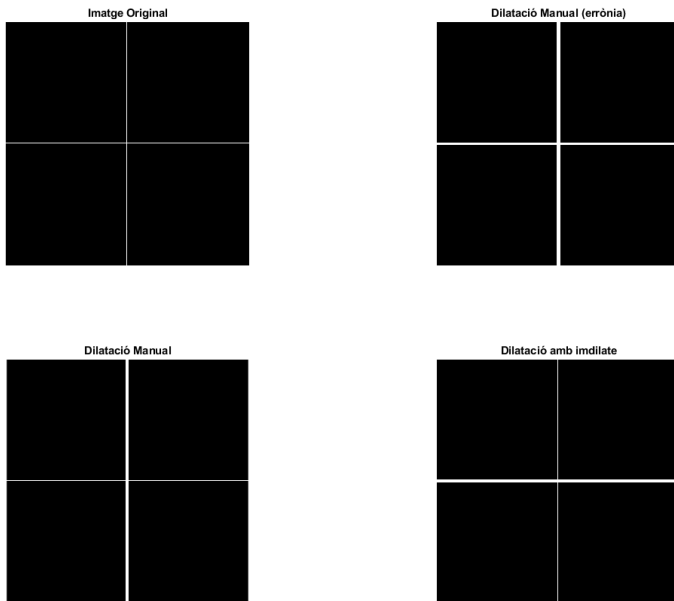
```
for i = 1:fils  
    for j = 1:cols  
        if im(i, j) == 1  
            for k = i - fix(n/2): i + ceil(n/2)  
                for l = j - fix(m/2): j + ceil(m/2)  
                    if k > 0 && l > 0 && k <= fils && l <= cols  
                        im_res(k, l) = 1;  
                    end  
                end  
            end  
        end  
    end  
end
```

```
% Implementació del Profe (s'implementa fent una OR)
```

```
res = false(128);  
res = im(:,2:end-1) | im(:,1:end-2) | im(:,3:end);
```

```
% Operació ja implementada
res2 = imdilate(im,ee'); % trasposem l'ee per comparar el resultat

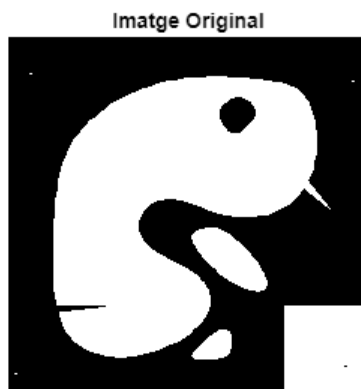
subplot(2,2,1), imshow(im), title('Imatge Original')
subplot(2,2,2), imshow(im_res), title('Dilatació Manual (errònia)')
subplot(2,2,3), imshow(res),title('Dilatació Manual')
subplot(2,2,4), imshow(res2), title('Dilatació amb imdilate')
```



Erosió

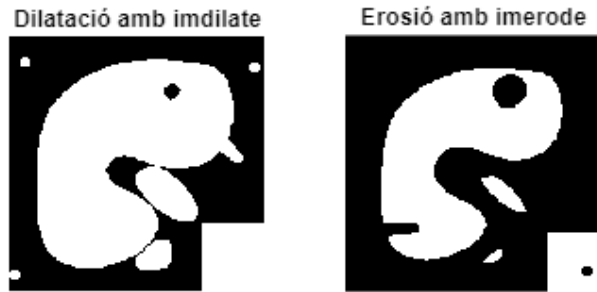
L'erosió és l'operació complementària de la dilatació. Mentre que la ja explicada s'encarrega d'engruixar les zones blanques d'una imatge binària, l'erosió expandeix les negres (aprima les blanques). Hem vist que la dilatació es pot implementar amb una OR lògica, així que l'erosió es pot implementar amb una AND.

```
im = imread('blob.tif');
figure, imshow(im), title('Imatge Original')
```



```
% Creem, un EE en forma de disc de radi 5 píxels
ee = strel('disk',5);
dil = imdilate(im,ee);
ero = imerode(im,ee);
subplot(1,2,1), imshow(dil), title('Dilatació amb imdilate')
```

```
subplot(1,2,2), imshow(ero), title('Erosió amb imerode')
```

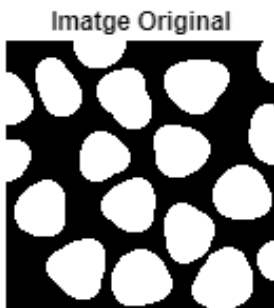


Propietats de la dilatació i la erosió

- **Són operacions complementàries:** dilatar blanc = erosionar negre i viceversa
- **Són creixents:** si la imatge A és una subimatge de B, el resultat d'aplicar una dilatació/erosió a A també serà subimatge del resultat d'aplicar el mateix procediment a B.
- **Són compostables:** $[3 \times 3] = [1 \times 3] \rightarrow [3 \times 1]$

Les operacions de dilatació i erosió combinades també ens poden servir per a trobar els passos per zero i els contorns de les figures d'una imatge.

```
im = imread('blob3.tif');
figure, imshow(im), title('Imatge Original')
```

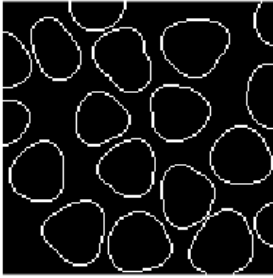


```
ee = strel('disk', 1);

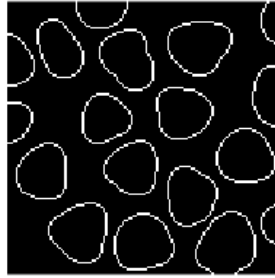
dil = imdilate(im, ee);
ero = imerode(im, ee);
subplot(1,2,1), imshow(dil), title('Imatge Dilatada')
subplot(1,2,2), imshow(ero), title('Imatge Erosionada')

cext = imsubtract(dil, im);
cint = imsubtract(im, ero);
subplot(1,2,1), imshow(cext), title('Contorn Extern')
subplot(1,2,2), imshow(cint), title('Contorn Intern')
```

Contorn Extern



Contorn Intern

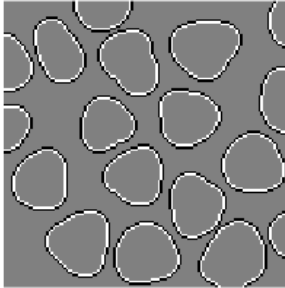


```
% Aproximació no lineal del Laplacià
```

```
lapmorf = imsubtract(double(cint), double(cext));
```

```
figure, imshow(lapmorf, []), title('Laplacià Morfològic')
```

Laplacià Morfològic



```
% Podem barrejar operadors lineals i no-lineals per a obtenir resultats més  
% precisos, aquí utilitzem un filtre Laplacià i una dilatació per a trobar les vores.
```

```
klap = fspecial('laplacian');
```

```
lap = imfilter(double(im), klap);
```

```
ee = strel('disk', 1);
```

```
pos = lap>0;
```

```
neg = lap<0;
```

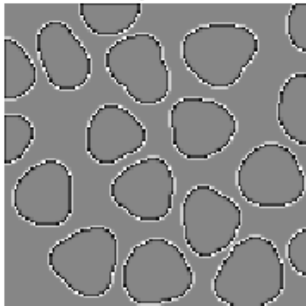
```
posdil = imdilate(pos, ee);
```

```
contorns = posdil&neg;
```

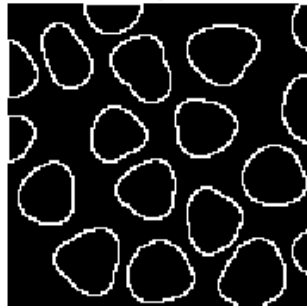
```
subplot(1,2,1), imshow(lap,[]), title('Laplacià')
```

```
subplot(1,2,2), imshow(contorns), title('Passos per Zero')
```

Laplacià



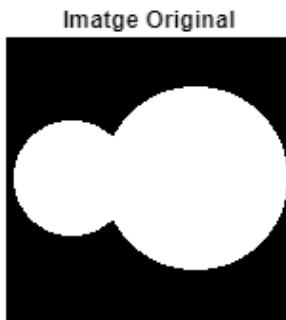
Passos per Zero



Transformada de Distància

Resulten en imatges de nivells de gris. Cada nivell de gris correspondrà a la seva distància amb la vora de la figura.

```
im=imread('touchcell.tif');  
figure, imshow(im), title('Imatge Original')
```

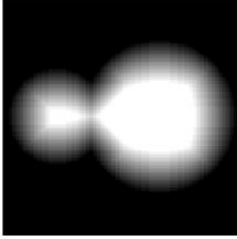


```
tdist = double(im);  
  
ero = imerode(im,ee);  
tdist = tdist + ero;  
% Observem que els contorns han prè una nova tonalitat gris  
figure, imshow(tdist, []), title('Primera Iteració')
```

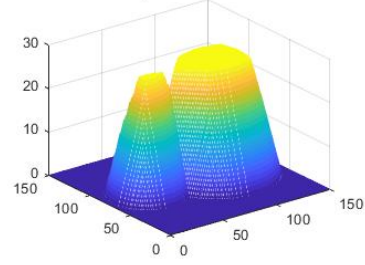


```
for i=1:25  
    ero = imerode(ero,ee);  
    tdist = tdist+ero;  
end  
figure,subplot(2,2,1), imshow(tdist, []), title('26a iteracio')  
subplot(2,2,2), mesh(tdist), title('Representació 3D')  
  
td = bwdist(~im);  
subplot(2,2,3), imshow(td,[]), title('Transformada de Distància')  
subplot(2,2,4), mesh(td), title('Respresentació 3D')
```

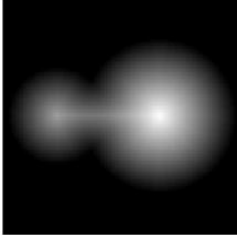
26a iteracio



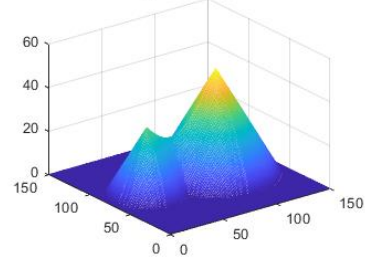
Representació 3D



Transformada de Distància



Representació 3D



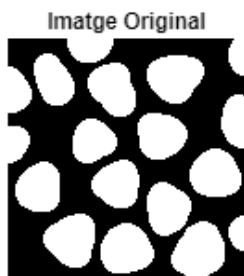
Entrega 9: Open/Close, reconstruccions i dilatacions condicionals.

Dilatació Condicional i Reconstruccions

Les reconstruccions ens permeten seleccionar els elements que ens interessin d'una imatge binària mitjançant dilatacions, o dit d'una altra manera, ens permeten eliminar els que no ens interessin.

El procés consisteix a generar una imatge amb unes marques col·locades estratègicament i dilatar aquesta 'imatge de marques' amb un element estructurant. Un cop dilatada s'intersecta el resultat amb la imatge original, així romanen en blanc només els píxels que ja eren blancs a l'original. Si repetim aquest procés diverses vegades, acabem obtenint una imatge on les úniques estructures que hi haurà seran les que, si solapéssim la Imatge de Marques amb la Original, estarien en contacte amb alguna de les marques.

```
im=imread('blob3.tif');  
imshow(im), title('Imatge Original')
```



```
marker = true(size(im));  
marker(2:end-1,2:end-1) = 0; % posem a 0 tots els píxels menys els de les vores  
figure, imshow(marker), title('Markers')
```



```
% Iteració 1  
ee = strel('disk',1);  
dil = imdilate(marker, ee);  
dilc = dil & im; % intersectem la imatge original i la dilatada  
subplot(1,3,1), imshow(dilc), title('Dilatació condicional 1')  
  
% Reconstrucció de la imatge amb la dilatació condicional  
n = 7;  
for i = 1:n  
    dilc=imdilate(dilc,ee)&im;  
end  
subplot(1,3,2), imshow(dilc), title('Dilatació condicional N')
```



```
% Funció de Reconstrucció ja implementada
rec = imreconstruct(marker, im);
subplot(1,3,3), imshow(rec), title('Reconstrucció de la Imatge')
```



```
% Per a treballar i treure estadístiques reals és important obviar les
% cèl·lules no completes
res = imsubtract(im,rec);
figure, imshow(res), title('Imatge Resultant amb Cèl·lules Senceres')
```

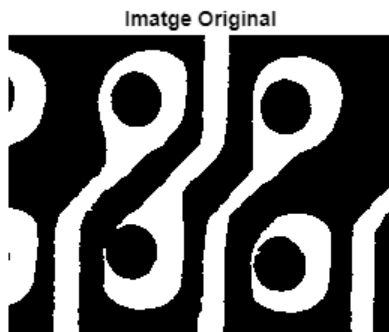
Imatge Resultant amb Cèl·lules Senceres



Tancar Forats

A l'hora de treure estadístiques o fer segons quines anàlisis, els forats que pugui haver dins de la imatge ens poden esbiaixar les dades. Així doncs hem de saber com eliminar-los.

```
im=imread('pcbholes.tif');
figure, imshow(im), title('Imatge Original')
```



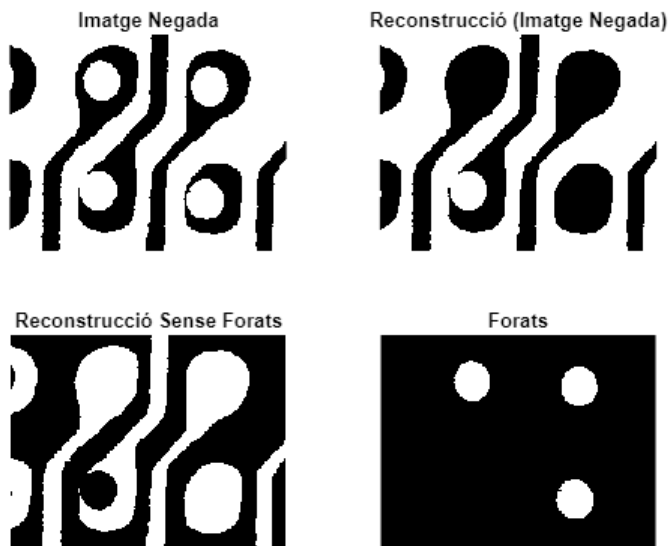
```
% Creem el EE i el marker
ee = strel('disk', 1);
marker = true(size(im));
marker(2:end-1,2:end-1) = 0;
```

```

neg = ~im;
figure
subplot(2,2,1), imshow(neg), title('Imatge Negada')
rec = imreconstruct(marker, neg);
subplot(2,2,2), imshow(rec), title('Reconstrucció (Imatge Negada)')
res = ~rec;
subplot(2,2,3), imshow(res), title('Reconstrucció Sense Forats')

forats = imsubtract(res,im);
subplot(2,2,4), imshow(forats), title('Forats')

```



Obtenir imatge de marques a partir de la Imatge Original

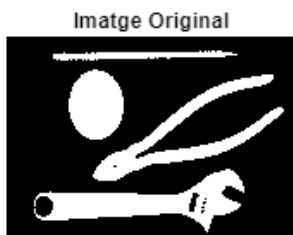
Per a aconseguir això, s'ha de ser original i a cada imatge s'ha trobar una forma adient d'identificar únicament el que ens interessa.

Exercici: reconstruir el disc i la clau anglesa i eliminar els demás estris mitjançant operacions de processat morfològic.

```

im = imread('tools.tif');
figure, imshow(im), title('Imatge Original')

```

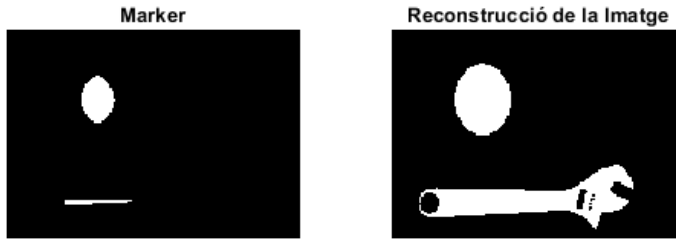


```

% Busquem un EE suficientment gran per eliminar les altres dues eines però
% no el disc ni la clau anglesa
ee = strel('disk', 7);
marker = imerode(im, ee);
subplot(1,2,1), imshow(marker), title('Marker')

```

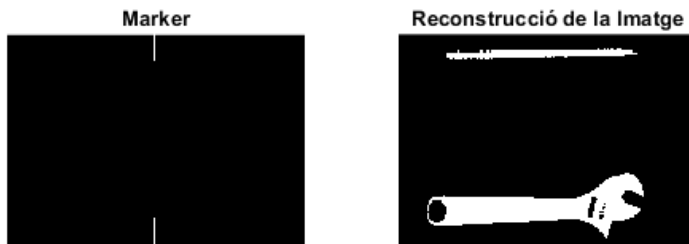
```
rec = imreconstruct(marker,im);
subplot(1,2,2), imshow(rec), title('Reconstrucció de la Imatge')
```



Exercici: reconstruir el llapis i la clau anglesa i eliminar els demás estris mitjançant operacions de processat morfològic.

```
[f, c] = size(im);
marker = false(size(im));
marker(1:15, c/2) = 1;
marker(end-15:end,c/2) = 1;
figure, subplot(1,2,1), imshow(marker), title('Marker')

rec = imreconstruct(marker,im);
subplot(1,2,2), imshow(rec), title('Reconstrucció de la Imatge')
```



Open i Close

Per a eliminar elements molt petits o "ponts" no desitjats entre elements de la imatge podem fer una erosió seguida d'una dilatació. Hi haurà estructures que desapareixeran amb l'erosió i per tant quan tornem a dilatar no reapareixeran.

Open → Elimina petites estructures blanques de la imatge (més petites que l'element estructural).

Close → Elimina petites estructures negres de la imatge (és la operació complementaria de l'Open).

Per a comprendre com funciona una d'aquestes operacions podem pensar en una 'roomba', quan es deixa anar en una habitació renta tot menys les cantonades, ja que no hi arriba. En aquesta analogia, la roomba és el EE, que renta tot allò que és més petit que ell.

```
im = imread('blob.tif');
ee = strel('disk', 5);
figure, imshow(im), title('Imatge Original')
```

Imatge Original



```
op = imopen(im,ee); % op = imdilate(imerode(im,ee),ee);
cl = imclose(im,ee); % cl = imerode(imdilate(im,ee),ee);
subplot(1,2,1), imshow(op), title('Open')
subplot(1,2,2), imshow(cl), title('Close')
```

Open



Close



Exercici Open

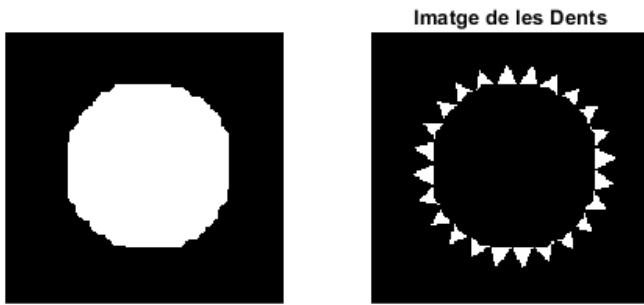
Control de qualitat d'engranatges: una empresa vol fer un control del desgast de les dents d'uns engranatges. Identifica les dents per a poder fer anàlisis posteriors.

```
im = imread('gear.tif');
figure, imshow(im), title('Imatge Original');
```

Imatge Original



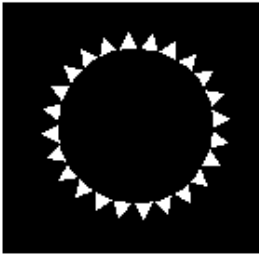
```
% Busquem un EE que càpiga a l'engranatge i no a les dents
ee = strel('disk', 10);
op = imopen(im, ee);
res = abs(im-op);
subplot(1,2,1), imshow(op)
subplot(1,2,2), imshow(res), title('Imatge de les Dents')
```



% Observem que té forma poligonal. Això es deu a que strel() ens retorna un
% octàgon per optimitzar però podem forçar-la a que ens doni un cercle.

```
ee2 = strel('disk',20,0);
op2 = imopen(im,ee2);
res2 = abs(im-op2);
figure, imshow(res2), title('Imatge de les Dents (EE circular)')
```

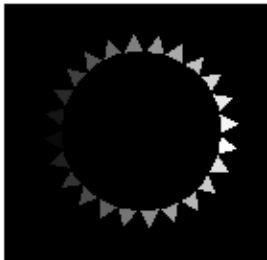
Imatge de les Dents (EE circular)



% Etiquetar ens permet identificar les diferents estructures connexes per a
% una posterior anàlisi de la imatge.

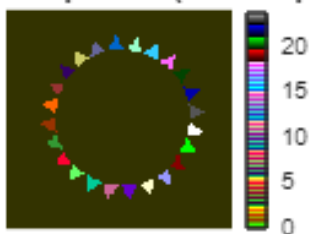
```
eti = bwlabel(res2,8); % El 8 indica el nivell de connectivitat
figure, imshow(eti,[]), title('Imatge Etiquetada')
```

Imatge Etiquetada



```
figure, imshow(eti,[]), title('Imatge Etiquetada (colormap)')
colorbar, colormap colorcube
```

Imatge Etiquetada (colormap)



```
Dades = regionprops(eti, 'Area');  
Arees = [Dades.Area];  
size(Arees) % Equival al número de dents de l'engranatge
```

```
ans = 1x2  
1    24
```

Entrega 10: Esquelets

SKIZ (Skeleton by Influence Zones)

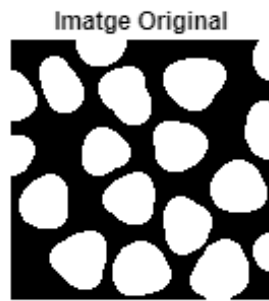
Serveixen per a tractar imatges manipulant menys dades. Perdem informació de la figura, però mantenim la connectivitat i la topologia. Per a saber com es fan, consultar a internet l'analogia 'grassfire'.

Esquelet del fons: equival als píxels equidistants de les diferents components connexes.

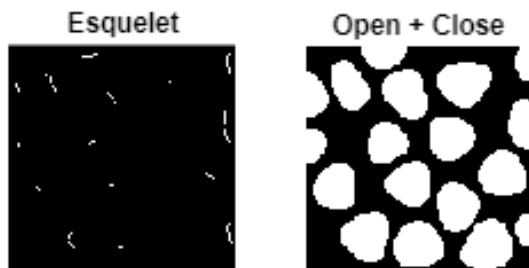
Zona d'influència: conjunt de píxels que estan més a prop d'una component connexa que d'una altra.

Un SKIZ es pot utilitzar com a Marker del fons d'una imatge.

```
im = imread('blob3.tif');  
figure, imshow(im), title('Imatge Original')
```



```
sk = bwskel(im);  
subplot(1,2,1), imshow(sk), title('Esquelet')  
subplot(1,2,2), imshow(imfuse(sk, im)), title('Esquelet + Imatge Og')  
  
ee = strel('disk', 1);  
op = imopen(im, ee);  
cl = imclose(op, ee);  
subplot(1,2,2), imshow(cl), title('Open + Close')
```



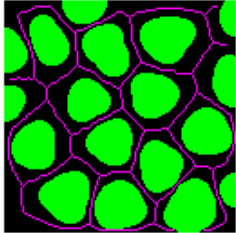
```
sk2 = bwskel(cl);  
% En aquest cas han sortit igual pq la imatge és molt petita, en una altra  
% imatge els 2 esquelets haurien de canviar una mica  
figure, imshow(sk2), title('Esquelet de la Imatge Filtrada')
```

Esquelet de la Imatge Filtrada



```
skiz = bwskel(~im);  
figure, imshow(imfuse(im,skiz)), title('SKIZ')
```

SKIZ

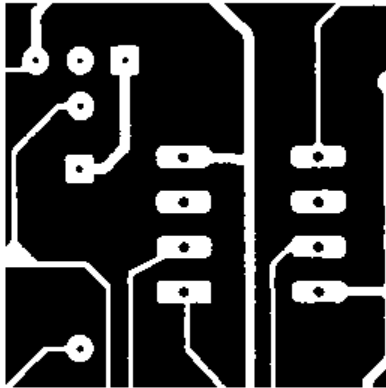


Exercici

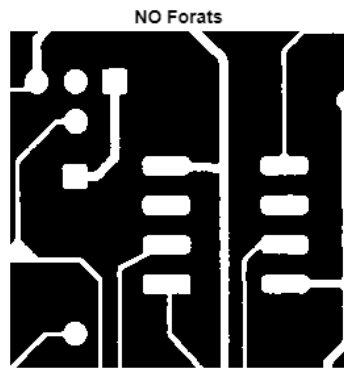
Una fàbrica de circuits ens demana fer un control de qualitat. Per a això hem d'identificar les diferents components del circuit i agrupar-les per tipus.

```
im = imread('pcb1bin.tif');  
figure, imshow(im), title('Imatge Original')
```

Imatge Original



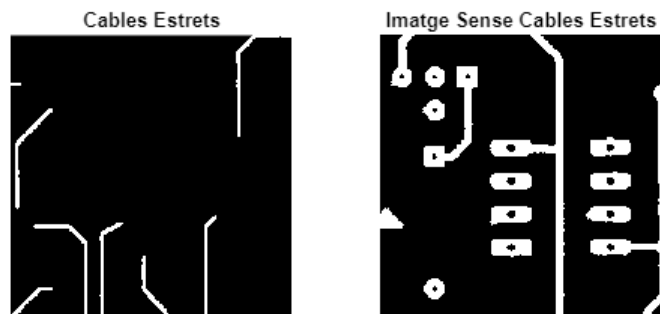
```
% Eliminar els Forats  
marker = true(size(im));  
marker(2:end-1, 2:end-1) = 0;  
  
neg = ~im;  
rec = imreconstruct(marker, neg);  
no_holes = ~rec;  
holes = imsubtract(no_holes, im);  
figure, imshow(no_holes), title('NO Forats')
```

```
% Identificar els cables estrets
ee = strel('disk', 3);
no_small_wires = imopen(no_holes, ee);
small_wires = abs(no_holes - no_small_wires);

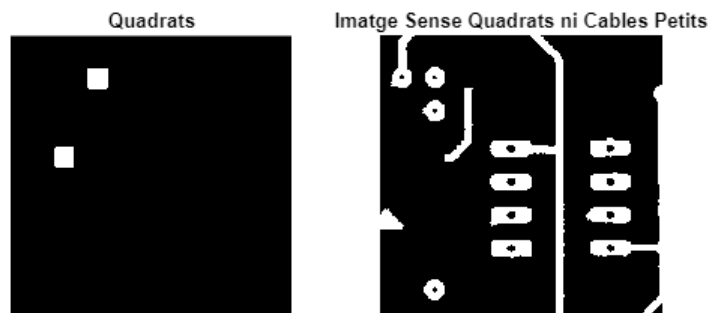
ee = strel('disk', 1);
small_wires = imopen(small_wires, ee); % eliminem les traces que quedin
no_small_wires = im-small_wires;

figure, subplot(1,2,1), imshow(small_wires), title('Cables Estrets')
subplot(1,2,2), imshow(no_small_wires), title('Imatge Sense Cables Estrets')
```



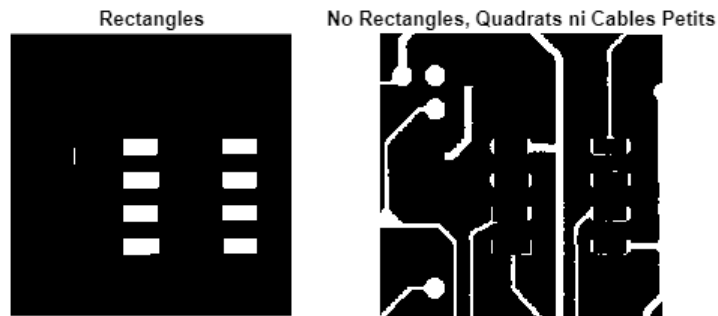
```
% Identificar Quadrats
ee = ones(17);
squares = imopen(no_holes, ee);
no_sq_sw = no_small_wires - squares;

figure, subplot(1,2,1), imshow(squares), title('Quadrats')
subplot(1,2,2), imshow(no_sq_sw), title('Imatge Sense Quadrats ni Cables Petits')
```



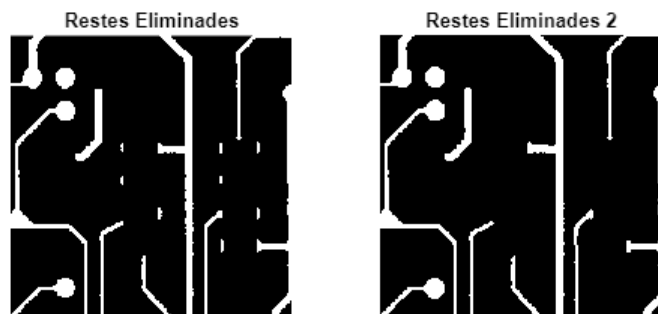
% Identificar Rectangles

```
ee = ones(14);  
rectangles = imopen(no_holes, ee);  
rectangles = imreconstruct(logical(holes), rectangles) - squares;  
no_sq_sw_rect = abs(no_holes - rectangles) - squares;  
  
figure, subplot(1,2,1), imshow(rectangles), title('Rectangles')  
subplot(1,2,2), imshow(no_sq_sw_rect), title('No Rectangles, Quadrats ni Cables Petits')
```



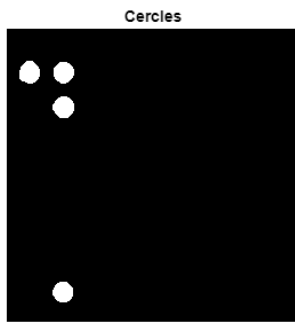
% Fem un Open per eliminar restes

```
ee = strel('disk', 1);  
no_sq_sw_rect = imopen(no_sq_sw_rect, ee);  
figure, subplot(1,2,1), imshow(no_sq_sw_rect), title('Restes Eliminades')  
  
marker = ones(size(im));  
marker(2:end-1, 2:end-1) = 0;  
aux = imreconstruct(marker, no_sq_sw_rect);  
  
ee = strel('disk', 3);  
aux2 = abs(no_sq_sw_rect - aux);  
aux2 = imopen(aux2, ee);  
  
no_sq_sw_rect = aux + aux2;  
subplot(1,2,2), imshow(no_sq_sw_rect), title('Restes Eliminades 2')
```

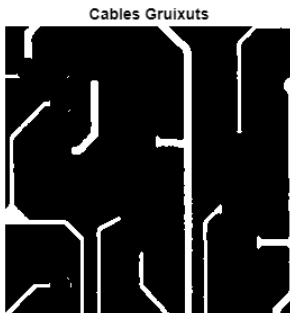


% Identificar Cercles

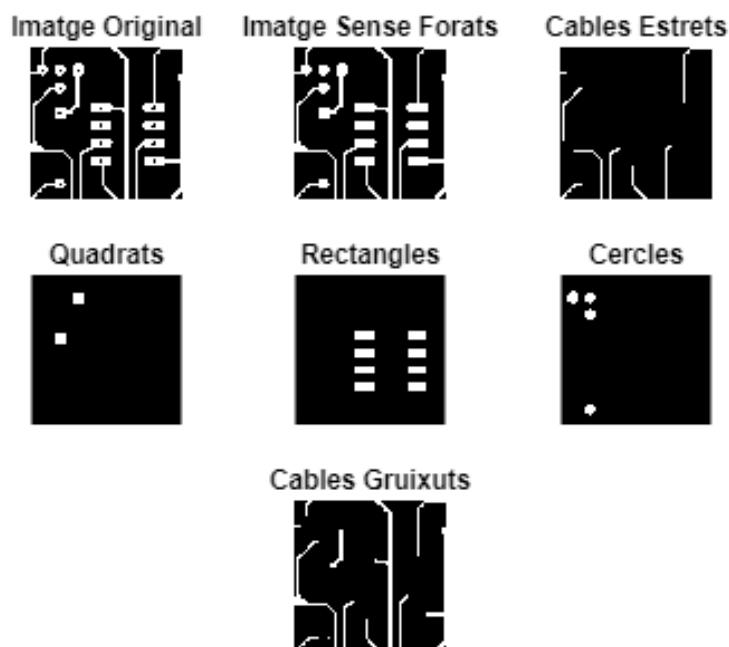
```
ee = strel('disk', 5);  
circles = imopen(no_sq_sw_rect, ee);  
circles = imreconstruct(holes, circles);  
figure, imshow(circles), title('Cercles')
```



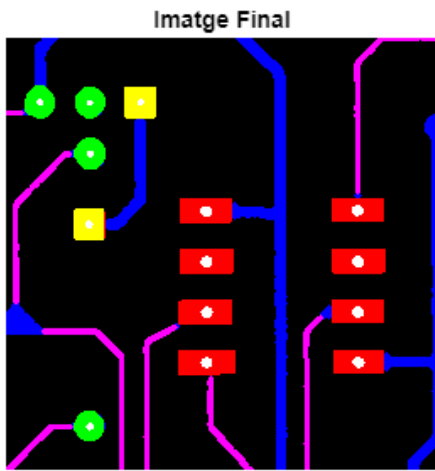
```
big_wires = abs(no_sq_sw_rect - circles);
figure, imshow(big_wires), title('Cables Gruixuts')
```



```
subplot(3,3,1), imshow(im), title('Imatge Original')
subplot(3,3,2), imshow(no_holes), title('Imatge Sense Forats')
subplot(3,3,3), imshow(small_wires), title('Cables Estrets')
subplot(3,3,4), imshow(squares), title('Quadrats')
subplot(3,3,5), imshow(rectangles), title('Rectangles')
subplot(3,3,6), imshow(circles), title('Cercles')
subplot(3,3,8), imshow(big_wires), title('Cables Gruixuts')
```



```
final1 = small_wires + rectangles + squares + holes;  
final2 = circles + squares + holes;  
rgb = cat(3, final1, final2, big_wires + holes);  
figure, imshow(rgb), title('Imatge Final')
```



Entrega 11: Operacions morfològiques multinivell

La idea és descompondre la imatge per llindars, és a dir tractar cada nivell de gris com una imatge binària diferent.

Open/Close Multinivell

```
im = imread('n2538.tif');  
figure, imshow(im), title('Imatge Original')
```

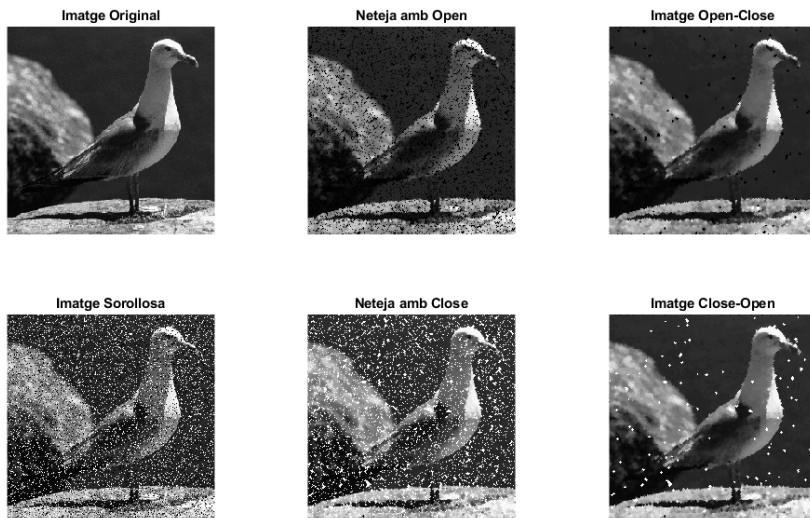


```
ee = strel('disk', 3);  
dil = imdilate(im, ee);  
ero = imerode(im, ee);  
cl = imclose(im, ee);  
op = imopen(im, ee);  
  
% Observem com afecten aquestes operacions a una imatge multinivell  
subplot(2,2,1), imshow(dil), title('Dilatació')  
subplot(2,2,2), imshow(ero), title('Erosió')  
subplot(2,2,3), imshow(cl), title('Close')  
subplot(2,2,4), imshow(op), title('Open')
```

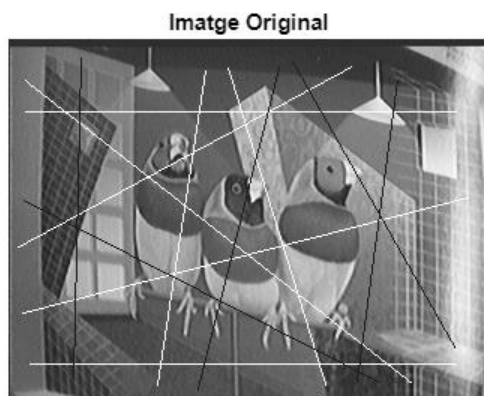


```
im = imread('gull.tif');  
imn = imnoise(im, 'salt & pepper', 0.2);  
  
ee = strel('disk', 1);  
imcl = imclose(imn, ee);  
imop = imopen(imn, ee);  
imopcl = imclose(imop, ee);  
imclop = imopen(imcl, ee);
```

```
figure % Observem que el resultat de Open-Close i Close-Open és diferent
subplot(3,2,1), imshow(im), title("Imatge Original")
subplot(3,2,2), imshow(imn), title("Imatge Sorollosa")
subplot(3,2,3), imshow(imop), title("Neteja amb Open")
subplot(3,2,4), imshow(imcl), title("Neteja amb Close")
subplot(3,2,5), imshow(imopcl), title("Imatge Open-Close")
subplot(3,2,6), imshow(imclop), title("Imatge Close-Open")
```

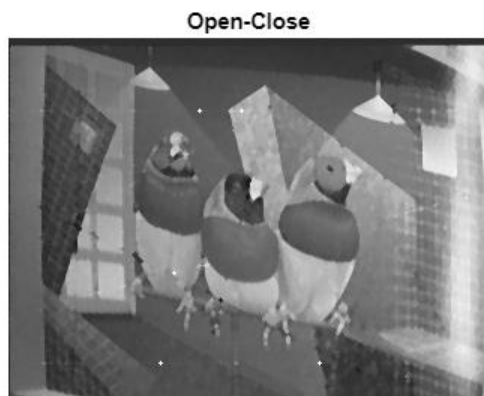


```
im = imread('Birds.tif');
figure, imshow(im), title('Imatge Original')
```



```
ee = strel('disk', 1);
op = imopen(im, ee); % Eliminem petites estructures blanques
opcl = imclose(op, ee); % Eliminem petites estructures negres

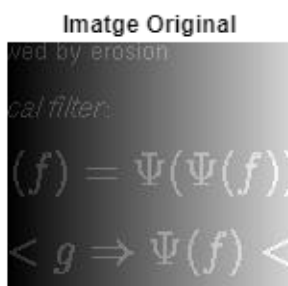
% Obtenim un resultat similar al que ens hauria donat un filtre de mediana
figure, imshow(opcl), title('Open-Close')
```



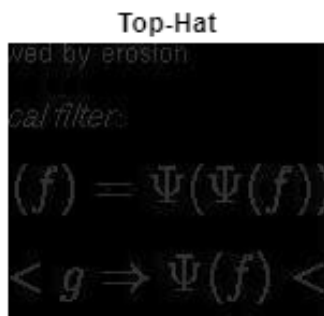
El residu de l'Open són les petites estructures clares que elimina, quan tractem amb imatges multinivell l'anomenem Top-Hat. En el cas del Close, el residu seran les estructures negres que elimina i l'anomenarem Bottom-Hat

En moltes ocasions ens trobarem amb imatges que tenen una il·luminació no uniforme, això provoca que la detecció de vores i la binarització es tornin processos complicats si tractem de fer-los amb les tècniques habituals. Una possible solució és fer un open i quedar-nos amb el residu (Top-Hat).

```
im = imread('nshadow.tif');
figure, imshow(im), title('Imatge Original')
```

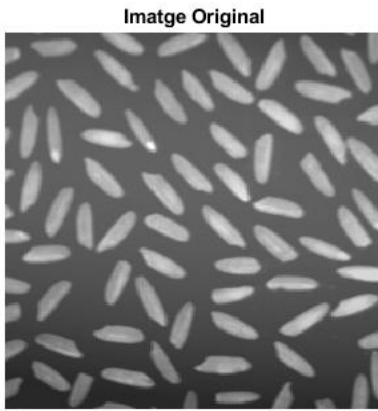


```
ee = strel('disk', 5);
op = imopen(im, ee);
th = abs(im-op);
% th = imtophat(im, ee); % equivalent
figure, imshow(th), title('Top-Hat')
```



Reconstruccions Multinivell

```
im = imread('arros.tif');  
figure, imshow(im), title('Imatge Original')
```

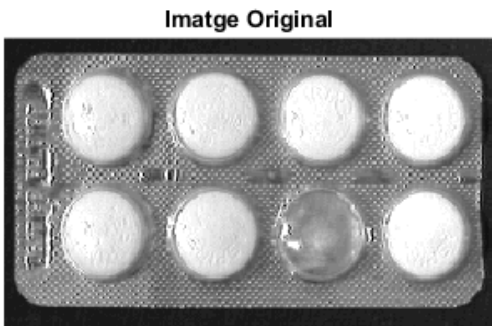


```
marker = im;  
marker(2:end-1, 2:end-1) = 0;  
rec = imreconstruct(marker, im);  
sub = abs(im-rec);  
  
figure  
subplot(1,2,1), imshow(rec), title("Grans d'Arros Partits")  
subplot(1,2,2), imshow(sub), title("Grans d'Arros Sencers")
```

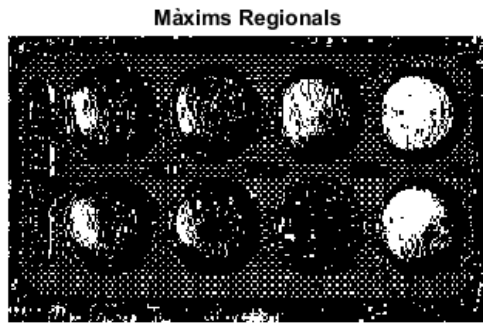


Control de qualitat de pastilles: Ens contacta una empresa farmacèutica per a què li programem un codi que faci un control de qualitat de la seva producció de blisters. L'objectiu és identificar quan un blister surt amb una pastilla faltant.

```
im = imread('astablet.tif');  
figure, imshow(im), title('Imatge Original')
```

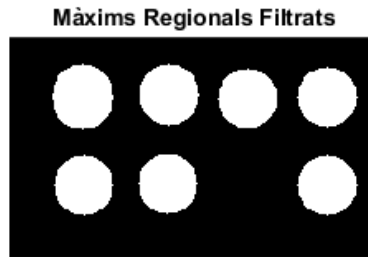
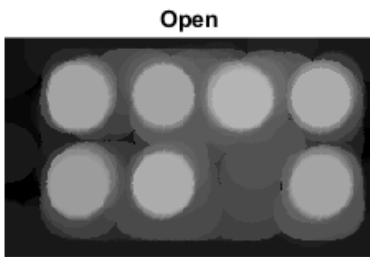



```
rm = imregionalmax(im); % Busquem màxims regionals i veiem que no funciona
figure, imshow(rm), title('Màxims Regionals')
```



```
% Fem un Open abans de buscar els màxims
ee = strel('disk', 20, 0);
op = imopen(im, ee);
rm2 = imregionalmax(op);

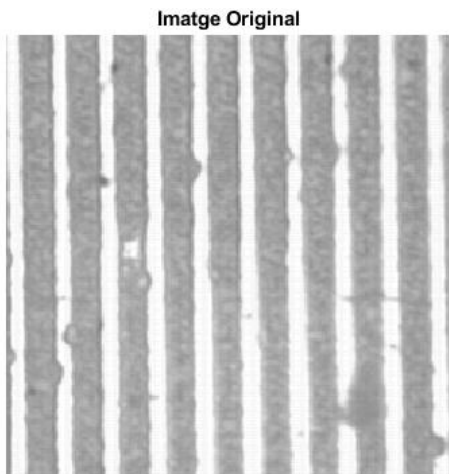
subplot(2,1,1), imshow(op), title('Open')
subplot(2,1,2), imshow(rm2), title('Màxims Regionals Filtrats')
```



Exercici

Control de qualitat: Una empresa ens demana fer un control de qualitat de la seva producció. L'objectiu és identificar els defectes de la peça de la imatge i representar-los sobre la imatge original per a què més endavant ells puguin fer una tria de quins són per llençar i quins són profitables.

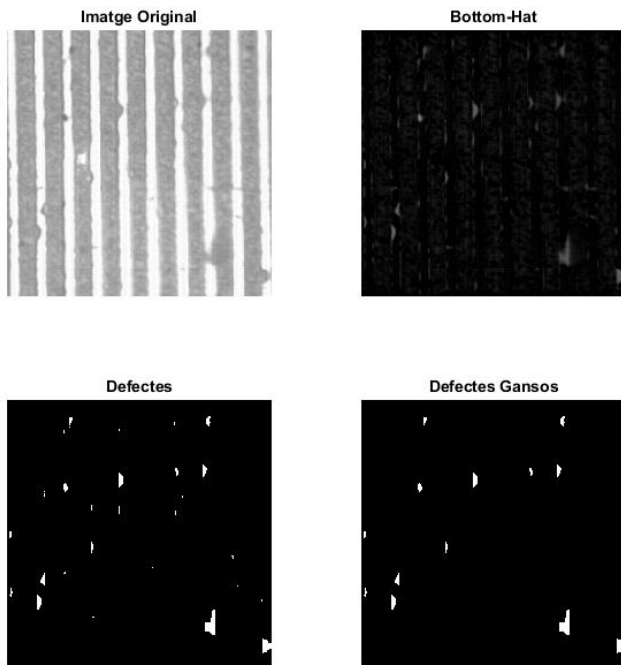
```
im = imread('r4x2_256.tif');
figure, imshow(im), title('Imatge Original')
```



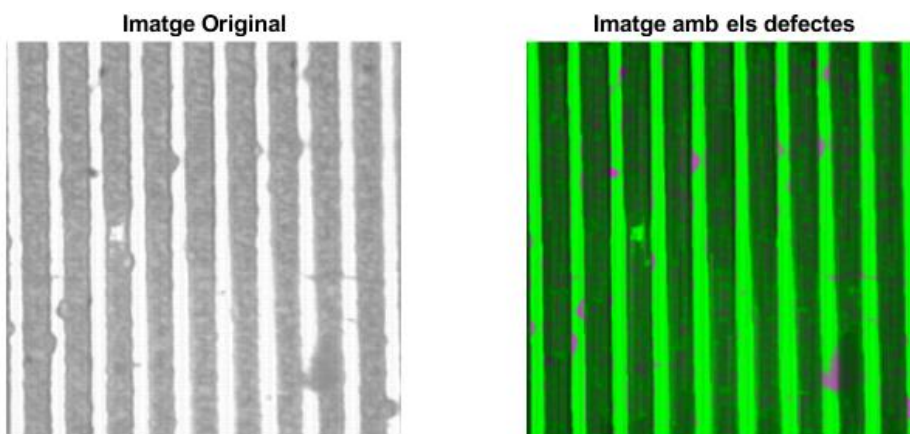
```
ee = ones(40, 1); % Posem 40 píxels d'alçada per identificar el defecte més gran, la resta ja  
els detecta 15
```

```
th = imbothat(im, ee); % Equival a agafar el residu d'un Close  
bin = th > 50; % Binaritzem la imatge  
res = bwareaopen(bin, 10); % Rentem Soroll
```

```
subplot(2,2,1), imshow(im), title('Imatge Original')  
subplot(2,2,2), imshow(th), title('Bottom-Hat')  
subplot(2,2,3), imshow(bin), title('Defectes')  
subplot(2,2,4), imshow(res), title('Defectes Gansos')
```



```
figure  
subplot(1,2,1), imshow(im), title('Imatge Original')  
subplot(1,2,2), imshow(imfuse(im, th)), title('Imatge amb els defectes')
```



Entrega 12: Binarització i Labelling

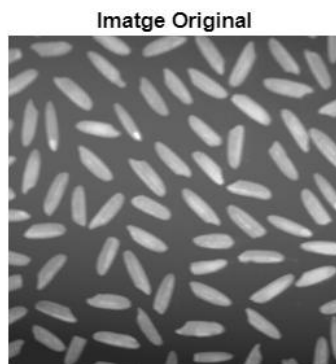
La binarització consisteix a transformar qualsevol imatge per a què tots els seus píxels valguin 0 o 1 (negre o blanc). Serveix per a segmentar objectes del fons. Cal tenir en compte que en binaritzar perdem informació de la imatge. Una bona binarització garanteix la connectivitat d'una estructura i la no-connectivitat de diferents objectes, a més de mantenir el gruix dels mateixos.

Hi ha diferents tècniques de binarització i cadascuna té els seus avantatges i desavantatges, és per això que segons les característiques de la imatge que vulguem binaritzar utilitzarem unes o altres segons ens convingui. La majoria de tècniques de binarització es basen en un o diversos *Thresholds* (llindars). Per a cada cas cal trobar el/s 'Optimal Threshold'.

Thresholding: binarització per llindar fixe

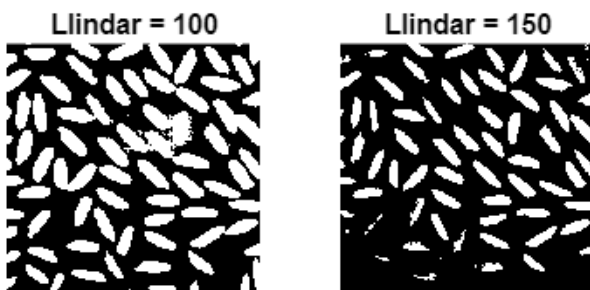
Rarament funcionen bé a causa de la il·luminació no uniforme. Consisteix a discriminar el valor del blanc o negre usant un llindar de color. S'utilitza molt en cadenes de producció industrial amb retro il·luminació.

```
% Binaritzat per llindar fixe: no acostuma a funcionar
im = imread("arros.tif");
figure, imshow(im), title('Imatge Original')
```

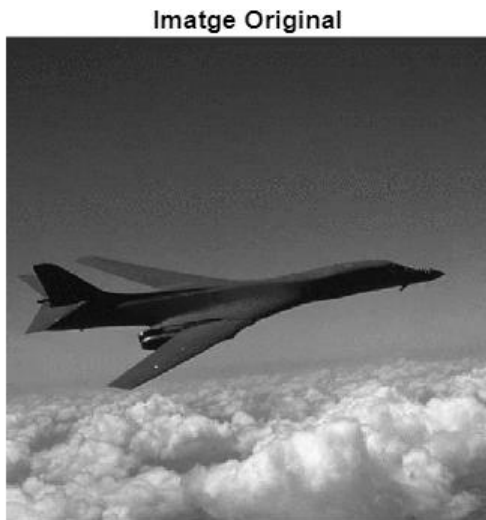


```
bin1 = im > 100;
bin2 = im > 150;
% Observem que a la 1a tenim falses connectivitats i a la 2a no es respecta
% el gruix de les estructures, a més de perdre alguns elements.
figure, sgtitle('Binaritzacions de la Imatge')
subplot(1,2,1), imshow(bin1), title('Llindar = 100')
subplot(1,2,2), imshow(bin2), title('Llindar = 150')
```

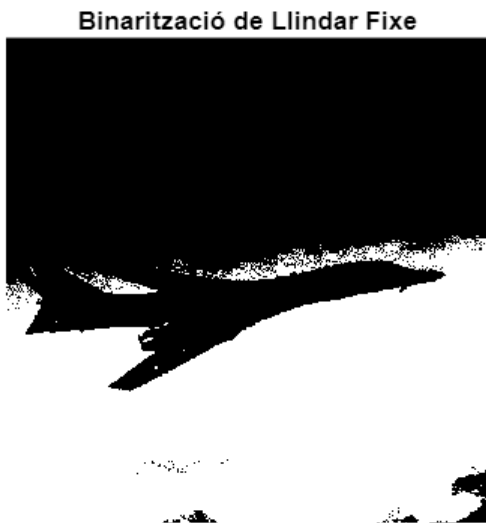
Binaritzacions de la Imatge



```
im = imread("airplane.tif");  
figure, imshow(im), title('Imatge Original')
```



```
bin = im > 100;  
% Observem que no mantenim totes les figures de la imatge  
figure, imshow(bin), title('Binarització de Llindar Fixe')
```

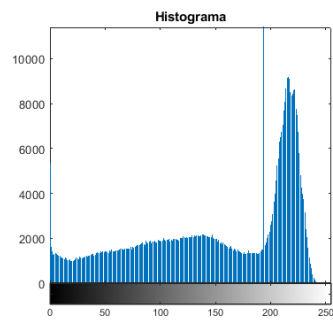


P-tile-thresholding

A partir d'un histograma bimodal decidim quin percentatge de píxels volem deixar per sobre o per sota del llindar. Aquesta tècnica presenta diversos problemes:

- L'histograma d'una imatge no té perquè ser bimodal.
- A vegades és difícil determinar si un histograma és bimodal.
- Tot i tenir un histograma bimodal, no tenim garantida una bona binarització.

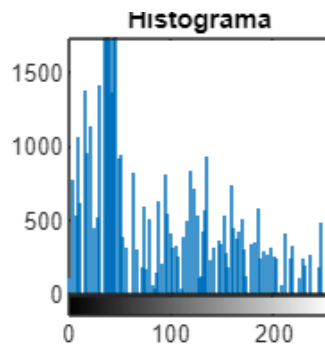
```
im2 = imread("dollar.tif");  
subplot(1,2,1), imshow(im2), title('Imatge Original')  
subplot(1,2,2), imhist(im2), title('Histograma')
```



```
bin = im2 > 150; % establim el valor segons l'histograma
figure, imshow(bin), title('Binarització')
```



```
im2 = imread("gull.tif");
subplot(1,2,1), imshow(im2), title('Imatge Original')
subplot(1,2,2), imhist(im2), title('Histograma')
```

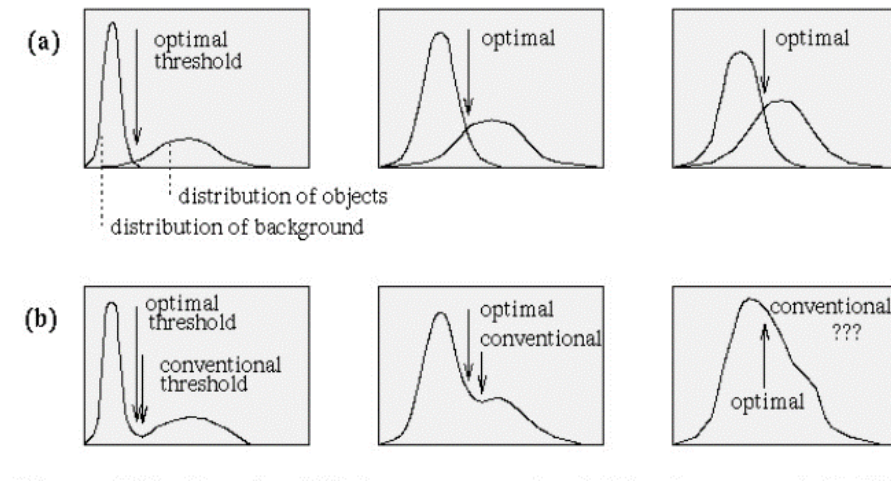


```
bin = im2 > 70; % establim el valor segons l'histograma
figure, imshow(bin), title('Binarització')
```



Criteri de Ridler

Assumeix que l'histograma sempre serà bimodal i busca quines dues campanes gaussianes modelarien aquest histograma. L'*Optimal Threshold* serà el punt de tall entre les dues gaussianes.



Si bé és cert que qualsevol histograma es pot aproximar amb dues gaussianes, hi ha histogrames que no val la pena modelar d'aquesta manera.

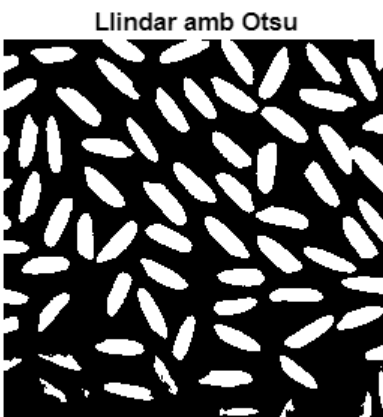
Otsu Thresholding

El criteri d'Otsu busca el llindar que maximitza la variància entre classes (B/N) o dit d'altra manera, busca minimitzar la variància intraclasse.

```
im = imread("arros.tif");  
th = graythresh(im)
```

th = 0.4902

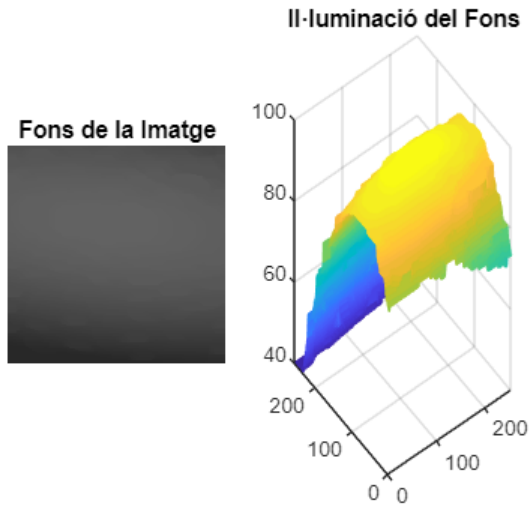
```
bin3 = im2bw(im,th);  
figure, imshow(bin3), title('Llindar amb Otsu')
```



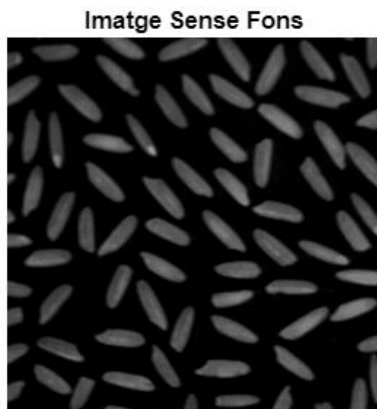
En moltes ocasions, els fons amb il·luminació no uniforme suposaran un obstacle a l'hora de binaritzar imatges. Una possible solució és eliminar-lo.

```
ee = strel('disk', 20);  
fons = imopen(im, ee);  
% Observem que la il·luminació no és uniforme
```

```
subplot(1,2,1), imshow(fons), title('Fons de la Imatge')
subplot(1,2,2), mesh(fons), title('Il·luminació del Fons')
```



```
res = imsubtract(im, fons);
figure, imshow(res), title('Imatge Sense Fons')
```



```
% Ara ja podem binaritzar amb un llindar global perquè hem equilibrat els
% nivells de gris del fons
bin_arros = im2bw(res, graythresh(res));
figure, imshow(bin_arros), title('Binarització Global')
```



Binarització de doble llindar

Consisteix a definir dos *thresholds*, un *High* (T_H) i un altre *Low* (T_L), això divideix la imatge en tres regions: *Low* (R_L), *Mid* (R_M) i *High* (R_H). Els píxels de R_L els posarem a 1, els píxels de R_H els posarem a 0 i els píxels que quedin a R_M decidirem segons algun altre criteri, per exemple la seva connectivitat:

Recorrerem tots els píxels R_M i tots aquells que siguin veïns d'un píxel d' R_L els posarem a 1. Repetirem aquest procés fins que deixi d'haver canvis en la imatge i aleshores posarem a 0 tots els píxels que quedin a R_M .

Llindars locals

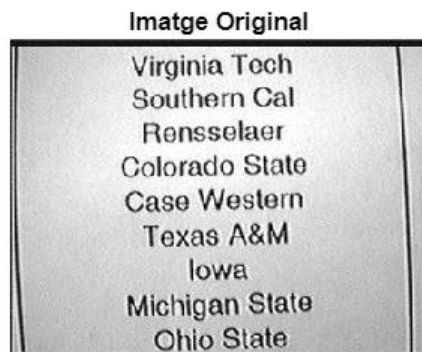
Aquesta tècnica va més enllà de definir dos llindars i defineix tants llindars com píxels tingui la imatge, ja que els definim segons les regions de la imatge (binarització local). Consisteix a passar una finestra lliscant per tota la imatge que calculi la mitjana dels píxels de l'entorn i, segons si el píxel que estiguem tractant en aquell moment està per sobre o per sota d'aquesta mitjana, li assignem el blanc o el negre.

D'aquesta forma, forcem que hi hagi valors de blanc i valors de negre a totes les àrees de la imatge. En cas que vulguem evitar-ho, podem afegir un desplaçament arbitrari al valor mig a l'hora de fer la comparació.

Exercici

Binarització d'una pàgina amb text: binaritzeu la següent imatge amb el mètode que considereu més adequat.

```
im = imread('textsheet.jpg');  
figure, imshow(im), title('Imatge Original')
```



```
th = graythresh(im); % Calculem el th d'otsu  
bin = im2bw(im,th);  
% Tot i que la binarització és bona, les lletres han perdut gruixor  
figure, imshow(bin), title('Llindar per Otsu')
```




```

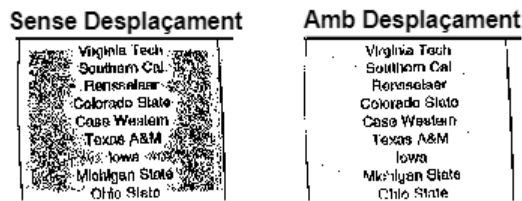
kernel = ones(31);
kernel(:, :) = 1/(31^2);

avg = imfilter(im, kernel, 'conv');
bin1 = im > (avg);
bin2 = im > (avg-20);

figure, sgtitle('Llindars Locals')
subplot(1,2,1), imshow(bin1), title('Sense Desplaçament')
subplot(1,2,2), imshow(bin2), title('Amb Desplaçament')

```

Llindars Locals



Etiquetar/Labelling

Etiquetar serveix per a identificar les diferents components connexes d'una imatge binària, que després ens pot servir per a treure dades i anàlisis de la imatge i les seves components.

L'etiquetatge s'implementa amb un algorisme de dues passades, quan es troba un nou píxel no etiquetat, si és veí d'un ja etiquetat, se li assigna l'etiqueta del veí, si és nou, se li assigna una nova etiqueta, cal fer dues passades per tal d'evitar etiquetes errònies.

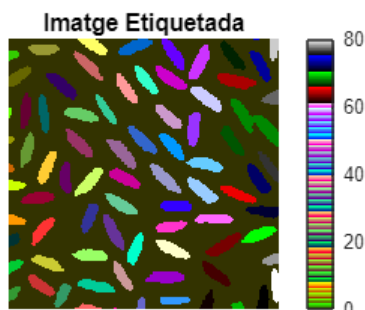
A l'hora d'establir el criteri de connectivitat, podem escollir entre connectivitat-4 o connectivitat-8, segons considerem a cada cas, però és important que siguem coherents amb la nostra decisió durant tot el procés, ja que sinó obtindrem dades incorrectes.

Un cop hem acabat amb la binarització dels grans d'arròs, el següent pas seria etiquetar-la i analitzar-ne les seves característiques.

```

[eti, num] = bwlabel(bin_arros, 4); % Establim criteri de connectivitat-4
figure, imshow(eti, []), title('Imatge Etiquetada'), colorbar, colormap colorcube

```

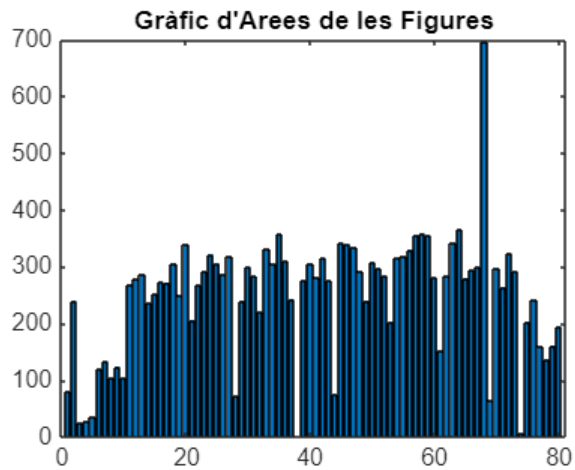


En moltes ocasions, cal eliminar els objectes de la imatge que toquen vores, ja que no estan complets i ens esbiaixen les dades.

```
dades = regionprops(eti, 'all');  
area50=dades(50).Area
```

```
area50 = 308
```

```
Arees=[dades.Area];  
% Observem un gra d'àrea desproporcionada i varis de mida anormalment  
% petita (els que toquen vores)  
figure, bar(Arees), title("Gràfic d'Arees de les Figures")
```



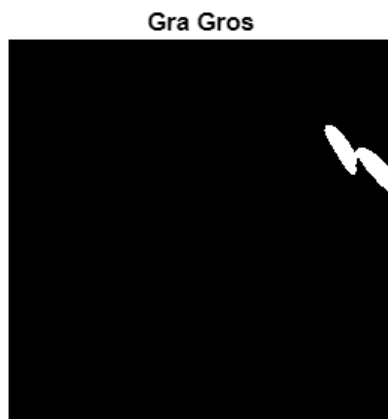
```
max_A=max(Arees)
```

```
max_A = 695
```

```
gros = find(Arees==max_A)
```

```
gros = 68
```

```
figure, imshow(eti == gros), title('Gra Gros')
```



Segmentació de la Imatge

La Segmentació d'imatges consisteix a dividir la imatge en regions amb característiques similars. Cada regió es representarà amb una vora tancada. Però quines característiques busquem i què significa que siguin similars?

Hi ha diversos criteris per a decidir què és el que conforma un objecte: proximitat, similitud, continuïtat, tancaments, etc. L'ésser humà sap discernir entre diversos objectes atenent a aquests criteris i més, però un ordinador no n'és capaç, així doncs s'han de trobar mètodes i algorismes que suposin una manera efectiva de segmentar imatges.



Podem abordar la segmentació de diferents maneres:

- **Binaritzat:** el mètode més senzill de tots.
- **Segmentació basada en contorns:** basada en la diferència entre píxels: Canny i Watershed.
- **Segmentació basada en *clustering*:** K-means *clustering*.

Quan segmentem imatges, hi haurà dos conceptes omnipresents:

- **Contorns:** es troben buscant diferències de valors entre píxels adjacents.
- **Regions:** es troben buscant similituds entre valors de píxels adjacents.

És important remarcar que l'extracció de contorns que hem fet anteriorment, tot i ser útil, no es pot considerar una segmentació, ja que hi poden aparèixer contorns que no es corresponguin amb les vores de la figura i també poden no aparèixer contorns que sí que haurien de ser presents en la detecció de vores. A més, les imatges de contorns acostumen a tenir soroll i a donar-nos contorns no fins.

Com a curiositat val la pena esmentar que la visió humana és molt sensible a la detecció de contorns.

Entrega 13: Algorisme de Canny

Algorisme per a trobar les vores d'una figura. Fins ara el que obteníem eren contorn interns o externs de la figura, que no eren exactes, eren gruixuts i no sempre estaven tancats. Canny ens permetrà trobar els contorns reals d'una figura.

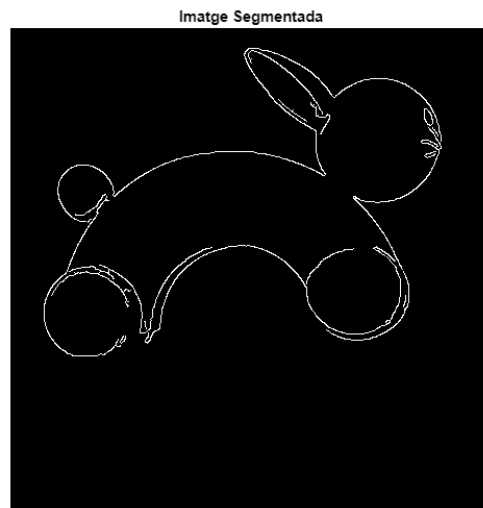
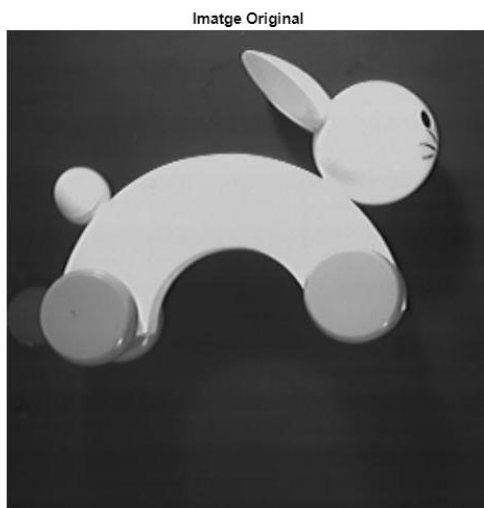
Té 3 etapes:

- **Realçar Contorns:** DoG. No només calcula el valor del gradient, sinó que també en calcula la direcció.
- **Supressió de no-màxims:** Trobem el màxim local. Obtenim un contorn fi i tancat. Com coneixem la direcció del gradient, en comptes d'haver de comparar el píxel amb tots els seus adjacents, només l'hem de comparar amb la direcció que ens marqui el gradient.
- **Binaritzat amb histèresi:** binaritzem amb dos llindars, qualsevol cosa menor que el *low threshold* la descartem com a vora, qualsevol cosa per sobre del *high threshold* la identifiquem com a vora. Els valors que quedin enmig de tots dos llindars, els seleccionem atenent a criteri de connectivitat: aquells que siguin connexos amb un contorn fort ens el quedem, aquells que no ho siguin els descartem.

Aleshores, quan apliquem Canny a una imatge podem "jugar" amb els següents elements:

- Desviació de la DoG: a major desviació perdrem més detalls.
- Low threshold i high threshold: si qualsevol dels dos és massa alt perdrem vores importants.

```
im = imread("rabbit.jpg");
res = edge(im, 'Canny',[0.01, 0.2], 0.51);
figure, imshow(im), title('Imatge Original')
figure, imshow(res), title('Imatge Segmentada')
```



Entrega 14: Watershed

Com ja vam veure, l'operador de Canny, que servia per a segmentar imatges, és un operador lineal. A continuació tractem la segmentació d'imatges amb 'Watershed', que és un algorisme morfològic.

L'analogia per a comprendre com funciona el watershed és un relleu topogràfic que s'inunda. La idea és que aquest relleu (que té muntanyes, serres, pics, etc.) comença a inundar-se fins que les valls són plenes d'aigua i només queden sense submergir els punts més elevats. Aquests punts més elevats seran la imatge de vores que busquem. Com pot haver contorns a diferents "alçades", la inundació "s'atura" sempre quan la vora d'aigua està a punt de trobar-se amb una altra vora. Tot aquest procés no es fa amb la imatge original, sinó amb la imatge de gradients.

Els passos per a segmentar amb watershed són:

1. Obtenir la imatge de gradients.
2. Aplicar watershed a la imatge de gradients partint dels mínims regionals.

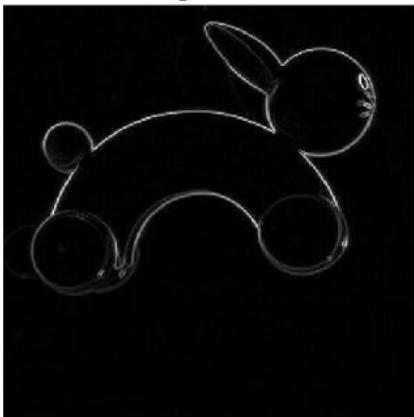
```
im = imread('rabbit.jpg');  
figure, imshow(im), title('Imatge Original')
```

Imatge Original

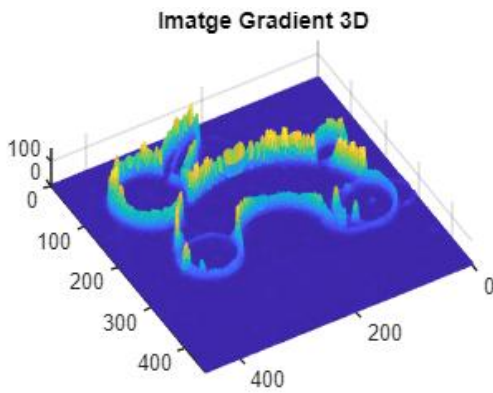


```
ee = strel('disk', 1);  
grad = imsubtract(imdilate(im,ee), imerode(im,ee));  
% No és una imatge de contorns, només els hem realçat  
figure, imshow(grad, []), title('Imatge Gradient')
```

Imatge Gradient



```
% Veiem que és un relleu 3D (muntanyes i valls)  
figure, mesh(grad), view(150,80), title('Imatge Gradient 3D')
```



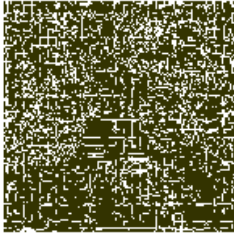
```
segm = watershed(grad); % Retorna la imatge etiquetada
```

```
% Observem que l'hem "sobresegmentat"
```

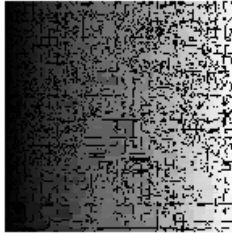
```
subplot(1,3,1), imshow(segm==0), title('Imatge Etiquetada')
subplot(1,3,3), imshow(segm, []), title('Colormap'), colormap colorcube
subplot(1,3,2), imshow(segm, []), title('Grey Scale')
sgtitle('Imatge Etiquetada')
```

Imatge Etiquetada

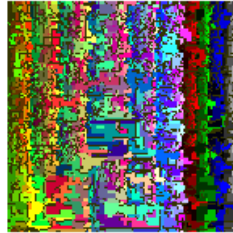
Imatge Etiquetada



Grey Scale



Colormap



```
% Degut a aquesta sobresegmentació, trobarem masses mínims regionals i el resultat del
% watershed no serà satisfactori.
```

```
rm = imregionalmin(grad);
grad = imimposemin(grad, rm);
segm = watershed(grad);
```

```
figure, sgtitle('Watershed Insatisfactòri')
subplot(1,3,1), imshow(rm, []), title('Mín. Regionals')
subplot(1,3,2), imshow(segm, []), title("Imatge etiquetada")
subplot(1,3,3), imshow(segm == 0), title("Watershed")
```

Watershed Insatisfactòri

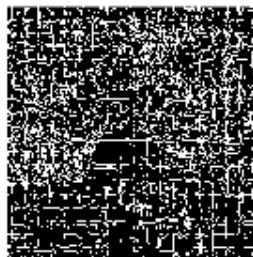
Mín. Regionals



Imatge etiquetada



Watershed



Watershed amb marques

Per tal d'evitar aquesta sobre-segmentació que hem observat, podem fer servir *markers*. Cal trobar un *marker* per cada objecte que vulguem segmentar i aplicar el watershed només a la imatge gradient generada a partir dels *markers*.

Però com situem els *markers*?

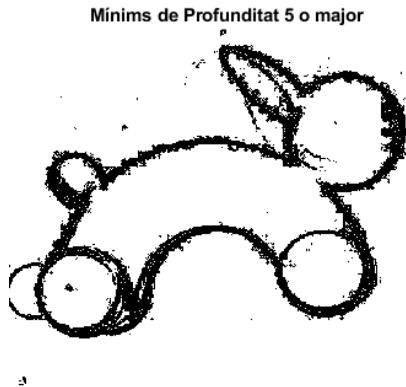
- Segmentació assistida: un humà situa els *markers*.
- Obtenció automàtica: s'ha de trobar un algorisme que aconseguixi situar marques robustes; bones marques implicaran una bona segmentació.

Generalment (no sempre) una bona forma d'obtenir unes bones marques de forma automàtica és a partir dels màxims o mínims regionals. Però com ja hem vist, en ocasions cal filtrar aquests màxims i mínims. Això podem fer-ho atenent a diferents característiques. Recordem la imatge de les aspirines a l'Entrega 11.

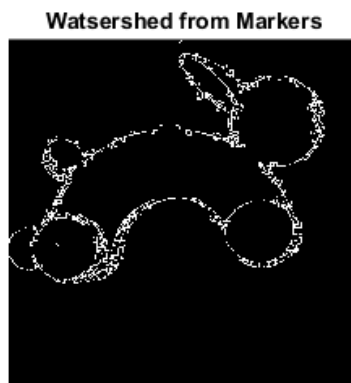
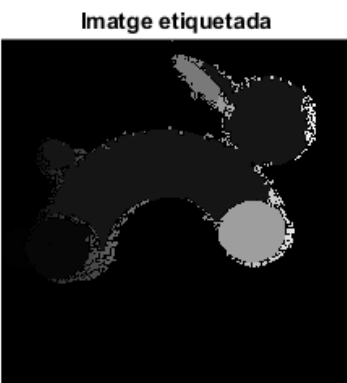
- Contrast: h-max i h-min.
- Forma: Opening.
- Mida: AreaOpening.

Markers per alçada:

```
rm2 = imextendedmin(grad, 5); % Trobem els mínims regionals de profunditat major de 5
figure, imshow(rm2, []), title('Mínims de Profunditat 5 o major')
```

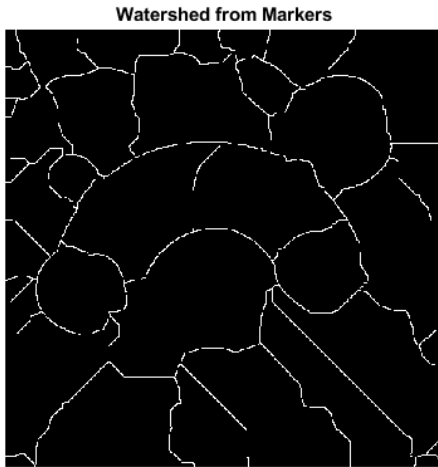


```
% Modifiquem la imatge de gradients per a que quedin només els que tinguin
% els mínims que ens interessin.
grad2 = imimposemin(grad, rm2);
segm2 = watershed(grad2);
% Obtenim un resultat millor. Però encara observem moltes regions petites.
subplot(1,2,1), imshow(segm2, []), title("Imatge etiquetada")
subplot(1,2,2), imshow(segm2 == 0), title("Watershed from Markers")
```



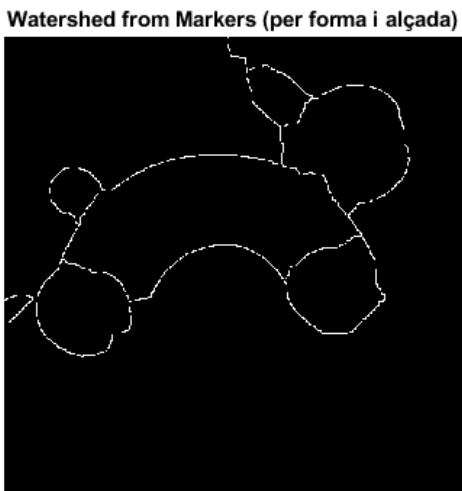
Markers per forma:

```
ee2 = strel('disk',20);  
grad3 = imclose(grad,ee2);  
segm3 = watershed(grad3);  
% Tenim masses formes no rellevants  
figure, imshow(segm3 == 0), title('Watershed from Markers')
```

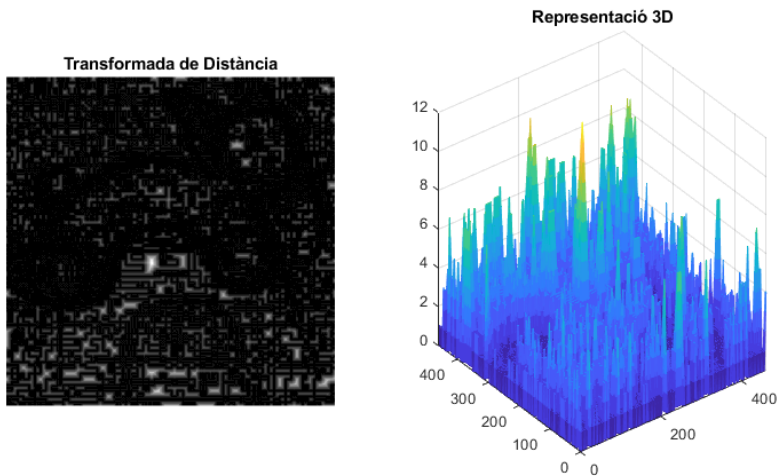


Markers per forma i alçada:

```
% Podem barrejar tots dos criteris:  
grad4 = imclose(grad2,ee2);  
segm4 = watershed(grad4);  
% Tot i no ser perfecta, és una segmentació prou sòlida.  
figure, imshow(segm4 == 0), title('Watershed from Markers (per forma i alçada)')
```



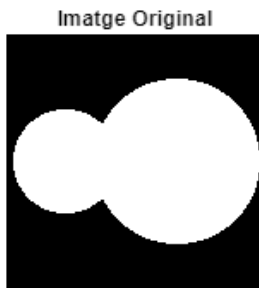

```
td = bwdist(grad);
subplot(1,2,1), imshow(td, []), title('Transformada de Distància')
subplot(1,2,2), mesh(td), title('Representació 3D')
```



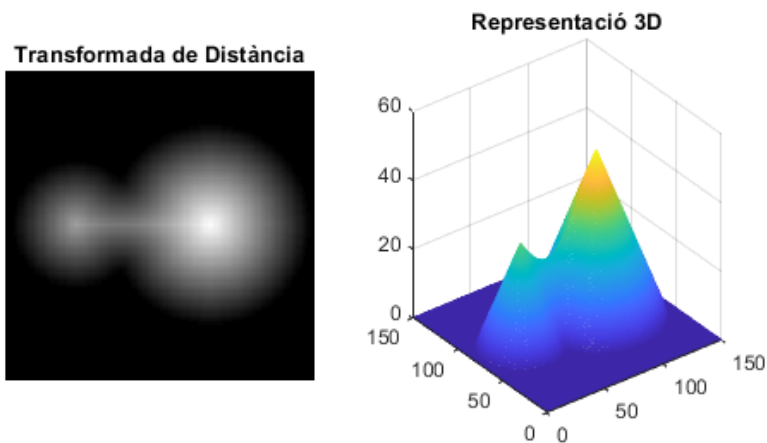
Aplicacions del Watershed

Una aplicació molt útil del Watershed és separar "Touching Blobs". Per a aconseguir separar figures en contacte, en comptes d'aplicar watershed sobre la imatge de contorns, haurem d'aplicar-lo sobre la transformada de distància o la seva inversa, depèn del que ens convingui en cada cas.

```
im = imread('touchcell.tif');
figure, imshow(im), title('Imatge Original')
```



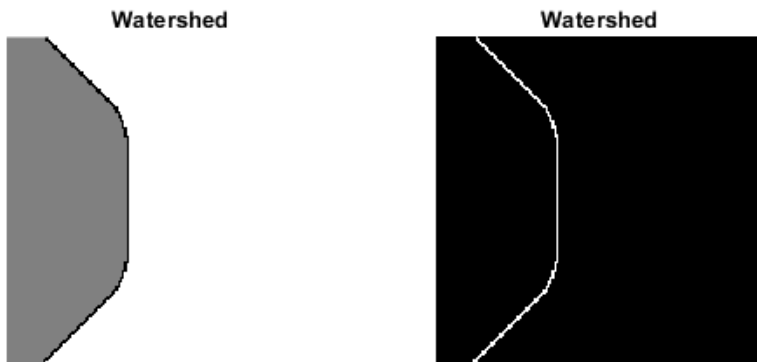
```
td = bwdist(~im);
subplot(1,2,1), imshow(td, []), title('Transformada de Distància')
subplot(1,2,2), mesh(td), title('Representació 3D')
```



```

segm = watershed(-td);
figure
subplot(1,2,1), imshow(segm, []), title('Watershed')
subplot(1,2,2), imshow(segm == 0), title('Watershed')

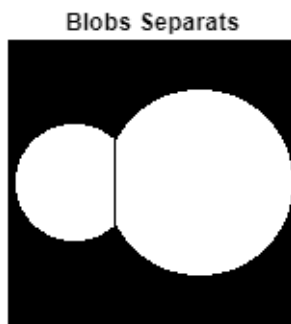
```



```

res = im;
res(segm==0) = 0;
figure, imshow(res), title('Blobs Separats')

```



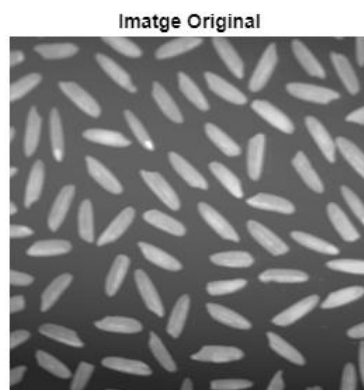
Exercici

Separar els diferents grans d'arròs.

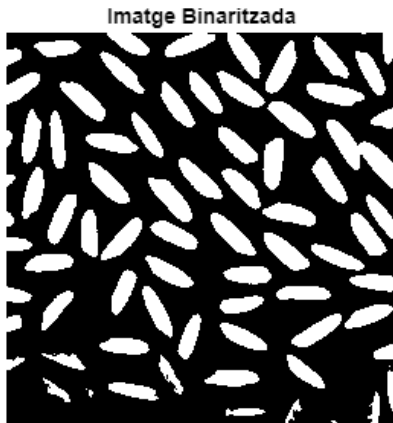
```

im = imread('arros.tif');
figure, imshow(im), title('Imatge Original')

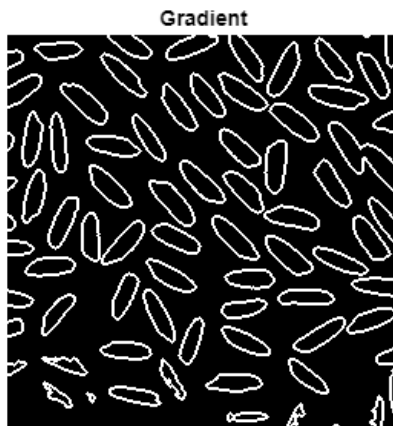
```



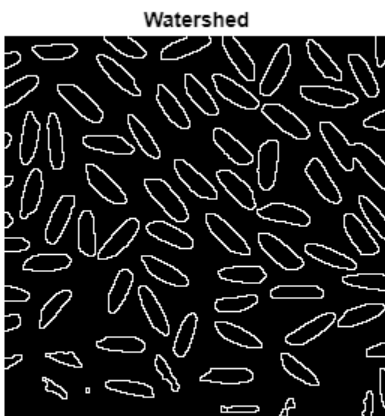
```
% Binaritzem amb Otsu
im_bw = im2bw(im, graythresh(im));
figure, imshow(im_bw), title('Imatge Binaritzada')
```



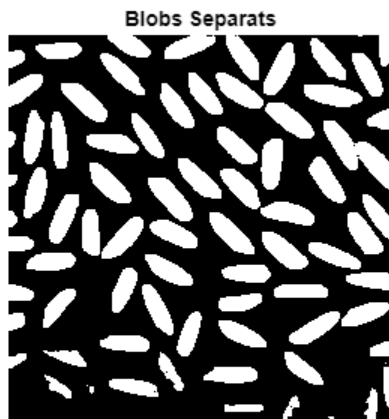
```
% Trobem la imatge gradient
ee = strel('disk', 1);
grad = imsubtract(imdilate(im_bw, ee), imerode(im_bw, ee));
figure, imshow(grad), title('Gradient')
```



```
% Generem el watershed
segm = watershed(grad);
figure, imshow(segm == 0), title('Watershed')
```



```
% Intersectem el watershed i la imatge binaritzada  
res = im_bw;  
res(segm==0) = 1;  
figure, imshow(res), title('Blobs Separats')
```



Entrega 15: K-Means Clustering

K-Means és un algoritme d'aprenentatge automàtic no supervisat, és a dir que partim de dades no classificades ni etiquetades i permetem que l'algorisme les agrupi, sense supervisió ni entrenament previ, atenent a alguna/es de les seves característiques. Té com a objectiu dividir un conjunt d' n observacions (en el nostre cas seran píxels) en k grups, on cada observació pertany al grup del què el valor mig sigui més proper. Això produeix un agrupament de dades en funció de les seves similituds i diferències, minimitza la variància intracluster i maximitza la variància intercluster.

Expressat matemàticament, l'algorisme busca minimitzar la següent funció:

$$J = \sum_{i=0}^k \sum_{x \in C_i} ||x - \mu_i||^2$$

Que en paraules es tradueix: minimitzar la suma de distàncies de tots els punts al centroides del seu cluster. Consideracions a tenir en compte:

- $0 \leq k \leq n$.
- La distància entre valors es calcula fent la **distància Euclidiana**.

El funcionament pas a pas de l'algorisme és el següent:

1. Es generen k punts aleatoris dins de l'espai de treball. Aquests punts seran el centroides dels *clusters*.
2. S'assigna un *cluster* a cada píxel de l'espai (el que estigui més a prop).
3. Es recalculen els centroides per a què equivalguin a la mitja dels valors del seu *cluster*.
4. Es repeteixen els punts 2 i 3 fins que no hi hagi canvis respecte a la iteració anterior.

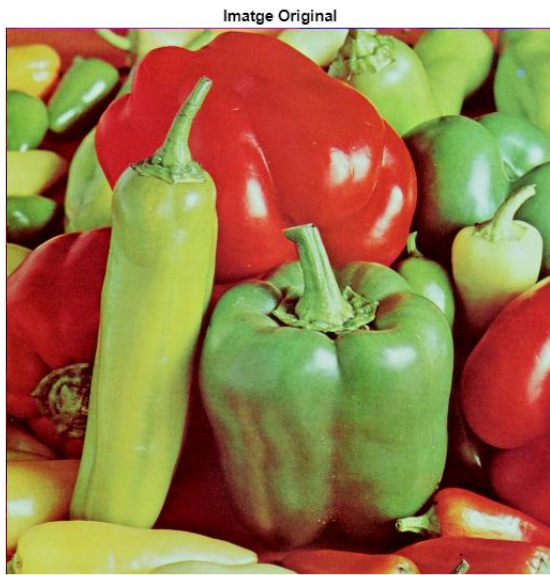
Degut a això, es pot deduir que cada cop que executem l'algorisme donarà un resultat diferent (no és un algorisme determinista), ja que els centroides inicials dels *clusters* es generen de manera aleatòria. També veiem que k ha de venir donada, l'ordinador no ens trobarà la k òptima, l'hem de decidir nosaltres en funció de les característiques de la imatge, una anàlisi prèvia i els nostres objectius. Finalment, cal remarcar que aquest és un mètode altament sensible al soroll ja que els *outliers* alteraran considerablement les mitjanes de posició.

Fins ara el nostre espai de treball sempre han estat els píxels de la imatge. Per a utilitzar aquest algorisme però, haurem de treballar en espais que representaran altres característiques de la imatge. Aquests espais de treball poden resultar més abstractes i menys visuals, per això és important tenir clar què estem fent a cada moment. Relatiu a això, cal dir que *K-means* només es pot aplicar a espais de característiques quantitatives, ja que no tindrà sentit aplicar-ho en espais de característiques qualitatives.

Nosaltres utilitzarem aquest algorisme per a segmentar per colors, però té aplicacions molt diverses si es troba una bona forma de representar les característiques quantitatives de la nostra imatge per a que siguin suficientment diferenciables en classes. En el següent enllaç hi ha un exemple d'un classificador de xifres escrites a mà mitjançant *K-means clustering*:

https://www.unioviado.es/compnum/laboratorios_py/kmeans/kmeans.html

```
im = imread('peppers.png');
figure, imshow(im), title('Imatge Original')
```



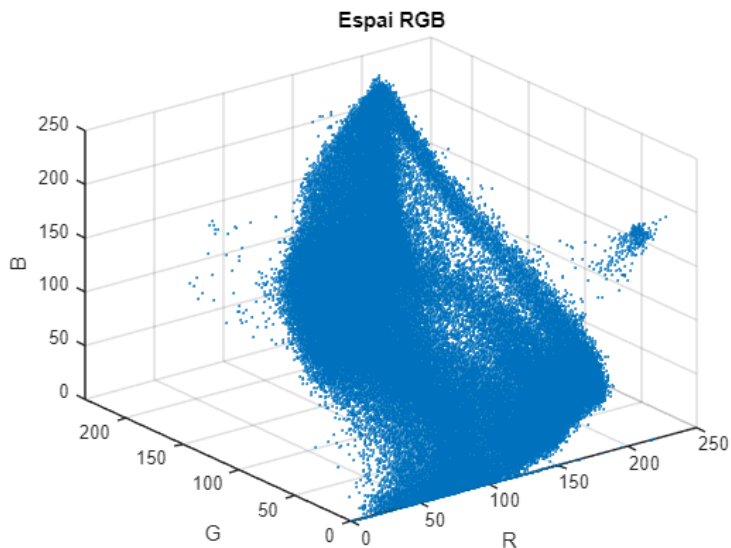
```
k = 2;
[rows cols chns] = size(im) % chns -> nº de canals (rgb = 3)
```

```
rows = 512
cols = 512
chns = 3
```

```
vec = reshape(double(im), rows*cols, chns); % vector de característiques
```

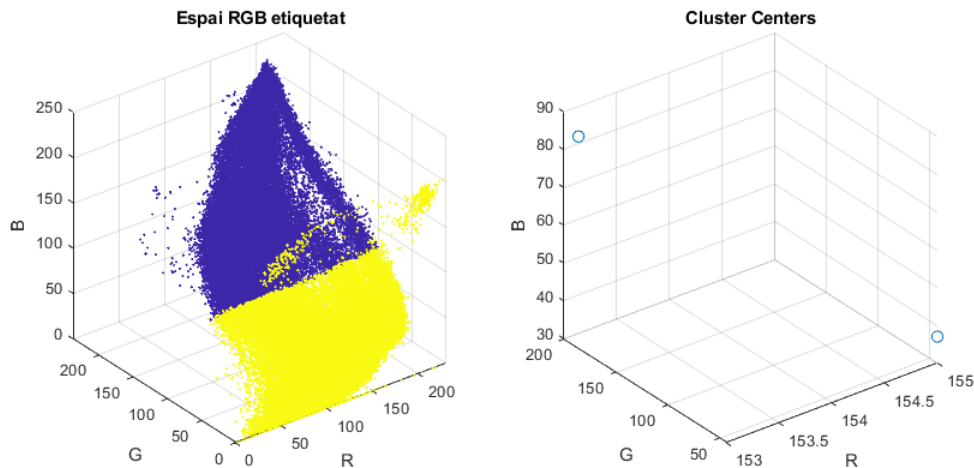
```
r = vec(:,1); g = vec(:,2); b = vec(:,3);
```

```
figure, scatter3(r,g,b,1) % últim paràmetre: mida de les mostres
xlabel('R'), ylabel('G'), zlabel('B'), title('Espai RGB')
```



```
% [index de la mostra, centre de la mostra]
[cl_idx, cl_ctr] = kmeans(vec,k,'Distance','cityblock');

figure
subplot(1,2,1), scatter3(r,g,b,1,cl_idx) % colorem segons l'índex de la mostra
xlabel('R'), ylabel('G'), zlabel('B'), title('Espai RGB etiquetat')
subplot(1,2,2), scatter3(cl_ctr(:,1),cl_ctr(:,2),cl_ctr(:,3),50)
xlabel('R'), ylabel('G'), zlabel('B'), title('Cluster Centers')
```



```
% Generem la imatge segmentada
eti = reshape(cl_idx,rows,cols);
figure, sgtitle('Segmentació amb K-Means')
subplot(1,3,1), imshow(im), title('Imatge Original')
subplot(1,3,2), imshow(eti, []), title('Segmentació B/W')

% Representem la imatge amb els valors dels centroides
rgb = ind2rgb(eti, cl_ctr/255);
subplot(1,3,3), imshow(rgb, []), title('Segmentacio Colorada')
```

Segmentació amb K-Means

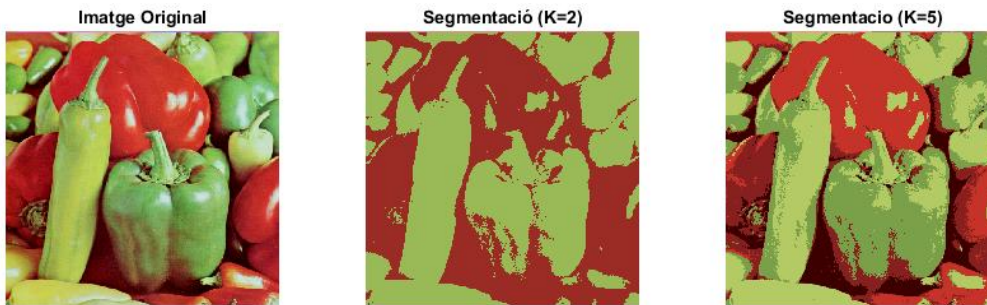


```

k = 5;
[cl_idx, cl_ctr] = kmeans(vec,k,'Distance','cityblock');
eti = reshape(cl_idx,rows,cols);
rgb2 = ind2rgb(eti, cl_ctr/255);

figure
subplot(1,3,1), imshow(im, []), title('Imatge Original')
subplot(1,3,2), imshow(rgb, []), title('Segmentació (K=2)')
subplot(1,3,3), imshow(rgb2, []), title('Segmentacio (K=5)')

```



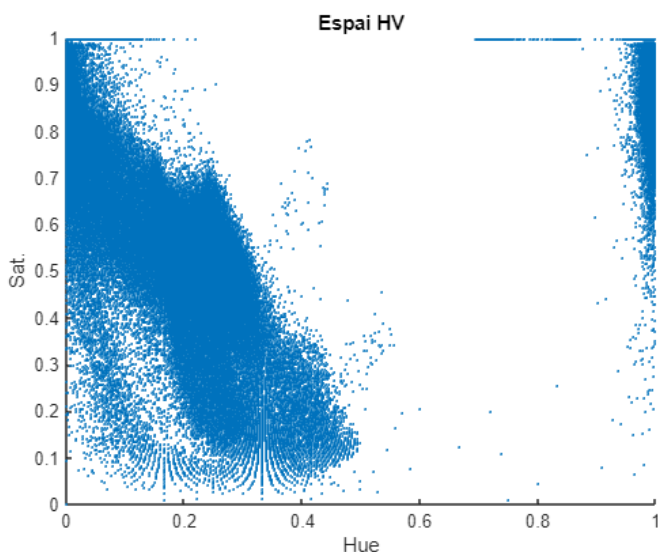
D'igual manera que hem utilitzat *K-Means* per a segmentar a l'espai RGB, podem fer-ho en l'espai HV, del format HSV.

```

im_hsv = rgb2hsv(im);
vec_hs = reshape(im_hsv(:,:,1:2), rows*cols, 2);

figure, scatter(vec_hs(:,1), vec_hs(:,2), 1), title('Espai HV');
xlabel('Hue'), ylabel('Sat.')

```



```

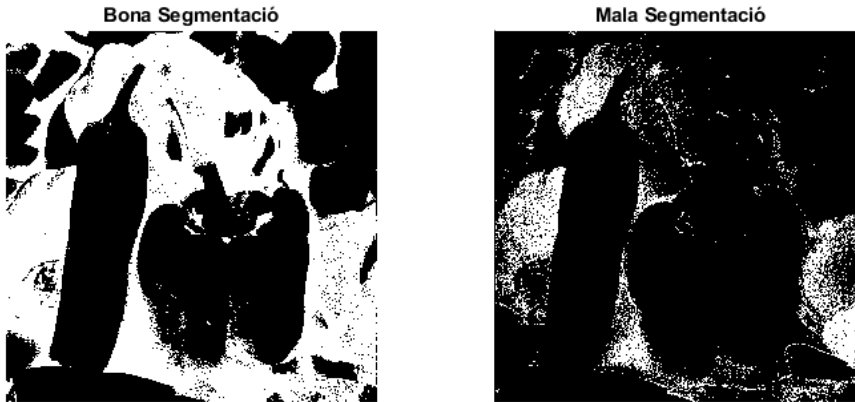
k = 2;
[cl_idx2, cl_ctr2] = kmeans(vec_hs, k, 'Distance', 'cityblock');
eti2 = reshape(cl_idx2, rows, cols);
[cl_idx3, cl_ctr3] = kmeans(vec_hs, k, 'Distance', 'cityblock');
eti3 = reshape(cl_idx3, rows, cols);
figure, sgtitle('Segmentació amb K-Means en HS')

```



```
subplot(1,2,1), imshow(eti2, []), title('Bona Segmentació')
subplot(1,2,2), imshow(eti3, []), title('Mala Segmentació')
```

Segmentació amb K-Means en HS

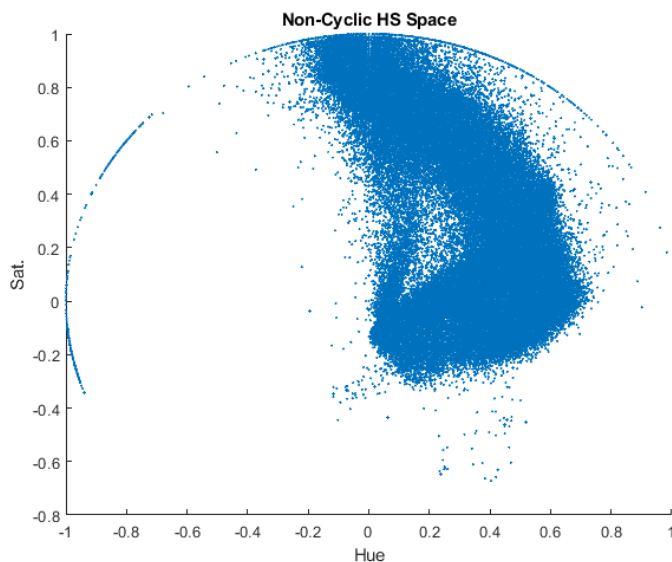


Veiem que en l'espai HV és relativament habitual trobar-nos amb males segmentacions. Això es deu a que el Hue és cíclic i no l'estem tractant com a tal, per tant hi ha valors que tot i ser molt semblants, a l'hora d'executar l'algorisme són tractats com si fossin completament diferents. Això ho solucionem adaptant el Hue:

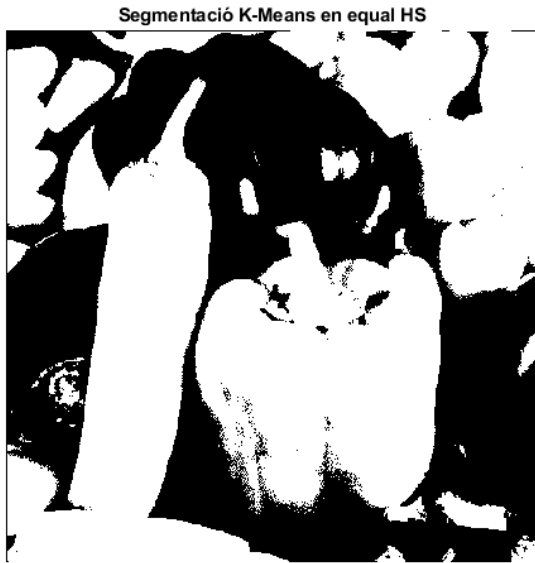
```
ch1 = vec_hs(:,2) .* sin(vec_hs(:,1)*2*pi);
ch2 = vec_hs(:,2) .* cos(vec_hs(:,1)*2*pi);

vec_hs(:,1) = ch1;
vec_hs(:,2) = ch2;

figure, scatter(vec_hs(:,1), vec_hs(:,2), 1), title('Non-Cyclic HS Space')
xlabel('Hue'), ylabel('Sat.')
```



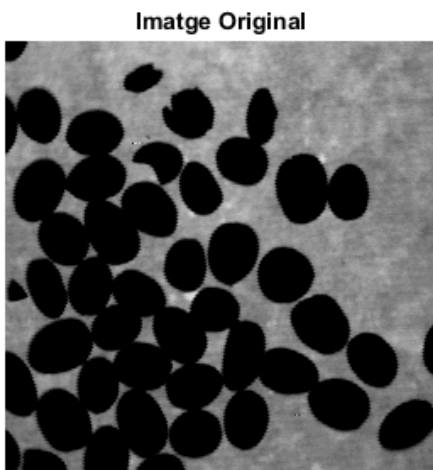
```
[cl_idx, cl_ctr] = kmeans(vec_hs, k, 'Distance','cityblock');
eti = reshape(cl_idx, rows, cols);
figure, imshow(eti, []), title('Segmentació K-Means en equal HS')
```



Exercici

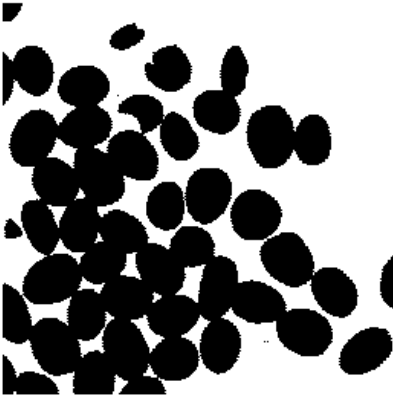
Separar touching blobs: cal separar els diferents grans de cafè per a que sigui possible fer una anàlisi de les seves característiques.

```
im = imread('cafe.tif');
figure, imshow(im), title('Imatge Original')
```



```
% Binaritzem amb Otsu
im_bw = im2bw(im, graythresh(im));
figure, imshow(im_bw), title('Imatge Binaritzada')
```

Imatge Binaritzada



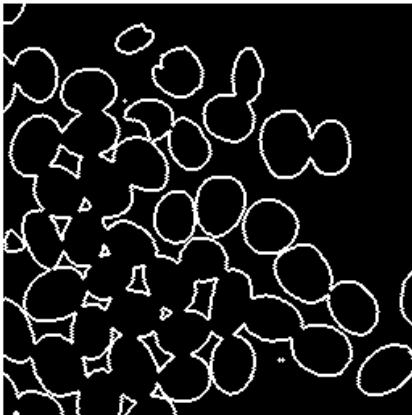
```
% Trobem la imatge gradient
```

```
ee = strel('disk', 1);
```

```
grad = imsubtract(imdilate(im_bw,ee), imerode(im_bw,ee));
```

```
figure, imshow(grad), title('Gradient')
```

Gradient



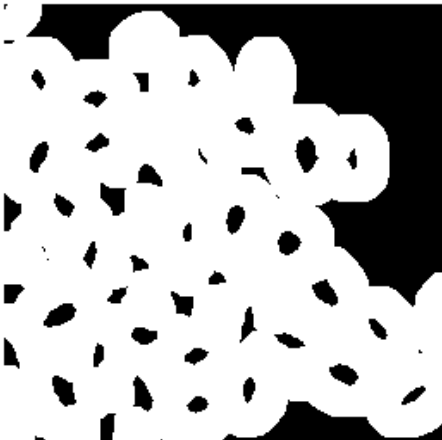
```
% Trobem el gradient dilatat
```

```
ee = strel('disk', 10);
```

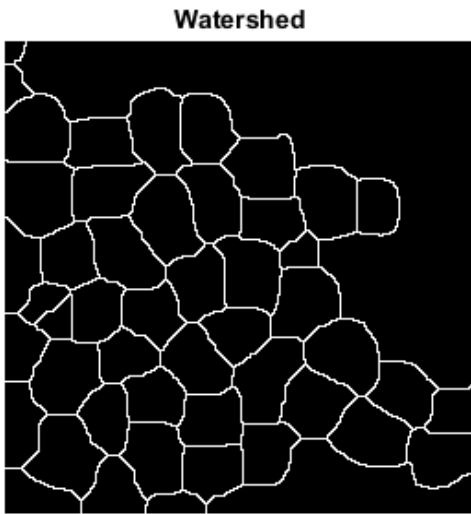
```
grad = imsubtract(imdilate(im_bw,ee), imerode(im_bw,ee));
```

```
figure, imshow(grad), title('Gradient (ee=10)')
```

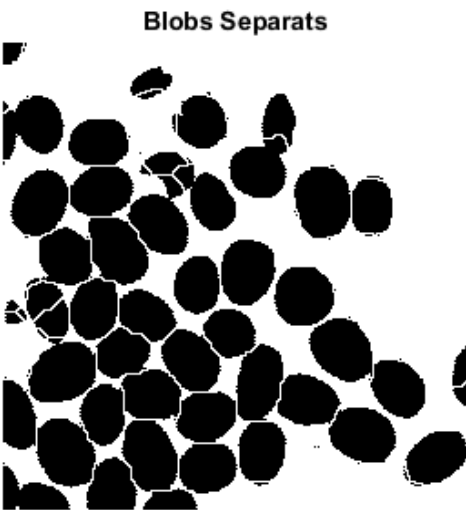
Gradient (ee=10)



```
% Generem el watershed  
segm = watershed(grad);  
figure, imshow(segm == 0), title('Watershed')
```



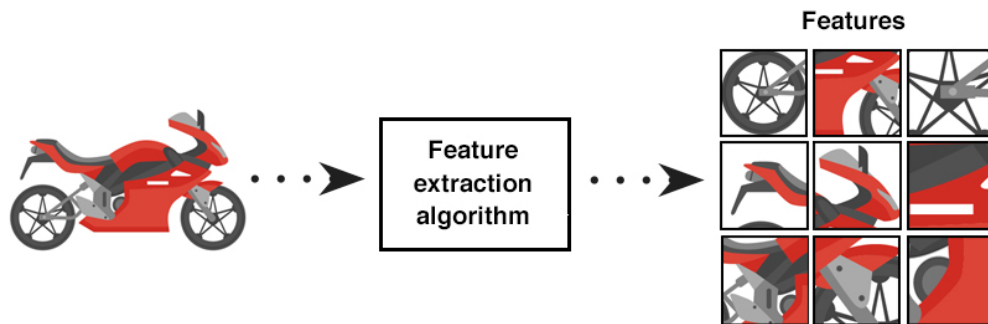
```
% Intersectem el watershed i la imatge binaritzada  
res = im_bw;  
res(segm==0) = 1;  
figure, imshow(res), title('Blobs Separats')
```



Extracció de Descriptors

Un cop hem preprocessat i segmentat les imatges, l'últim pas abans de passar al reconeixement amb l'ajuda de models o l'entrenament dels propis models és obtenir uns bons descriptors. Els descriptors són característiques que representen les regions d'interès d'una imatge, generalment són propietats quantitatives i per tant es poden expressar numèricament.

Aquests descriptors els aconseguim extraient característiques numèriques de les regions. Però per a això necessitem una certa invariància de les característiques entre imatges o, com a mínim, tenir una manera de transformar aquesta característica entre imatges.



Un mateix element pot estar present en diverses imatges amb diferents característiques modificades:

- Rotacions
- Translacions
- Escales
- Resolucions
- Il·luminacions
- Oclusions
- Perspectives
- etc.

A continuació s'expliquen una selecció de descriptors geomètrics de contorns i com obtenir-los, però val la pena saber que no només existeixen els descriptors geomètrics, sinó que també hi ha descriptors estadístics (p.ex. l'acumulació de nivells de gris d'una figura).

Podem classificar els descriptors geomètrics en dos tipus segons com els obtenim: descriptors basats en contorns i descriptors basats en regions.

Els descriptors de contorns són útils quan els contorns representen la major part de la informació rellevant de la forma, per exemple en siluetes o formes clarament definides (p.ex. una imatge binaritzada o segmentada). Els descriptors de regions, per altra banda, són més útils quan necessitem analitzar el contingut global de la regió, no només les vores. Nosaltres ens centrarem en els descriptors de contorns i deixarem de banda els descriptors de regions.

Entrega 16: Descriptors I

Perímetre

El descriptor de contorn més senzill, el propi perímetre de la figura. No és robust a escalats, translacions ni rotacions, molt menys a les oclusions, però és el més senzill d'implementar.

Relacions Geomètriques

Uns dels descriptors de contorns més genèrics són les relacions geomètriques d'una figura, per exemple la **compacitat**. La compacitat és la relació existent en una figura entre l'àrea i el perímetre. Es calcula:

$$C = \frac{4\pi * A}{p^2}$$

La compacitat és un descriptor invariant al zoom (l'escalat) de la imatge. El cercle és la forma amb compacitat màxima.

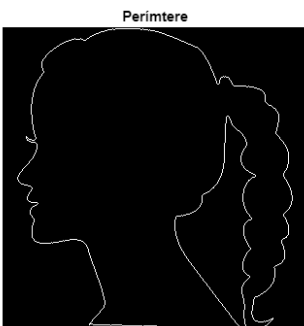
```
im = imread('head.png');  
im = imresize(im, 0.5);  
figure, imshow(im), title('Imatge Original')
```



```
area = sum(im(:))
```

```
area = 58501
```

```
% Obtenim el perímetre de la figura  
ero = imerode(im, strel('disk', 1));  
contorn = imsubtract(im,ero);  
figure, imshow(contorn), title('Perímetre')
```



```
perímetre = sum(contorn(:))
```

```
perímetre = 1526
```

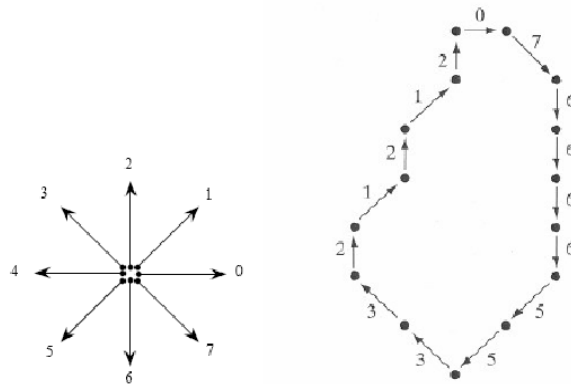
```
compacitat = 4*pi*area/perimetre/perimetre
```

```
compacitat = 0.3157
```

Codis de Freeman / Codis de Cadena

Descriuen una forma com un seguit de direccions. Es pren un píxel inicial i es segueix el perímetre de la figura fins que es torna a l'inici.

Són molt útils per a descriure contorns de manera compacta i fer operacions. En l'exemple següent es pren com a píxel inicial el de dalt a l'esquerra:



0, 7, 6, 6, 6, 5, 5, 3, 3, 2, 1, 2, 1, 2

És fàcil veure que aquests codis són invariants a les translacions però no a les rotacions. Per això és millor codificar-los amb les direccions incrementals, no amb les absolutes (indicar els moviments que fem des del punt de vista del píxel, no de l'observador), ja que així els fem robustos a rotacions. Matemàticament podem dir que el codi incremental és la derivada del codi absolut.

Tot i que calculant les direccions relatives sí que aconseguim robustesa davant de les translacions i rotacions no l'aconseguim davant dels escalats, ja que l'escala determinarà la quantitat de moviments que repetim en cada direcció abans de canviar-la. Això es pot solucionar fàcilment escalant la imatge o el codi.

Finalment, cal indicar que aquests codis no són 100% robustos a la rotació a causa de la discretització. És impossible representar la totalitat de direccions en una matriu quadrada i, per tant, es perd la inclinació estrictament real per una aproximada. Però això rarament ens afectarà a excepció de casos molt concrets o una molt mala resolució d'imatge.

Llistes de Coordenades

Són poc eficients computacionalment degut a la seva mida, que també suposa un desavantatge per l'espai en memòria, i no són invariants a transformacions. La seva característica principal que ressalta davant d'altres descriptors és que són representacions exactes del contorn de la figura, això pot ser bo o dolent, segons els detalls que necessitem pel nostre estudi. Es poden utilitzar per a extreure altres descriptors per exemple els Descriptors de Fourier.

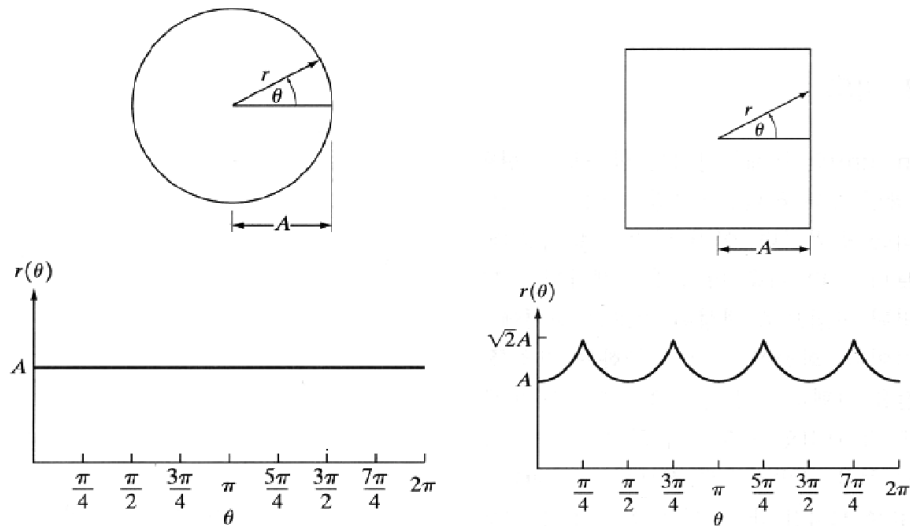
```
% Trobem les coordenades del 1r píxel blanc  
[row col] = find(contorn, 1)
```

```
row = 155  
col = 16
```

```
% Obtenim un volcat amb totes les coordeandes dels píxels de contorn  
B = bwtraceboundary(contorn, [row,col], 'E'); % direcció Est
```

Signatures

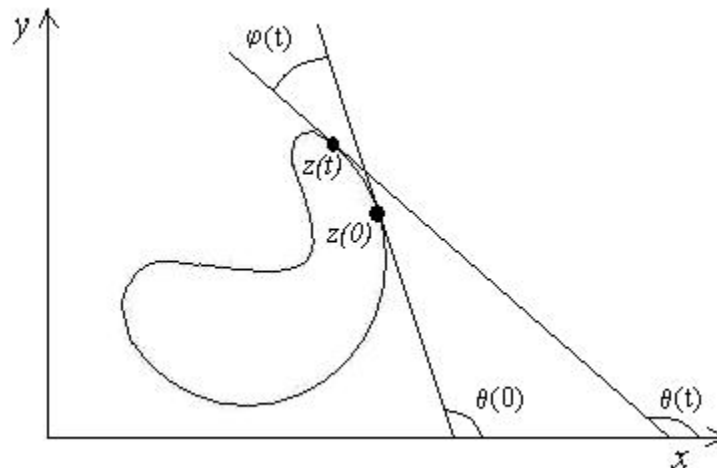
Són una seqüència de distància des de la posició central de la figura als contorns de l'objecte.



Les signatures ens poden donar problemes a les figures amb concavitats, ja que fent un "escombrat" des del centre de la imatge, podem no considerar tots els punts de la figura.

Slope Density Function

Recorren tots els píxels del contorn del primer a l'últim i la descripció final de l'objecte és la direcció del gradient a cada píxel. Això ens soluciona el problema que tenien les signatures amb les formes còncaves.



L'avantatge de les signatures és que totes les formes tenen el mateix nombre d'elements en el descriptor. L'*slope density function* ens dificulta comparar les característiques entre objectes perquè no tots tenen el mateix nombre d'elements.

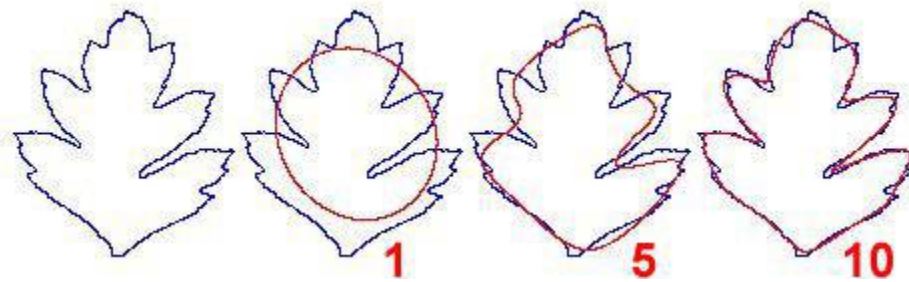
Descriptors de Fourier

El teorema de Fourier afirma:

- **Sèries de Fourier:** tota funció periòdica es pot expressar com una suma de sin/cos de diferents freqüències, cadascun ponderat per coeficients corresponents.
- **Transformada de Fourier:** encara que la funció no sigui periòdica, es pot expressar com la integral de sin/cos ponderats per les funcions corresponents.

En resum, Fourier assegura que tota funció periòdica (qualsevol senyal) no és més que una suma de sin/cos de diferents freqüències.

Si prenem la signatura de les figures i la tractem com una funció, obtenim una descripció freqüencial de la figura. I aquesta descripció podem transformar-la utilitzant Fourier. Els coeficients de Fourier que obtenim quan fem la transformada s'anomenen 'Descriptors de Fourier'. I atenent al teorema de Fourier, els primers sin/cos de la sèrie seran els que descriguin la nostra signatura de manera més genèrica i a mida que n'agafem més anirem descrivint més detalls de la imatge. Així doncs, ens podem limitar el nombre de descriptors amb què treballem decidint quantes funcions de sin/cos prenem com a descriptors. Això ens permet trobar descriptors genèrics a partir de la signatura que ens serveixin per a altres imatges.



Els descriptors de Fourier són robustos a translacions, rotacions i escales, a més, representen una forma molt compacta de guardar la informació de la forma d'una figura. Però com podem obtenir els descriptors de Fourier?

S'obtenen amb la llista ordenada de coordenades del perímetre. Per a treballar amb Fourier passarem les coordenades (x, y) a nombres complexos, així reduïrem la dimensionalitat de les dades que tractem.

$$P = x + jy \quad (2D \rightarrow 1D)$$

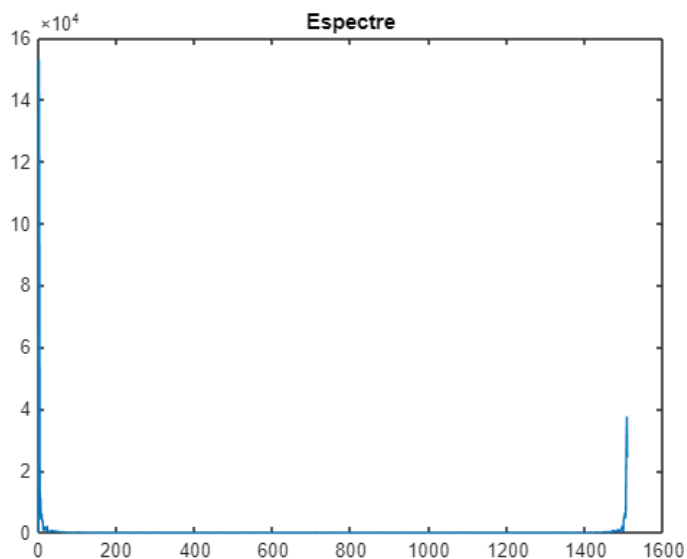
```
% Calculem el centre de la imatge
centre = mean(B);

% Obtenim la signatura
Bc(:,1) = B(:,1) - centre(1);
Bc(:,2) = B(:,2) - centre(2);

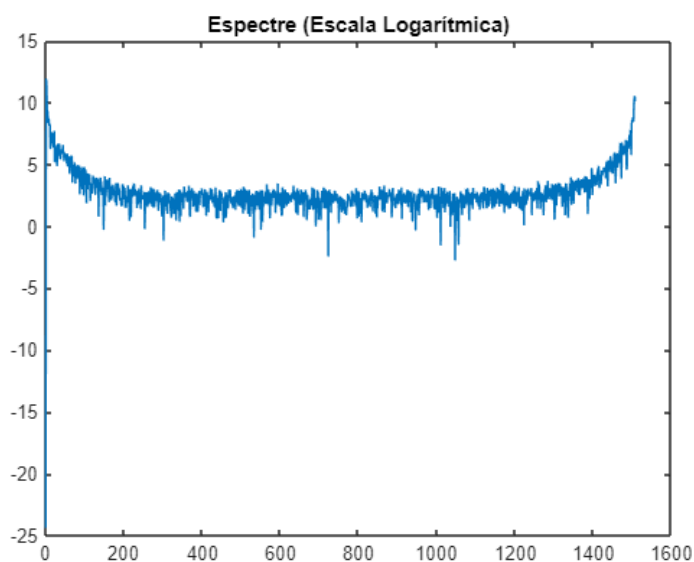
% Coords en l'espai imatge (real/imaginari)
s = Bc(:,1)+i*Bc(:,2);

% Obtenim els descriptors en l'espai freqüencial
z = fft(s);

% Les representacions acostumen a fer-se en escala logarítmica
% Els descriptors van de l'últim a la meitat, els demés son efecte miralls
figure, plot(abs(z)), title('Espectre')
```



```
figure, plot(log(abs(z))), title('Espectre (Escala Logarítmica)')
```



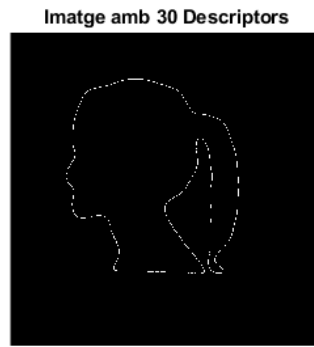
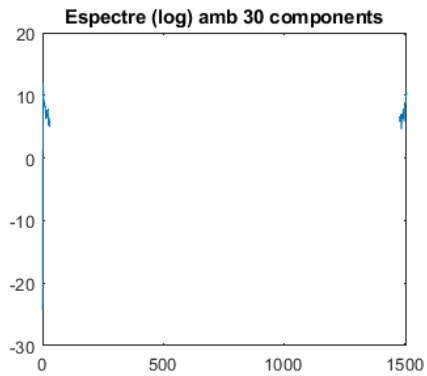
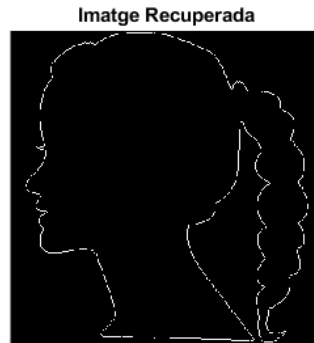
```
% Tornem les dades al món real calculant la inversa de Fourier
ss = ifft(z);
rows = round(real(ss)+centre(1));
cols = round(imag(ss)+centre(2));

aux = zeros(size(im));
aux(sub2ind(size(aux),rows,cols)) = 1;
subplot(2,2,2), imshow(aux), title('Imatge Recuperada')

% Prenem només els 30 primers descriptors
N = 30;
z2 = z;
z2(N+1:end-N) = 0;
subplot(2,2,3), plot(log(abs(z2))), title('Espectre (log) amb 30 components')
```

```
% Generem un Descriptor de la Imatge amb 30 Descriptors de Fourier
mida = 500;
ss2 = ifft(z2);
rows = round(real(ss2)+250); % Li sumem la meitat de la mida
cols = round(imag(ss2)+250);

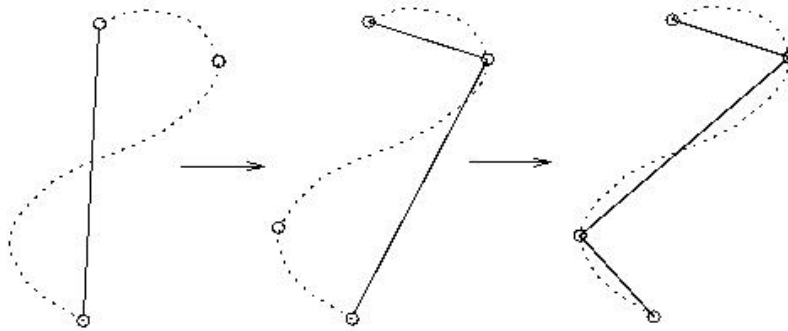
aux = zeros(mida);
aux(sub2ind(size(aux),rows,cols)) = 1;
subplot(2,2,4), imshow(aux), title('Imatge amb 30 Descriptors')
subplot(2,2,1), imshow(im), title('Imatge Original')
```



Entrega 17: Descriptors II

Aproximacions Poligonals

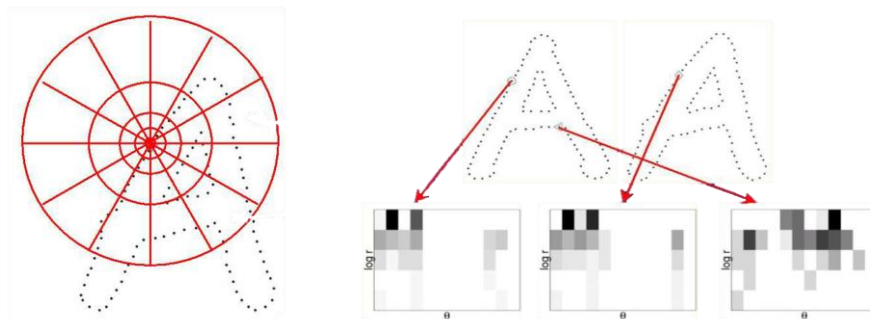
Es fan construint un codi de cadena amb punts arbitraris del contorn d'una regió i ajuntant aquests vèrtexs perquè formin un polígon. D'aquesta manera simplifiquem el contorn representant la regió de manera aproximada i amb un error controlat.



Els punts de control també es poden ajuntar amb **B-splines** en comptes de segments rectilinis.

Shape Context

Són mapes d'acumulació de píxels. Per a cada píxel del contorn, creem una "diana" i comptem quants píxels del contorn cauen a cada casella de la diana. Si ens fixem en la imatge veiem que les caselles més properes al píxel que estem tractant (el centre de la diana) són menors que les més llunyanes, això es deu a que hem de ser més rigorosos a les àrees properes.



Quan fem ús d'aquesta mena de descriptors cal prendre el nombre òptim de punts de contorn. Si ens quedem curts ens arrisquem a que no siguin representatius, però si n'agafem massa implicarà molt espai de memòria i molt temps de computació. També haurem de tenir en compte els escalats i prendre sempre el mateix nombre de punts sense importar la mida de la figura.

Aquests descriptors són robustos a oclusions, ja que tot i que una part del contorn quedi fora de visió, la resta del contorn pot seguir generant *match*.

Descripció per parts

Consisteix a dividir l'objecte en parts i representar-lo a partir de les característiques de les parts rellevants per a la identificació i les relacions entre aquestes parts. Pot barrejar elements de regions i elements de contorns. Bàsicament consisteix a fer el reconeixement de diverses parts per separat (p.ex. en un vehicle reconèixer les rodes i la matricula).

Com treballar amb grans quantitats d'informació

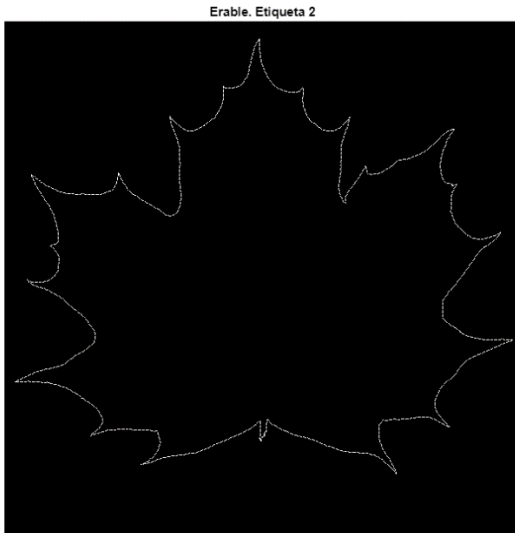
A continuació es mostra un exemple de codi per saber com tractar amb grans quantitats d'informació un cop hem obtingut diversos descriptors. El codi no té cap finalitat concreta més enllà de presentar com es maneguen aquestes dades i, al final, introduir una nova funció molt útil a l'hora d'escollir quins descriptors de Fourier són significatius per a una figura.

Les dades tractades estan emmagatzemades en una matriu i són descriptors de fulles de tres tipus d'arbre diferents. Primer reconstruïm les figures a partir dels descriptors i després crearem un vector de característiques.

```
load df_fulles.mat; % Cada fila és una fulla, cada columna un descriptor de fourier

% Classe 1
tmp = df_norm(1,:);
ss = ifft(tmp);

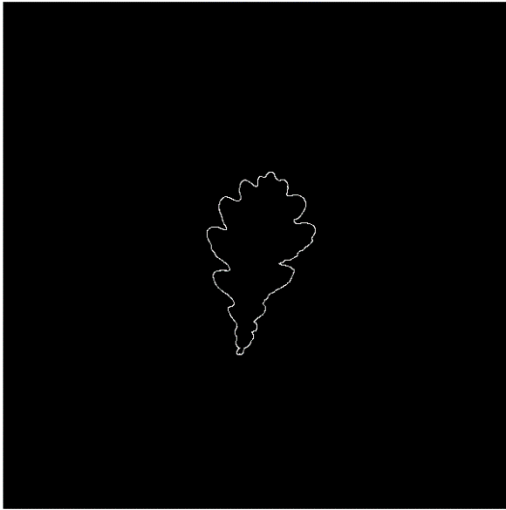
mida = 600;
files = round(real(ss) + mida/2);
cols = round(imag(ss) + mida/2);
aux = logical(zeros(mida));
aux(sub2ind(size(aux),files,cols)) = 1;
figure, imshow(aux), title('Erable. Etiqueta 2')
```



```
% Classe 2
tmp = df_norm(21,:);
ss = ifft(tmp);

mida = 600;
files = round(real(ss) + mida/2);
cols = round(imag(ss) + mida/2);
aux = logical(zeros(mida));
aux(sub2ind(size(aux),files,cols)) = 1;
figure, imshow(aux), title('Roure. Etiqueta 4')
```

Roure. Etiqueta 4

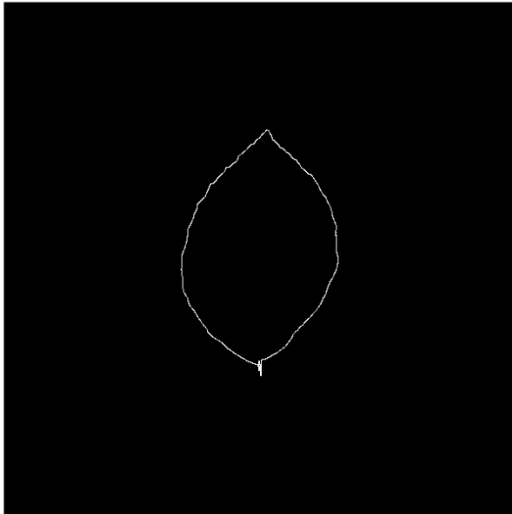


% Classe 3

```
tmp = df_norm(end, :);
ss = ifft(tmp);
```

```
mida = 600;
files = round(real(ss) + mida/2);
cols = round(imag(ss) + mida/2);
aux = logical(zeros(mida));
aux(sub2ind(size(aux),files,cols)) = 1;
figure, imshow(aux), title('Faig. Etiqueta 15')
```

Faig. Etiqueta 15



% Creem un 'feature vector' (vector de característiques)

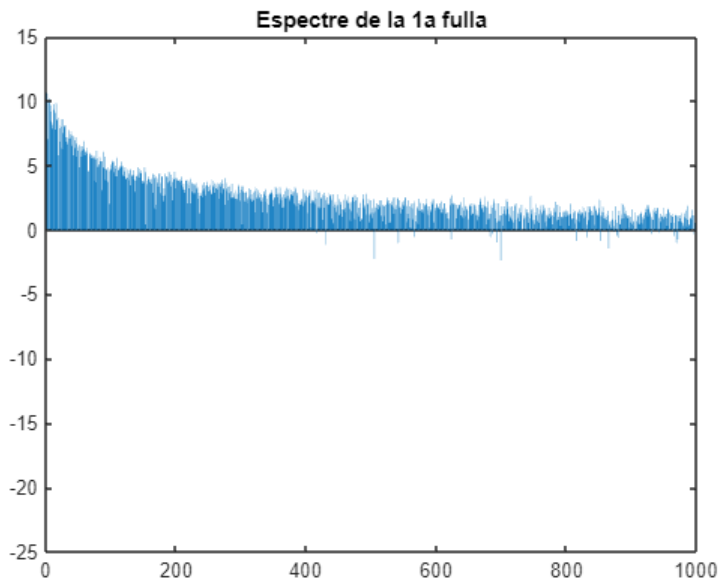
% Sabem que: df_norm.size = 48x2000

% Prenem 1000+1, ja que l'espectre de fourier està duplicat (+1 per l'etiqueta)

```
feat_vec = zeros(48, 1000+1);
feat_vec(:,1:end-1) = abs(df_norm(:,1:1000));
feat_vec(:,end) = label_tree(:);
```

% 1000 descriptors per a descriure cadascuna de les fulles

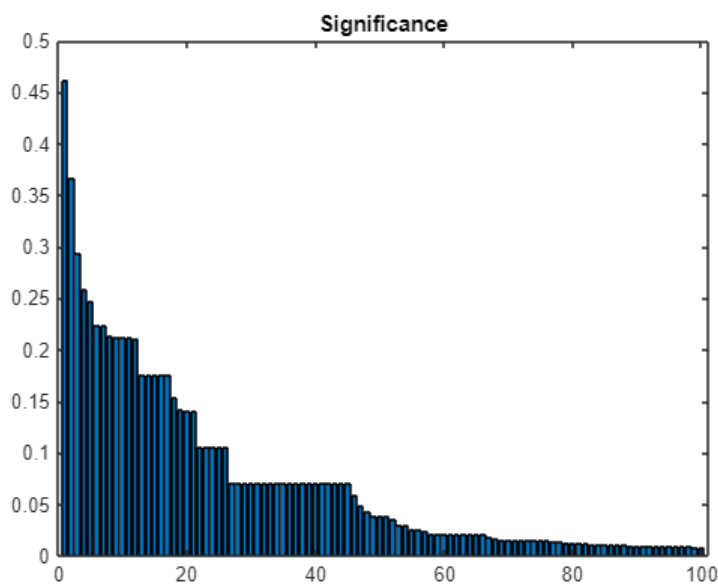
```
figure, bar(log(feet_vec(1,1:1000))), title('Espectre de la 1a fulla')
```



Finalment, utilitzem la funció *Minimum Redundancy Maximum Relevance* per a determinar quants i quins descriptors de Fourier val la pena agafar per tal de reduir al màxim la informació redundant o no rellevant. Aquesta funció assignarà a `idx_mrmr` els índex de `feat_vec` ordenats per rellevància; és a dir, si el 9è descriptor de Fourier és el més important, la primera posició de `idx_mrmr` serà un 9; i a `scores_mrmr` assignarà els valors corresponents a la rellevància (compresos en el rang $[0,1]$).

Al gràfic podem apreciar que realment només hi ha 25-45 descriptors rellevants (dependrà del grau de sensibilitat que busquem pel nostre sistema), els demés només serveixen per a afinar els detalls de les figures. Així doncs, podríem mirar quins són els descriptors que ens serveixen a `idx_mrmr` i descartar els demés.

```
[idx_mrmr, scores_mrmr] = fscmrmr(feet_vec(:,1:end-1), feet_vec(:,end));  
figure, bar(scores_mrmr(idx_mrmr(1:100))), title('Significance')
```



Introducció a la Classificació

Matching i Pattern Recognition

Finalment, després de tots els passos previs que hem estudiat fins ara, arribem a l'últim estadi de la Visió per Computador, el reconeixement de les imatges (*matching*). El *matching* consisteix a trobar coincidències entre imatges, és a dir localitzar objectes (sub-imatges) en elles, treball pel qual necessitem tenir uns models que li serviran com a referència a l'ordinador.

Per a dur a terme el reconeixement existeixen infinitat de mètodes, cadascun amb els seus avantatges i desavantatges. A continuació se'ns presenten uns quants, però abans de res cal saber que la immensa majoria de tècniques fan ús de *classes*, és a dir que classifiquen les imatges d'entrenament en grups que compleixen les mateixes característiques, aquests grups són els que podrà identificar el sistema de reconeixement amb les noves imatges.

Template matching

Consisteix a comparar dues imatges píxel a píxel. Per a això hem de tenir guardades una o diverses imatges de referència per a poder identificar l'objecte, aquestes imatges de referència s'anomenen *templates*. Per a fer aquestes comparacions, el que es fa és "situar" el *template* a totes les posicions de la imatge i fer la comparació amb totes elles. Hi ha diverses formes de mesurar la similitud entre el *template* i la imatge, algunes són:

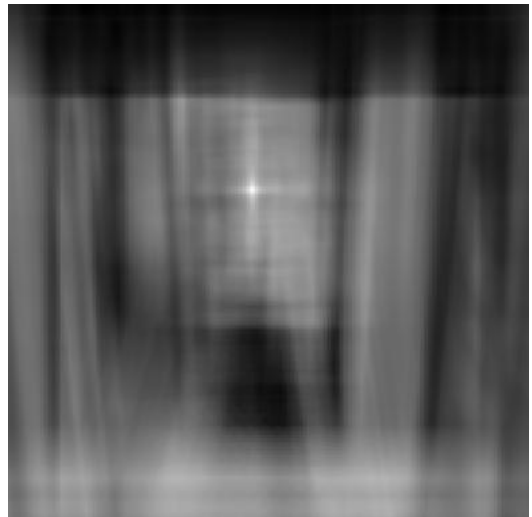
- SSD: Squared Sum Difference
- SAD: Sum of Absolute Difference



Template



Imatge



Imatge Resultant

Aquest tipus de *matching* és molt poc robust si l'objecte a la imatge està escalat, rotat o transformat d'alguna manera. La possible solució passa per tenir diverses *Templates* de l'objecte amb diferents escales, rotacions, etc. i comparar-les totes. Però com es pot intuir, aquesta és una solució molt costosa tant en espai d'emmagatzematge com computacionalment.

A més d'aquest problema, el *Template Matching* depèn massa de la il·luminació, d'oclusions, la fotografia del model, etc. Així doncs, com a conclusió podem afirmar que el *Template Matching*, tot i que pot ser útil en casos concrets, en general podem considerar-la una mala tècnica.

Pattern Recognition

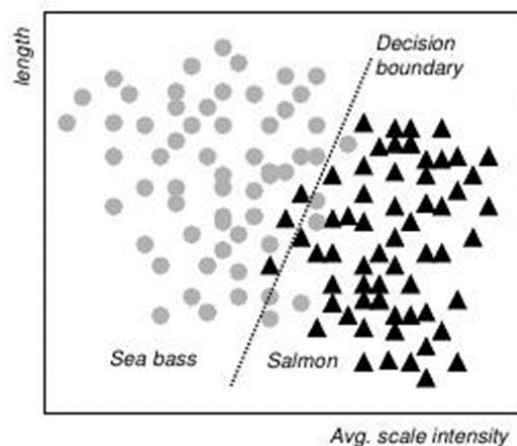
Per a obtenir bons models necessitem obtenir uns descriptors robustos als canvis i transformacions entre imatges. És per això que en comptes de treballar directament amb els píxels com es fa amb el *Template Matching*, treballarem amb espais de característiques.

Li diem *feature* o característica a qualsevol aspecte distintiu de la imatge. La combinació d' n *features* dona lloc a un vector de característiques (*feature vector*) i l'espai n -dimensional definit per aquest vector s'anomena espai de característiques (*feature space*). La combinació d'espais de característiques junt amb una etiqueta identificadora donen lloc als *patterns* (patrons).



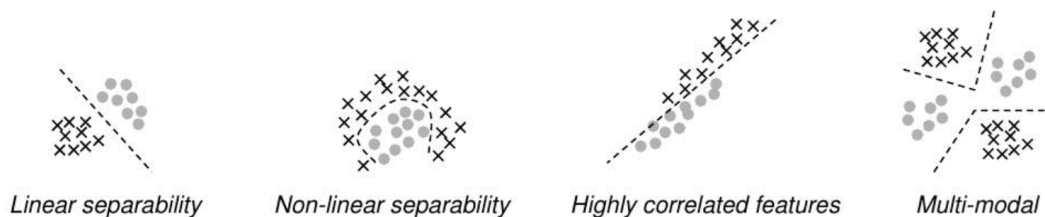
Un cop hem recollit totes aquestes dades, se les passem a un *classifier*, que ha d'identificar els objectes i classificar-los en diferents classes/categories. El classificador és l'algorisme que fent ús de les dades dels models prèviament entrenats i els *feature vectors* de la imatge a classificar assigna un input dins d'una categoria predefinida. Amb l'entrenament previ del sistema de reconeixement, hem hagut de definir quines característiques conformen cada classe.

El punt de trobada de les característiques que conformen cada classe s'anomena límit de decisió (*decision boundary*).



És important trobar unes característiques que siguin realment discriminatives, però sense caure en l'*overfitting*, l'obsessió per trobar un classificador amb un error tan mínim, que en treure'l de les imatges d'entrenament acostuma a donar mals resultats perquè ha estat entrenat massa específicament per a les dades dels models. Així doncs haurem de buscar poques característiques ben discriminatives i evitar tenir-ne moltes poc discriminatives, s'ha de trobar un equilibri entre el cost i l'error, però el cost acostumarà a tenir un pes més important a l'hora de decidir.

En separar les diferents classes tenint en compte les *features* de cadascuna, podem obtenir diferents tipus de model segons la seva separabilitat. En cas que tinguem dues regions no separables, significa que les característiques que hem pres no són suficients ni suficientment discriminatives i haurem de buscar-ne noves.



Hi ha diversos criteris per a classificar una nova entrada en el sistema, alguns d'ells són: la mitjana de classe més propera, el veïnatge més proper, els k-veïnatges més propers o els k-veïnatges més propers donant un major pes als més pròxims, també pot ser que la distància d'un objecte a la classe o als veïns sigui exageradament gran, aleshores el marcarem com a classe no identificada.

Hi ha diverses maneres de mesurar distàncies entre una mostra i un altre punt de l'espai de característiques. Quan desenvolupem un projecte, és important ser coherents amb la decisió que prenem al principi i mantenir la mateixa forma de mesurar durant tot el seu desenvolupament. Algunes de les maneres més habituals per a calcular distàncies són:

- **Distància Manhattan:** compta el nombre de cel·les que cal recórrer per arribar d'un lloc a l'altre assumint un espai de graella.

$$|x_1 - x_2| = \sum_{i=1, N} |x_1[i] - x_2[i]|$$

- **Distància Euclidiana:** la distància real si tracem una línia recta entre els dos punts de mesura.

$$||x_1 - x_2|| = \sqrt{\sum_{i=1, N} (x_1[i] - x_2[i])^2}$$

- **Distància Mahalanobis:** és una mesura que té en compte la distribució estadística de les dades i la correlació entre les variables. A la fórmula, Σ és la matriu de covariància.

$$MH(x_1, x_2) = (x_1 - x_2)^t * \Sigma^{-1} * (x_1 - x_2)$$

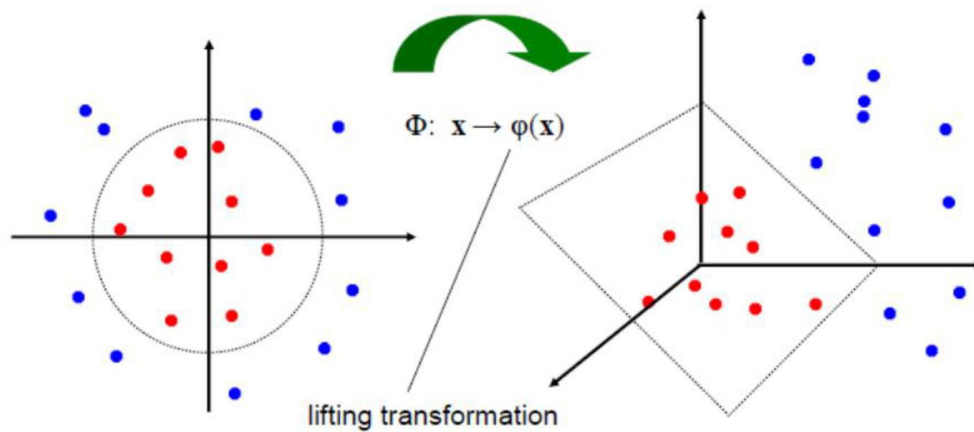
Per tant, de manera resumida, el que fa el *pattern recognition* és identificar patrons o estructures en les dades que volem reconèixer i classificar-les en classes, després, utilitzant tècniques estadístiques o de *machine learning*, s'entrena un model per a reconèixer patrons en els *feature space* de les imatges que li passem.

A continuació s'expliquen diversos tipus de models de reconeixement.

SVM (Support Vector Machines)

Un SVM és un **algorisme d'aprenentatge supervisat** que té l'objectiu de trobar un hiperplà que separi de la millor manera possible dues classes, això implica deixar el màxim marge possible entre el pla i els elements més propers de cada classe. L'algorisme permet un cert nombre de classificacions errònies a l'hora de buscar aquest pla. SVM realment està dissenyat per a problemes de classificació binària, però com generalment els problemes multi classe es poden dividir en diversos problemes binaris, també es pot aplicar a aquests.

Com es pot intuir, buscar un hiperplà només soluciona problemes que permetin una separabilitat lineal, és així que per a problemes que no compleixin aquesta característica es fa ús d'un **mètode kernel**. Els mètodes *kernel* consisteixen a passar les dades de l'estudi a una dimensionalitat major on presumiblement si que es podran separar mitjançant un hiperplà (compliran la separabilitat lineal). En el següent exemple veiem un cas on les observacions no poden separar-se linealment, per tant incrementem la dimensionalitat afegint-li la distància de les mostres del centre com a nova dimensió i es torna possible la separabilitat lineal.



Statistical Pattern Recognition

Aquests mètodes algorísmics classifiquen les imatges atenent a complexos **models probabilístics i estadístics**, cerquen d'aquesta manera les similituds que pot haver entre els *feature vectors* de l'input a classificar i les dades del model entrenat.

Decision Trees

Els arbres de decisió són models d'aprenentatge que representen decisions en forma d'arbre. Es recorre l'arbre començant per l'arrel fins que s'arriba a una fulla assegurant així la presa de decisions en un ordre específic.

- Cada **node** representa una pregunta sobre una característica de la imatge.
- Cada **branca** representa una possible resposta.
- Les **fulles** representen la classe final assignada a la imatge.

Cal esmentar que una mateixa classe pot aparèixer en múltiples fulles.

La construcció d'arbres de decisió consisteix a fer una divisió recursiva de característiques, a cada node que generem aplicarem algun criteri per a seleccionar la característica més distintiva per a dividir les dades i seguirem el procés fins a complir algun criteri de parada (cas base). La selecció de quina és la característica més distintiva es pot fer amb diversos mètodes, per exemple la impuresa de Gini o l'entropia.

Els criteris de parada poden ser:

- Totes les dades del node pertanyen a una mateixa classe (arbre generat satisfactòriament).
- La branca ha arribat a la profunditat màxima permesa i encara no ha pres una decisió final.
- Generem un node que no té dades suficients per seguir discriminant característiques.

Un dels problemes principals dels arbres de decisió és que acostumen a cenyir-se massa a les dades d'entrenament si no es poden correctament. La tècnica "Random Forest" és un mètode d'**aprenentatge de conjunt** que crea múltiples arbres de decisió i els combina per a millorar la precisió i reduir l'*overfitting*. La generació d'aquests arbres es basa en dues tècniques clau:

- **Bagging**: es creen múltiples subconjunts de dades d'entrenament de forma aleatòria i s'entrena un arbre amb cada subconjunt.
- **Selecció aleatòria de Característiques**: en comptes de considerar totes les característiques a cada node, se selecciona un conjunt aleatori d'elles. Això redueix la correlació entre els arbres i en millora la generalització.

Structural or Syntactic Pattern Recognition

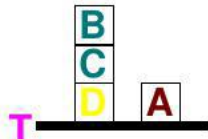
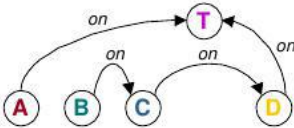
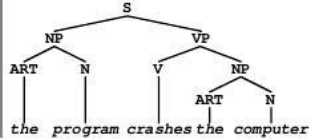
El reconeixement estructural o sintàctic es basa en la idea que els patrons poden descriure's mitjançant **estructures jeràrquiques** i **regles sintàctiques** en comptes d'utilitzar només característiques numèriques.

Els patrons d'una mateixa classe es modelen mitjançant **gramàtiques formals** o estructures similars a un llenguatge. Poden utilitzar-se gramàtiques incontextuals (CFG), atribuïdes o grafs per a definir com es combinen/relacionen les diferents components bàsiques.

Treballen amb parts i relacions entre elles, que donen lloc a una representació jeràrquica. Per exemple, una cara pot modelar-se amb diferents parts (ulls, nas i boca) i després relacionar les diferents parts segons la seva disposició espacial.

El reconeixement d'un patró passa per comprovar si una estructura generada a partir de la imatge d'entrada és reconeguda per la gramàtica del model. Aquests reconeixements fan ús d'**autòmats finits** i **arbres d'anàlisi**.

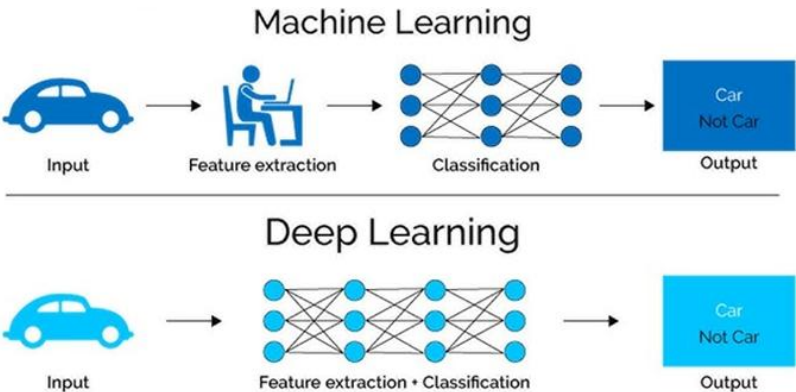
Aquests mètodes són especialment útils per a reconèixer objectes amb relacions espacials ben definides, com podrien ser una cara o un cotxe, i permeten representar variacions dins d'una mateixa classe. Com a desavantatge, poden ser altament sensibles a imatges amb soroll o deformacions importants.

Example	Model	Description
	Directed Graph	
"the program crashes the computer"	Grammar	P = { SENTENCE → NP + VP NOUN PHRASE → ART + N VERB PHRASE → V + NP NOUN → computer program VERB → crashes ARTICLE → the } 

Neural Networks

Les xarxes neuronals artificials (ANNs) s'utilitzen pel processat d'imatges des de fa dècades, però el seu impacte va incrementar-se amb l'aparició del **Deep Learning** i les xarxes neuronals convolucionals (**CNNs**).

Amb els mètodes anteriors era necessari que un humà generés un algorisme d'extracció de característiques que permetés un correcte entrenament del model i una correcta classificació de les imatges, amb aquestes xarxes però, ens podem saltar aquest pas, ja que permeten que un model extregui característiques automàticament sense necessitat d'una enginyeria manual prèvia. L'única intervenció humana amb aquests sistemes serà l'etiquetat de les imatges i, en alguns casos un preprocessat.



A més d'utilitzar-se per a la classificació d'imatges tenen una infinitat més d'utilitats com poden ser la detecció d'objectes (no només identificar què hi ha a la imatge, sinó també on està), la segmentació d'imatges, la super-resolució (augmentar la resolució d'una imatge), la generació d'imatges, etc.

Avaluar el rendiment d'un classificador

A l'hora d'avaluar l'efectivitat d'un classificador cal conèixer els següents conceptes:

- **Independent Test Data:** conjunt de dades que no han estat utilitzades durant l'entrenament i s'utilitzen a l'hora d'avaluar el seu rendiment.
- **Classification Error:** taxa de mostres mal classificades.
- **Empirical Reject Rate:** taxa de mostres no-classificades per no haver-se trobat suficients similituds amb cap classe.
- **False Positives:** en decisions binàries, nombre de casos en què el classificador marca una mostra dins d'una classe a la qual no pertany.
- **False Negatives:** en decisions binàries, nombre de casos en què el classificador marca una mostra fora de la classe a la qual pertany.
- **Precision:** mesura l'exactitud de les prediccions positives del classificador

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

- **Recall (True Positive Rate):** mesura la capacitat del classificador per a identificar correctament les instàncies positives.

$$TPR = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

- **False Positive Rate:** mesura la capacitat del classificador per a identificar correctament les instàncies negatives.

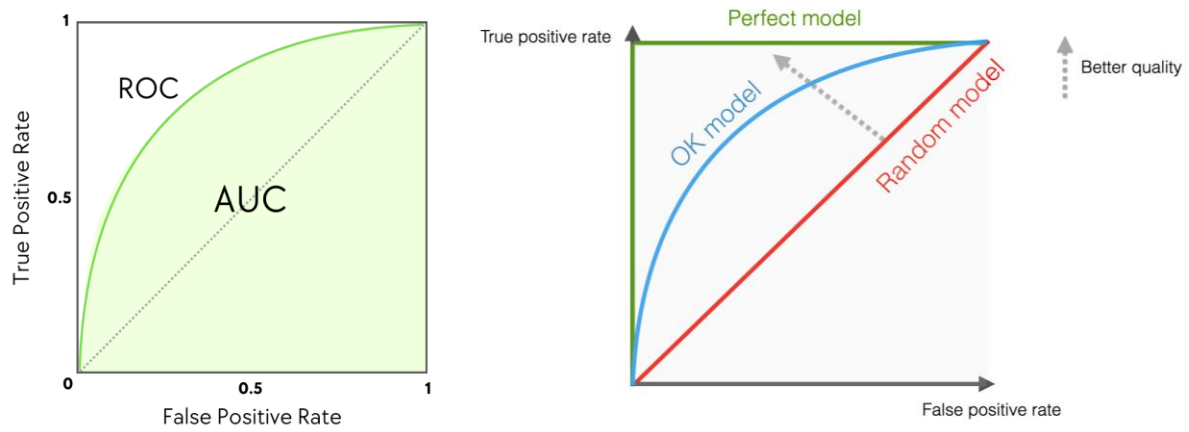
$$FPR = \frac{False\ Positives}{False\ Positives + True\ Negatives}$$

- **Llindar de decisió:** generalment un classificador retorna una probabilitat de confiança, no un veredict final, el llindar de decisió és el valor de la probabilitat a partir del qual classifiquem una mostra com a positiva o negativa.

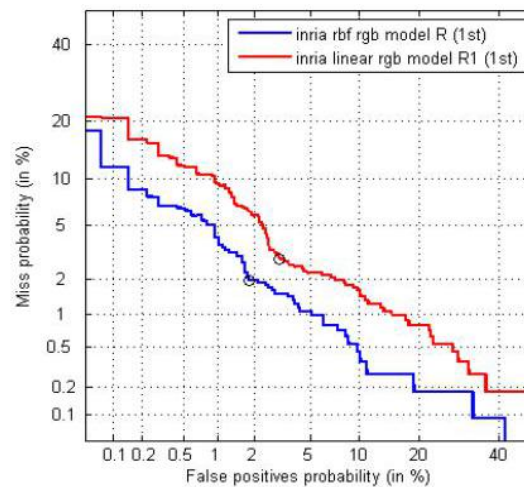
És important que a l'hora de provar un sistema per a comprovar que funciona s'utilitzin imatges no utilitzades durant l'entrenament i també que el sistema tingui una "classe no definida" per a classificar aquells objectes que consideri no pertanyents a cap classe definida.

A més d'aquests conceptes senzills, es fa ús dels següents indicadors:

- **Corba ROC:** representa la relació entre TPR i FPR a mida que variem el llindar de decisió. A l'àrea que queda per sota de la funció se li diu AUC i pot prendre valors dins del rang $[0.5, 1]$, on 0.5 correspon a un classificador random i 1 correspon a un classificador perfecte.



- **Corba DET:** és una variació de la corba ROC utilitzada per comparar taxes d'error amb major precisió. Representa la relació entre FNR i FPR.

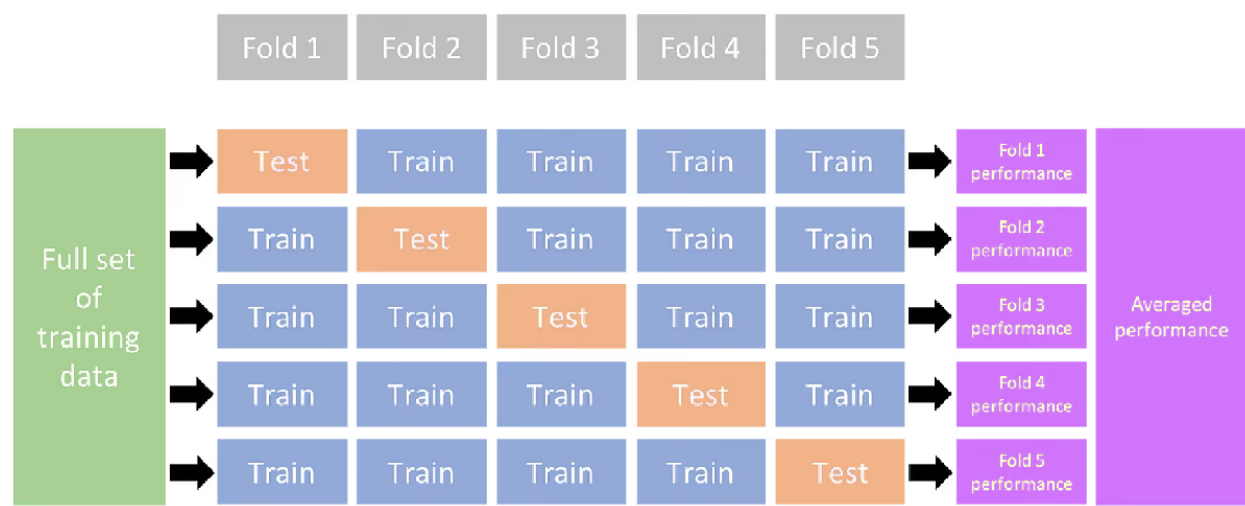


- **Confussion Matrix:** és una representació de les prediccions d'un classificador enfront de les classes reals. S'utilitza per a extreure mètriques com les ja explicades.

Testing data									
True Class \ Predicted Class	0	1	2	3	4	5	6	7	8
0	199						1	3	1
1		228				1		3	
2	1		201		3	4	3	1	2
3	1		5	179		4		2	
4		2	1	1	179	1		2	2
5	1			5	1	153	3	1	2
6			1		2	2	197		
7		1	4	1		2		191	
8				3	3	3	2		195
9			1	1	10	2		5	174

- **K-Fold Cross-Validation:** és una metodologia per a entrenar models i avaluar el seu rendiment, és bo per a detectar l'*overfitting* i especialment útil quan tenim un *dataset* d'entrenament reduït. Consisteix a dividir el

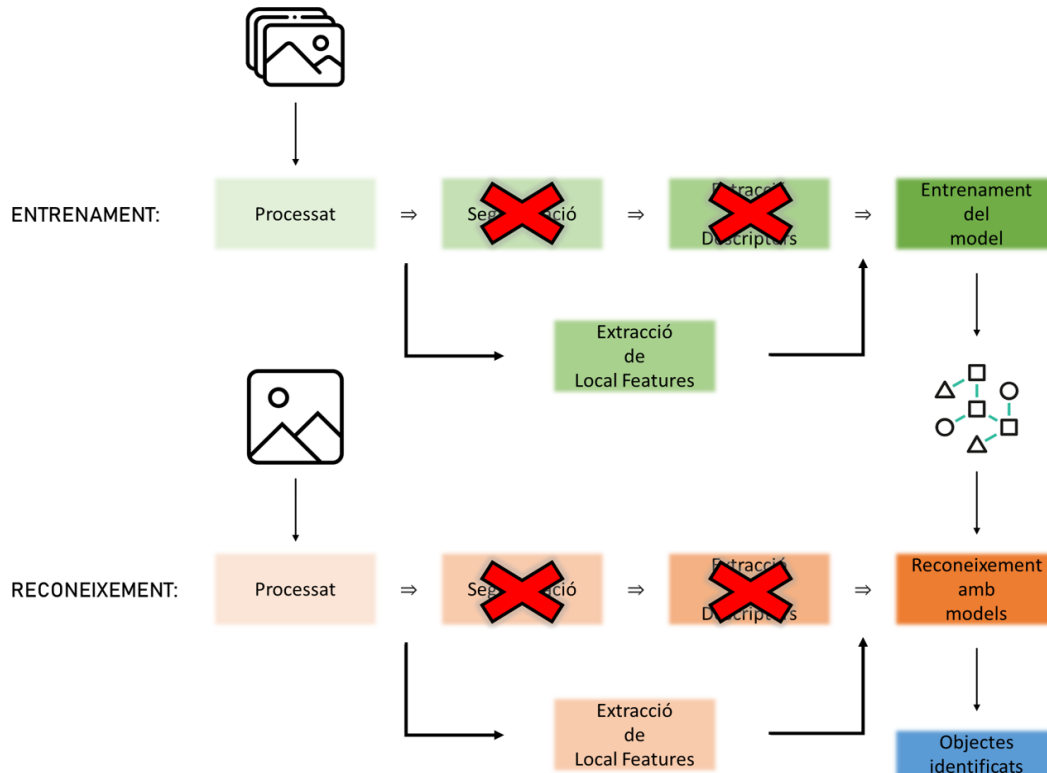
dataset d'entrenament en k sub-sets i entrenar el model k vegades. A cada iteració i utilitzarem $k - 1$ sub-sets i deixarem el sub-set i per a fer el *testing*. La fiabilitat final del model serà la mitjana dels resultats de cadascun dels *folds*.



Local Features

Tot i que els descriptors són una bona forma d'identificar imatges i entrenar models de reconeixement, si no obtenim una bona segmentació l'extracció d'uns bons descriptors pot ser extremadament complicada o fins i tot impossible. L'alternativa: no segmentar ni extreure descriptors.

La solució que presentem passa per eliminar els passos intermedis de la *pipeline* i no segmentar ni extreure descriptors. El que farem és buscar característiques directament després del processat (*Local Features*). De la mateixa manera que amb els descriptors hi havia alguns que ens resultaven particularment útils o totalment inservibles segons el que estiguéssim modelant, amb les *Local Features* passa igual. En cada cas i cada model de reconeixement que creem, haurem de trobar unes característiques particulars que ens serveixin per a aquell model concret. Òbviament, haurem d'intentar trobar característiques que siguin invariants a diferents transformacions.



Unes bones *Local Features* han de complir les següents característiques:

- **Saliency/discriminatives:** característiques locals que siguin significatives.
- **Repetitivitat:** un cop l'hem trobada hem de ser capaços de trobar-les en altres imatges.
- **Compactes:** que siguin el més tractables possible computacionalment.

A l'assignatura treballarem 5 tècniques:

- Histogrames
- Transformada de Hough
- Detecció i aparellament de Vèrtexs
- Scale Invariant Feature Transform (SIFT)
- Haar Features (face detection)

Val la pena esmentar que els *Local Features* tenen més aplicacions a més del reconeixement d'imatges, entre les que destaca l'*Image Stitching*.

Entrega 18: HOGs i Histogrames de Color

HOGs (Histograms of Oriented Gradients)

Els HOGs són poc robustos als canvis de forma i demés transformacions geomètriques, però molt robustos als canvis d'il·luminació.

S'utilitzen molt per a la detecció de persones, els procediment general en aquests casos és:

1. Descriure cada mostra (imatge) com a imatge de gradients.
2. Entrenar el SVM (Support Vector Machines) com a classificador amb els HOGs de les imatges.

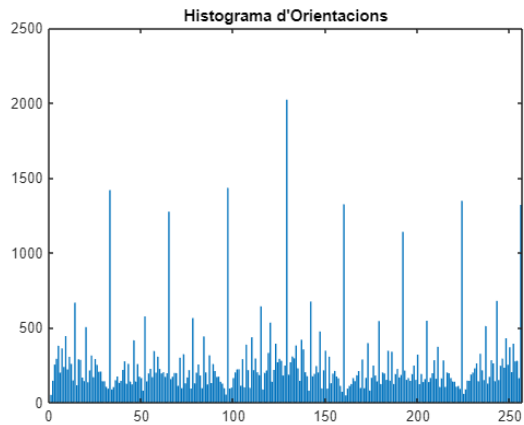
```
im = double(imread('cameraman.jpg'));  
figure, imshow(im, []), title('Imatge Original')
```



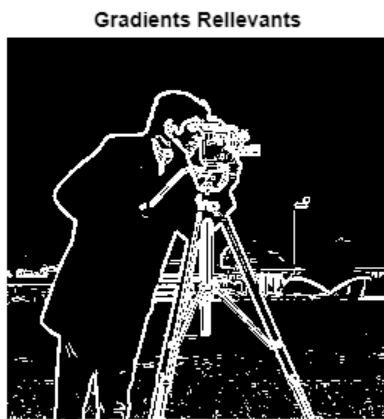
```
% Calculem els gradients amb Sobel  
sob = fspecial('sobel')/4;  
Gx = imfilter(im, sob, 'conv');  
Gy = imfilter(im, sob, 'conv');  
  
% Genereu la imatge de gradients  
alfa = uint8(255*(atan2(Gy, Gx)+pi)/2/pi);  
figure, imshow(alfa), title('Direcció del Gradient')
```



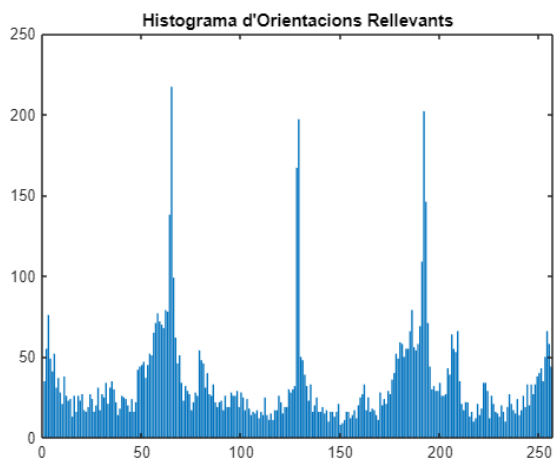
```
% Generem l'histograma
h = imhist(alfa);
figure, bar(h), title("Histograma d'Orientacions")
```



```
% Seleccionem els gradients rellevants
mod = sqrt(Gy.^2+Gx.^2);
cont = (mod > 40);
figure, imshow(cont), title('Gradients Rellevants')
```

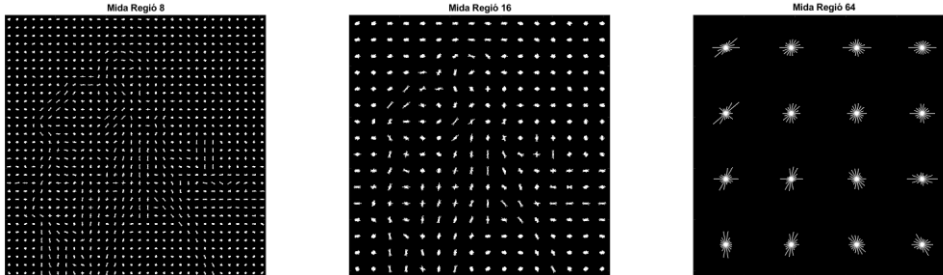


```
% Generem l'histograma de gradients rellevants
bons = alfa(cont);
h2 = imhist(bons);
figure, bar(h2), title("Histograma d'Orientacions Rellevants")
```



```
% Generem una imatge amb els HOGs de cada regió
[hog_1, vis_1] = extractHOGFeatures(im, 'CellSize', [8,8]);
[hog_2, vis_2] = extractHOGFeatures(im, 'CellSize', [16,16]);
[hog_3, vis_3] = extractHOGFeatures(im, 'CellSize', [64,64]);

subplot(1,3,1), plot(vis_1), title('Mida Regió 8')
subplot(1,3,2), plot(vis_2), title('Mida Regió 16')
subplot(1,3,3), plot(vis_3), title('Mida Regió 64')
```



Histogrames de Color

A diferència dels HOGs, els histogrames de colors són molt robustos a deformacions i poc robustos a canvis d'il·luminació, no són totalment invariants a les transformacions geomètriques (els canvis de reflexivitat poden fer que canviïn) però sí que són molt robustos. Un altre punt a favor d'aquest mètode és que és robust a oclusions.

Tot i que ser histogrames, hem de tenir en compte que se surten de la clàssica representació bidimensional de barres ja que són tridimensionals (RGB).

Com ja hem vist amb anterioritat, una de les característiques importants de les *Local Features* és que siguin compactes per a que siguin tractables computacionalment. Per això en comptes de generar histogrames 256x256x256, que seria la forma més intuïtiva de fer-ho (cada dimensió del RGB pot prendre valors dins del rang [0, 255]), treballarem amb histogrames 16x16x16.

Generalment abans de generar l'histograma normalitzem els colors per a obtenir resultats més acurats que no depenguin tant de la il·luminació.

```
im = imread('team1.jpg');
figure, subplot(2,3,1), imshow(im), title('Model')

% Transformem a double per a treballar millor
im = double(im);

% Normalitzem l'espai RGB (sumem 1 per a evitar dividir entre 0)
imax = im ./ (im(:,:,1)+im(:,:,2)+im(:,:,3)+1);
subplot(2,3,2), imshow(imaux), title('Imatge Normalitzada')

% Generem l'histograma. Mostra quants cops apareix cada píxel amb el valor
% (r,g) concret
h1 = histcounts2(imaux(:,:,1), imaux(:,:,2), 16);
subplot(2,3,3), mesh(h1), title('Histograma r-g')

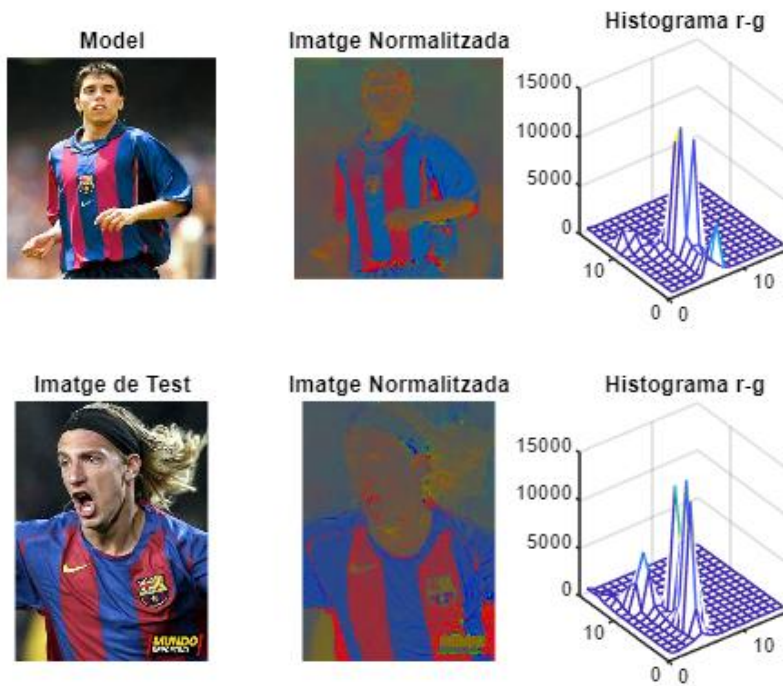
% Normalitzem l'histograma
h1 = h1/sum(h1, 'all');
```

```
% -----
% Repetim tot el procés
im2 = imread('team4.jpg');
subplot(2,3,4), imshow(im2), title('Imatge de Test')

im2 = double(im2);

imaux = im2 ./ (im2(:,:,1)+im2(:,:,2)+im2(:,:,3)+1);
subplot(2,3,5), imshow(imaux), title('Imatge Normalitzada')

h2 = histcounts2(imaux(:,:,1), imaux(:,:,2), 16);
subplot(2,3,6), mesh(h2), title('Histograma r-g')
```



```
h2 = h2/sum(h2, 'all');
% -----

% Trobem la similitud entre els histogrames de color
aux = min(h1, h2);
sim = sum(aux, 'all')
```

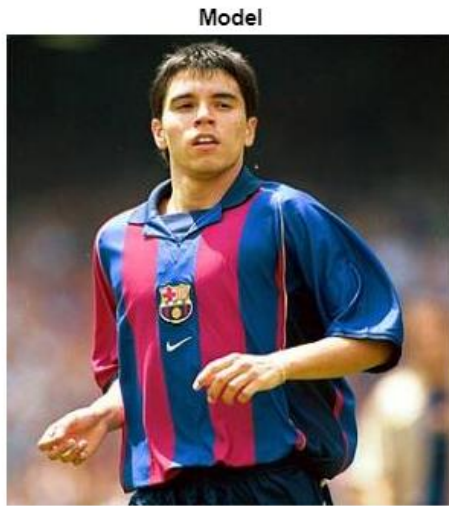
```
sim = 0.5514
```

```
clear
```

Exercici

Exercici (Selecció Manual): seleccionant les regions d'interès manualment, cal implementar un algorisme que trobi quina de les imatges correspon a un jugador amb samarreta del mateix equip que el del model. Les coordenades de la regió de referència del model seran: [248:272, 92:156].

```
im = imread('team1.jpg');  
figure, imshow(im), title('Model')
```

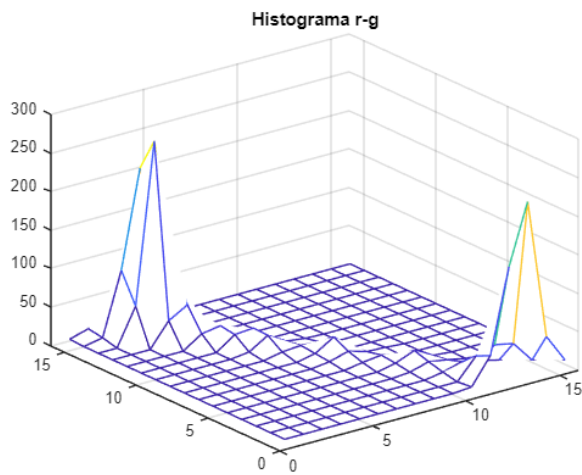


```
% Transformem a double per a treballar millor  
im = double(im);  
  
% Normalitzem l'espai RGB  
imaux = im ./ (im(:,:,1)+im(:,:,2)+im(:,:,3)+1);  
  
% Prenem la regió indicada per l'enunciat  
patch = imaux(248:272, 92:156, :);  
figure, imshow(patch), title('Patch Model (RGB Normalitzat)')
```

Patch Model (RGB Normalitzat)



```
% Generem l'histograma  
h1 = histcounts2(patch(:,:,1), patch(:,:,2), 16);  
figure, mesh(h1), title('Histograma r-g')
```



```
% Histograma del patch d'interès a comparar
h1 = h1/sum(h1, "all");

% -----

for i=[4,6,9,11,12]
    % Llegim la imatge
    imatge = sprintf('team%d.jpg',i);
    im = imread(imatge);

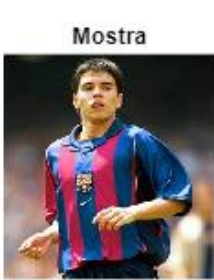
    % Prenem el patch manualment
    patch = double(imcrop(im));

    % Normalitzem l'RGB del patch
    patch = patch ./ (patch(:,:,1)+patch(:,:,2)+patch(:,:,3)+1);

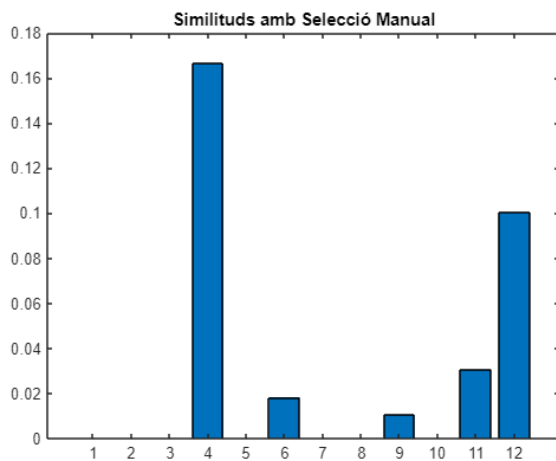
    % Creem l'hitograma del patch
    h = histcounts2(patch(:,:,1), patch(:,:,2), 16);
    h = h/sum(h, "all");

    % Busquem les similituds
    aux = min(h1, h);
    sim(i) = sum(aux, 'all');
end

figure
subplot(2,3,1), imshow('team1.jpg'), title('Mostra')
subplot(2,3,2), imshow('team4.jpg'), title('Im 4')
subplot(2,3,3), imshow('team6.jpg'), title('Im 6')
subplot(2,3,4), imshow('team9.jpg'), title('Im 9')
subplot(2,3,5), imshow('team11.jpg'), title('Im 11')
subplot(2,3,6), imshow('team12.jpg'), title('Im 12')
```



```
figure, bar(sim), title('Similituds amb Selecció Manual')
```



Exercici (Selecció Automàtica): repetir l'exercici anterior però sense seleccionar manualment les regions d'interès.

```
for i=[4,6,9,11,12]
    % Llegim la imatge i n'extraiem la mida
    imatge = sprintf('team%d.jpg',i);
    im = imread(imatge);
    [rows, cols, ~] = size(im);

    % Partirem la imatge en 16 regions
    for j=0:3
        for k=0:3
            % Creem els 16 patch de la imatge
            patch = double(im(j*floor(rows/4)+1 : (j+1)*floor(rows/4), ...
                             k*floor(cols/4)+1 : (k+1)*floor(cols/4), ...
                             1:end));

            % Normalitzem l'RGB del patch
```

```

patch = patch ./ (patch(:,:,1)+patch(:,:,2)+patch(:,:,3)+1);

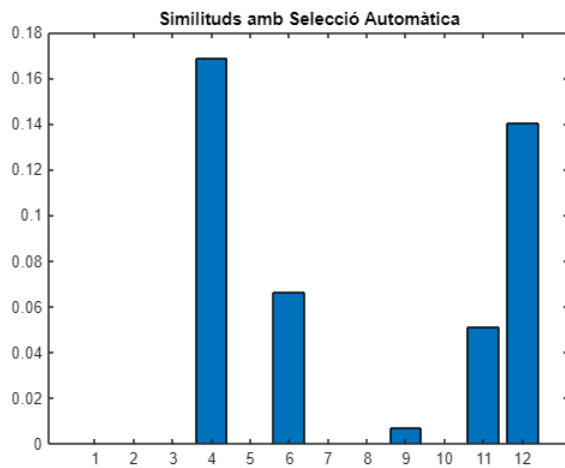
% Generem l'histograma del patch
h = histcounts2(patch(:,:,1), patch(:,:,2), 16);
h = h/sum(h, "all");

% Busquem la similitud amb la regió del model
aux = min(h1, h);
s = sum(aux, 'all');

% En cas que sigui més similar que els anteriors, el prenem
if j ~= 0 && k ~= 0
    if s > sim(i)
        sim(i) = sum(aux, 'all');
    end
else
    sim(i) = sum(aux, 'all');
end
end
end
end

figure, bar(sim), title('Similituds amb Selecció Automàtica')

```



Entrega 19: Transformada de Hough

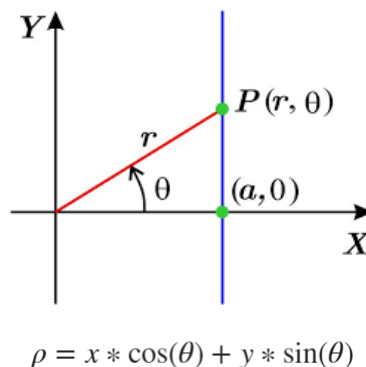
La Transformada de Hough és una tècnica utilitzada per a detectar formes geomètriques parametritzables (que es puguin expressar en funció d'uns paràmetres/variables) p.ex. rectes o cercles. Consisteix a convertir els punts d'una imatge a l'espai de Hough (que serà l'espai de característiques dels paràmetres de la forma).

A continuació mostrem un exemple de quins serien els passos a seguir per a detectar rectes. Recordem que l'equació d'una recta pot expressar-se: $y = mx + b$.

Algorisme de la Transformada de Hough:

1. Crear una matriu de Hough bidimensional (m,b) on cada posició de la matriu representa un acumulador de vots.
2. Per a cada píxel (x,y) de la imatge de contorns, incrementar les posicions (m,b) que satisfan l'equació. Això significa que per a cada píxel en blanc que trobem, haurem d'afegir un vot a cada cel·la de la matriu que representi una recta que passa per aquell píxel.
3. Cercar els màxims regionals, que ens indicaran els valors dels paràmetres que descriuen la nostra recta.

Però com dimensionem la pendent a l'hora de generar la matriu de Hough? Està clar que b pot prendre valors dins del rang $[0, \text{alçada de la imatge}]$ però la m pot prendre infinits valors i això no és representable computacionalment. Cal afitar la pendent per a que sigui representable, així doncs en comptes de treballar amb l'equació habitual de la recta, utilitzarem la seva forma polar:

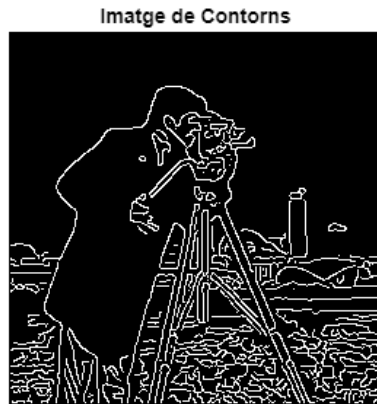


Podem aplicar una millora a l'hora d'implementar la transformada Hough si ens adonem que realment no cal generar provar totes les rectes per a cada punt (x,y), ja que tenint els contorns coneixem la direcció del gradient i per tant ja tenim θ que serà perpendicular.

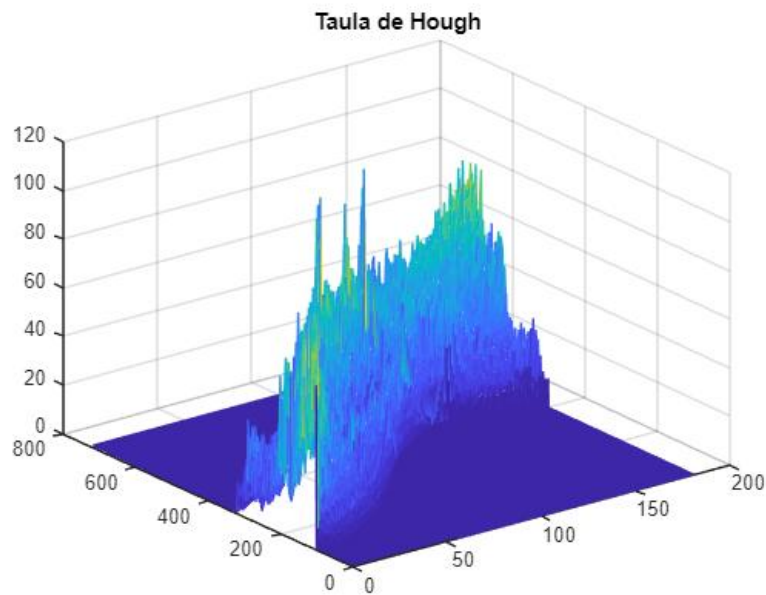
```
im = imread('cameraman.jpg');  
figure, imshow(im), title('Imatge Original')
```



```
bw = edge(im,'canny');
figure, imshow(bw), title('Imatge de Contorns')
```



```
[H, alfa, rho] = hough(bw); % Utilitzem els parametres per defecte
figure, mesh(H), title('Taula de Hough')
figure, imshow(H, []), title('Taula de Hough')
```



```
% Identifiquem els màxims de la taula
figure, imshow(mat2gray(H), 'XData', alfa, 'YData', rho, 'InitialMagnification','fit')
xlabel('\alpha'), ylabel('\rho'), axis on, axis normal

[fil, col] = find(H==max(H(:)))
```

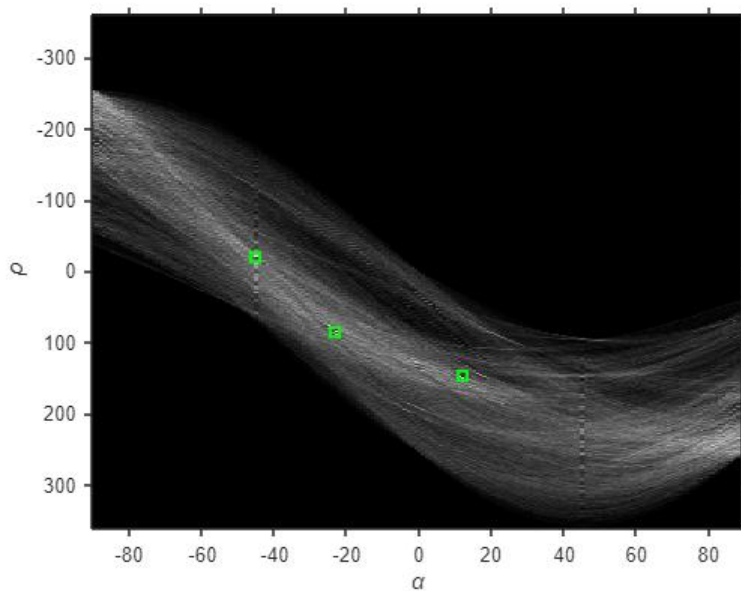
```
fil = 341
col = 46
```

```
hold on
x = alfa(col);
y = rho(fil);
plot(x, y, 's', 'Color','r', 'LineWidth', 2)
```

```
P = houghpeaks(H, 3) % Aquesta funció ja fa filtratge
```

```
P = 3x2
    341     46
    446     68
    508    103
```

```
hold on
x = alfa(P(:,2));
y = rho(P(:,1));
plot(x, y, 's', 'Color', 'g', 'LineWidth', 2)
```



```
% Prenem els valors màxims i dibuixem les rectes que representen
lines = houghlines(bw, alfa, rho, P);
figure, imshow(im), title('3 rectes principals')
hold on
for k=1:length(lines)
    xy=[lines(k).point1;lines(k).point2];
    plot(xy(:,1), xy(:,2), 'LineWidth',2, 'Color','r')
end
```

3 rectes principals



Hough és un mètode molt robust a oclusions, com podem observar en aquesta imatge.

De la mateixa manera que podem identificar rectes, podem utilitzar Hough per a identificar cercles de mides diverses. Recordem que l'equació del cercle és: $r^2 = (x - a)^2 + (y - b)^2$ per tant la taula de Hough en aquest cas haurà de ser tridimensional, ja que depenem de 3 paràmetres: x , a i b .

Si coneixem el radi dels cercles que busquem o el rang de valors que pot prendre, podem estalviar-nos una dimensió a l'espai de Hough o reduir-ne la mida.

```
im = imread('coins.png');  
figure, imshow(im), title('Imatge Original')  
  
% center, radi, metric  
[centres, radis, H] = imfindcircles(im, [15 30]); % Funció implementada amb Hough  
  
hold on  
viscircles(centres(1:5,:), radis(1:5), 'EdgeColor', 'b');
```

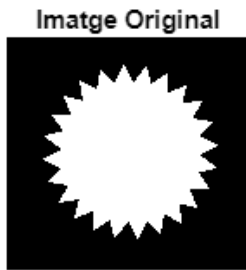


Exercici Individual 2

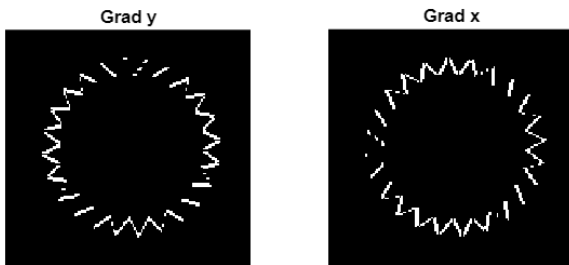
Trobar R ("Corners Measure") de la imatge.

Cal usar supressió de no-màxims i, si cal, llindaritzar per a trobar els vèrtex.

```
im = imread('gear.tif');  
figure, imshow(im), title('Imatge Original')
```



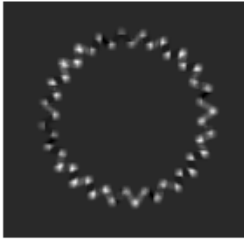
```
% Calcul de les derivades en x i y  
Sy = fspecial("sobel")/4;  
Gy = imfilter(im,Sy,'conv');  
Gx = imfilter(im,Sy','conv');  
  
figure  
subplot(1,2,1), imshow(Gy), title('Grad y')  
subplot(1,2,2), imshow(Gx), title('Grad x')
```



```
% Calculs intermitjos  
Gx_sq = Gx.^2;  
Gy_sq = Gy.^2;  
Gxy = Gx.*Gy;  
  
kernel = ones(5);  
kernel = kernel/25;  
  
% Sumatori de concolucio  
SGx_sq = imfilter(Gx_sq,kernel,"replicate",'conv');  
SGy_sq = imfilter(Gy_sq,kernel,"replicate",'conv');  
SGxy = imfilter(Gxy,kernel,"replicate",'conv');  
  
% Corneress Measure  
R = (SGx_sq.*SGy_sq - SGxy.^2) - 0.04*(SGx_sq + SGy_sq).^2;
```

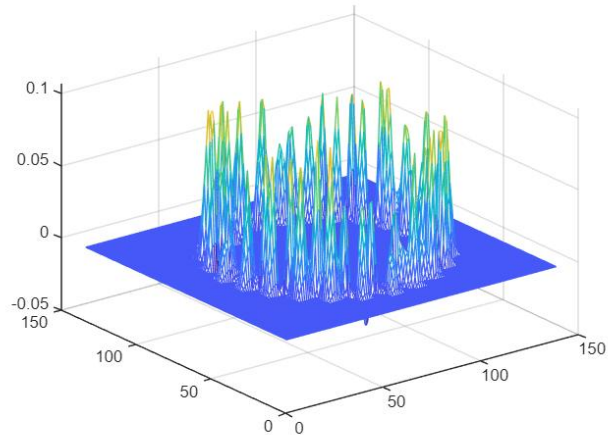
```
figure, imshow(R, []), title('Corners Measure')
```

Corners Measure



```
figure, mesh(R), title('Corners Measures 3D')
```

Corners Measures 3D

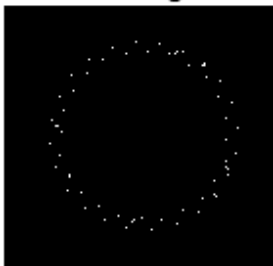


```
% Supressió de no-màxims
```

```
rm = imregionalmax(R);
```

```
figure, imshow(rm), title('Màxims Regionals')
```

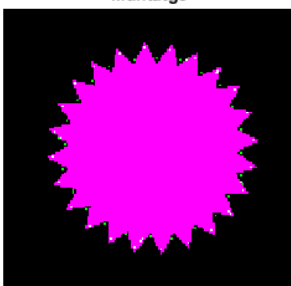
Màxims Regionals



```
res = imfuse(rm, im);
```

```
figure, imshow(res), title('Muntatge')
```

Muntatge



Entrega 20: Keypoints I

Els keypoints són punts d'interès en una imatge que són fàcilment recognoscibles, punts de la imatge "únics" o "inusuals" (saliency). Són un mètode invariant a rotacions i translacions i molt robust a oclusions, canvis d'escala i canvis d'il·luminació.



Els estadis per a identificar imatges mitjançant keypoints són:

1. **Detecció** de Keypoints (Detector de Harris)
2. **Descriure**'ls (Descriptors SIFT)
3. **Aparellament** de keypoints
4. **Matching** global d'objectes

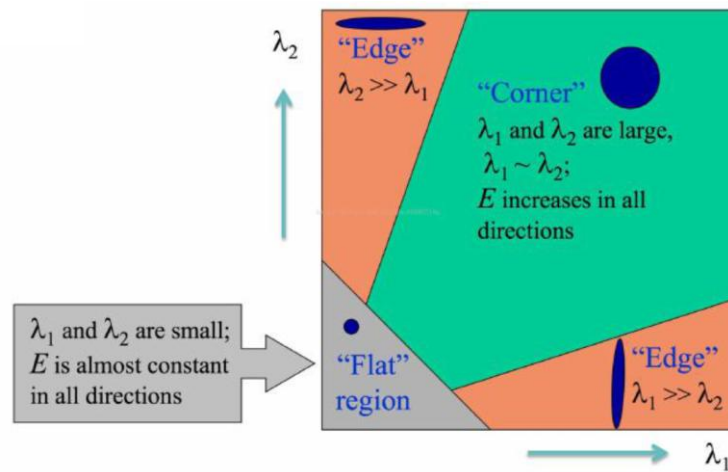
Detector de Harris

Per a trobar els keypoints (kp d'ara en endavant) cal buscar parts de la imatge "inusuals" (*saliency*), normalment seran vèrtexs. Buscarem derivades, als vèrtexs els gradients apuntaran en totes direccions, en canvi als contorns i a les zones planes apuntaran en una o cap direcció. Per tant, allà on hi hagi derivada dx i dy hi considerarem un vèrtex. Per a detectar-los utilitzarem la següent matriu, prèviament tractada amb un filtre de suavitzat per a evitar una alta sensibilitat al soroll.

$$C = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \quad \text{on } I_x \text{ i } I_y \text{ són els gradients.}$$

Però ens sorgeix un problema: els contorns diagonals (aquells no alineats amb els eixos de derivació) també donaran resultats tant en dx com en dy . Per a solucionar aquest obstacle diagonalitzarem la matriu (equival a alinear els eixos principals). Així doncs obtindrem una matriu amb forma: $C = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$. Un cop diagonalitzada podem prescindir dels vectors propis (veps) i treballar només amb els valors propis (vaps).

- Si $\text{vap1} \gg \text{vap2}$ o $\text{vap2} \gg \text{vap1} \implies$ ens trobem davant d'un contorn horitzontal.
- Si tots dos tenen valors molt baixos $\implies$ ens trobem davant d'una zona plana.
- Si tots dos tenen variacions $\implies$ ens trobem davant d'un vèrtex.

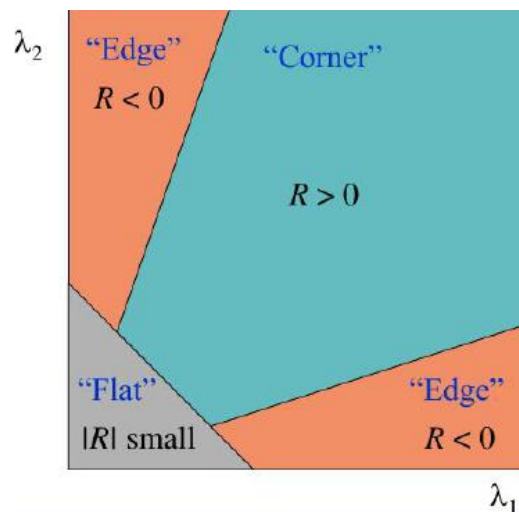


Però quin criteri seguim per a decidir si tots dos vaps són suficientment grans? Utilitzarem el 'Harris Corner Detector' (Cornerness Measure).

$$R = \det(M) - k * (\text{trace } M)^2 \quad \text{on} \quad \det(M) = \lambda_1 \lambda_2$$

$$\text{trace}(M) = \lambda_1 + \lambda_2$$

$$k \in [0.04, 0.06]$$



Fins aquí hem presentat els principis matemàtics pels que hauria de funcionar la detecció de vèrtexs, però ens adonem que implementar aquest procés és computacionalment molt costós ja que trobar els vaps d'una matriu és molt car. És així que Harris demostra el següent, que ens evita haver de calcular els vaps:

$$R = \left(\sum I_x^2 * \sum I_y^2 - \left(\sum I_x * I_y \right)^2 \right) - k * \left(\sum I_x^2 + \sum I_y^2 \right)^2$$

Així doncs els passos que segueix el Harris Corner Detector són:

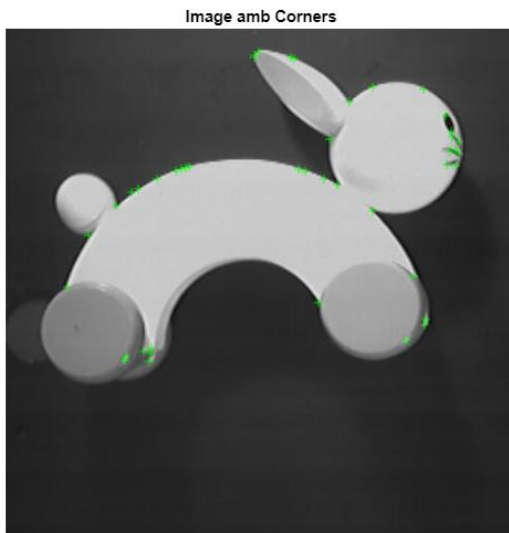
1. Computar les derivades I_x i I_y de la imatge d'entrada.
2. Computar els quadrats I_x^2 , I_y^2 i el producte de les derivades $I_x I_y$.
3. Per a cada píxel, computar la matriu M per a una regió de veïns.
4. Calcular R per a cada píxel.
5. Limitar els valors de R per a seleccionar els vèrtexs.
6. Fer supressió de no-màxims per a eliminar soroll.

A l'hora d'aplicar Harris podem jugar amb els següents paràmetres: k , el *threshold* de binarització i la mida dels sumatoris de les convolucions de la matriu.

```
im = imread('rabbit.jpg');  
figure, imshow(im), title('Image Original')
```



```
corners = detectHarrisFeatures(im); % Prenem valors per defecte  
  
% Ja tenim les coordenades amb punts d'interès  
figure, imshow(im), title('Image amb Corners')  
hold on, plot(corners)
```



Podem determinar doncs que la detecció de Harris:

- És invariant a rotacions (alineem amb el vap).
- És invariant a canvis d'il·luminació.
- No és invariant a canvis d'escala, ja que el concepte de vèrtex no es invariant a l'escala (p.ex. un pic a un mapa és un vèrtex fins que l'ampliem i es va tornant un contorn) així doncs és un problema de naturalesa irresoluble, no un problema de l'algorisme. Possible solució: buscar vèrtexs a models amb diferents escales.

SIFT (Scale Invariant Feature Transform)

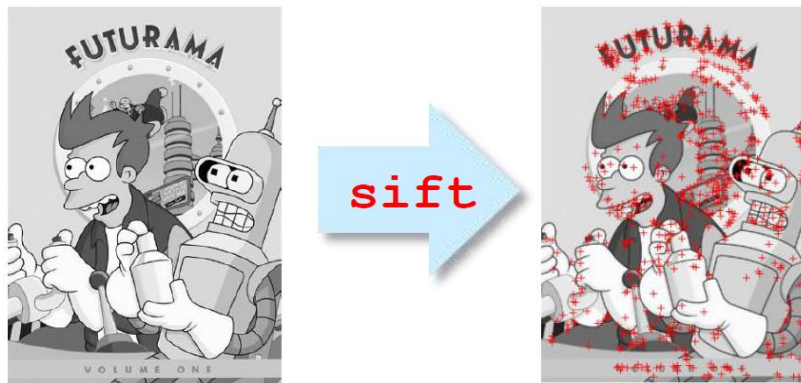
Ja sabem com detectar vèrtexs en una imatge, per tant podem obtenir kp de dues imatges diferents. Ara toca aprendre com relacionar-los per fer l'aparellament d'imatges i decidir si són o no una imatge del mateix objecte. Cal que aquest aparellament sigui invariant a translacions, rotacions, canvis d'il·luminació, etc. així doncs, el *template matching* no és una opció satisfactòria.

Per a ser capaços d'aparellar-los, haurem de trobar descriptors dels kp. Aquí entren en joc els descriptors SIFT. Per a obtenir els descriptors SIFT hem de:

1. Prendre una finestra de 16x16 píxels centrada al kp.
2. Dividir aquesta finestra en cel·les 4x4, obtenint així 16 cel·les.
3. Per a cada cel·la, construir un HOG de 8 direccions.
4. Concatenar els 16 HOGs en un vector de 128 dimensions.

Els descriptors SIFT són robustos a canvis d'il·luminació i de perspectiva, ja que es normalitzen per a reduir l'efecte de variacions de brillantor.

Els inventors d'aquests descriptors també van presentar un detector SIFT, però el de Harris dona millors resultats.

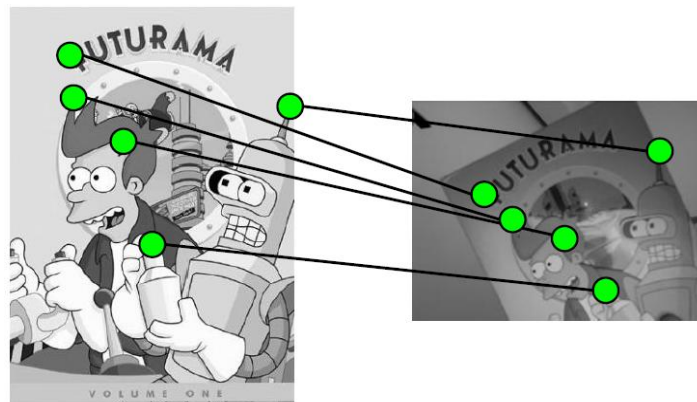


Aparellament de Keypoints

Per a fer l'aparellament de kp es fa un comparen tots els kp trobats a la Imatge 1 amb tots els kp trobats a la Imatge 2 (producte cartesià). S'ha d'establir una mètrica per a calcular la distància/diferència entre descriptors i definir un llindar de decisió per a determinar si són o no equivalents dos kp.

Algunes de les mètriques més habituals són:

- **SSD**: Squared Sum Difference
- **SAD**: Sum of Absolute Differences

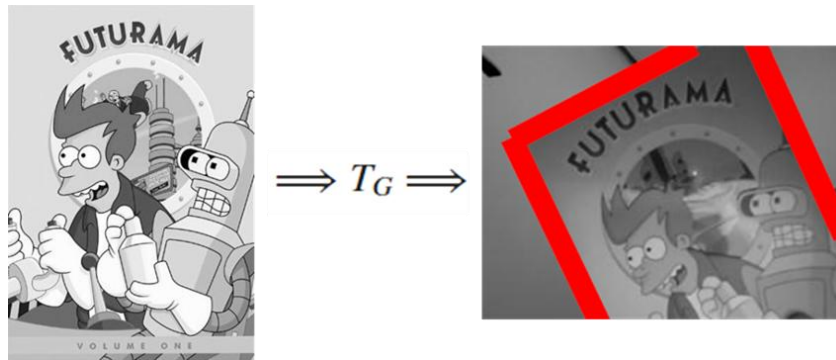


En cas que el nostre objectiu fos només reconèixer si un determinat objecte de la Imatge 1 és present a la Imatge 2, un cop arribats aquí ja tenim suficient informació per a decidir-ho. Ara podríem analitzar les diferències de descriptors dels kp aparellats i la proporció i distribució d'aparellaments en la imatge per a determinar si la imatge conté o no el/s element/s a reconèixer.

Matching Global

Els kp, a més del reconeixement, tenen altres utilitats, per exemple l'*Image Stitching* o la detecció d'objectes dins d'una imatge (no només reconèixer si apareix, sinó que a més s'indica on apareix). En cas que vulguem dur a terme alguna d'aquestes coses, requerirem un *matching* global de la imatge.

Un cop aparellats els kp, cal trobar una matriu de transformació T_G que mapegi els píxels de la Imatge 1 a la Imatge 2 i resoldre el sistema d'equacions que pertogui.



La qüestió és que per a generar una matriu, ens calen només tres punts, si necessitem una transformació afí (rotació, translació i escala) o quatre si necessitem una transformació projectiva (rotació, translació, escala i perspectiva). La solució fàcil és prendre els tres/quatre aparellaments amb més similituds dels que ja haguem detectat a la fase d'aparellament. Però és una tècnica amb molta baixa fiabilitat, ja que ens arisquem a que dos descriptors de kp totalment diferents siguin molt similars. Així doncs, ens convé prendre múltiples aparellaments. Volem aparellar els punts correctament de manera no assistida, doncs, necessitem algorismes que generin aparellaments robustos. Haurem de filtrar els incorrectes per a garantir una coherència global.

A continuació veurem dues estratègies, la **Transformada de Hough** i **RANSAC**. La transformada de Hough no és un mètode que garanteixi l'èxit, ja que tot i tenir una fiabilitat alta, pot donar resultats incorrectes, el mètode RANSAC en canvi, és computacionalment més costós però té una taxa major d'èxit (segueix sense garantir un 100% d'encerts). S'ha de valorar, depenent de quines siguin les característiques del nostre sistema i el problema a resoldre, quina tècnica cal utilitzar.

Transformada de Hough

En comptes de tenir una sola taula de Hough en tindrem dues tridimensionals, en cas d'una transformació afí, o tetradimensionals, en cas d'una transformació projectiva (una taula per a cada equació, i una dimensió per incògnita). Provarem a fer les matrius de translació amb tots els conjunts possibles de 3 keypoints i incrementarem les taules en els punts pertinents. En acabar el procés, els màxims de les taules ens indicaran les T_G adequades.

És un mètode sensible al soroll i per tant no garanteix l'èxit de l'aparellament global, ja que pot passar que hi hagin molts *outliers* (aparellaments erronis) i tenir la mala sort que votin consistentment una mateixa T_G .

RANSAC

Aquest mètode té una component aleatòria i per tant, segueix existint la possibilitat de fallar. El seu principal atractiu és que filtra molt bé dades que tinguin molts *outliers*.

RANSAC assumeix que les dades d'entrada estan conformades per:

1. **Inliners**: dades que s'ajusten al model desitjat i estan afectades per un soroll nemi.
2. **Outliers**: dades que no s'ajusten al model desitjat degut a errors, soroll extrem o característiques alienes al fenomen a modelar.

El seu objectiu és trobar el model que millor s'ajusti als *inliners* ignorant els *outliers*. Els passos de l'algoritme són:

1. **Inicialització**: se selecciona un nombre fix N d'iteracions que defineix quantes vegades es repetirà el procés.
2. **Mostreig aleatori**: se selecciona un subconjunt de quatre aparellaments al atzar. S'anomena "consens inicial".
3. **Estimació del Model**: amb els punts seleccionats es calcula un model (en el nostre cas la matriu de transformació).
4. **Avaluació del consens**: s'avaluen tots els punts de les dades d'entrada amb el model estimat. Es consideren *inliners* aquells punts que compleixin una certa condició de tolerància (per exemple estar per sobre d'un llindar de decisió).
5. **Actualització del millor model**: si el nombre d'inliners pel model actual és major que l'obtingut amb les iteracions anteriors, es guarda el model com al millor trobat fins al moment.
6. **Repetició**: es repeteixen els passos 2-5 fins a complir un criteri de parada, p.ex. trobar un model amb un nombre suficient d'*inliners* o amb un error acceptable, o arribar a N iteracions.
7. **Resultat final**: se selecciona el model amb més *inliners* com a millor aproximació.

La següent equació ens permet calcular aspectes com la probabilitat d'èxit donades unes condicions inicials o el nombre d'iteracions necessari per a tenir una fiabilitat determinada:

$$(1 - (1 - e)^s)^N = 1 - p \quad \text{on}$$

N : iteracions
 p : probabilitat d'error
 e : outlier ratio
 s : nombre inicial d'aparellaments

En cas de no coneixer la proporció d'outliers, és bona pràctica assumir el cas pitjor: 50% outliers.

La següent taula indica, donats el nombre inicial d'aparellaments putatius i la proporció d'outliers, les iteracions necessàries per a tenir una probabilitat d'èxit del 99%.

proportion of outliers e							
s	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

Procés complet

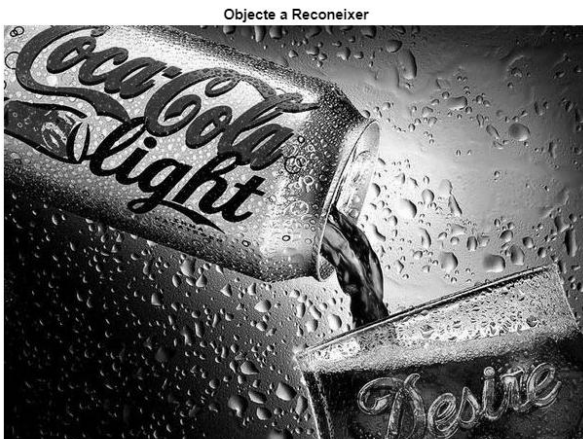
```
im_obj = imread('coke.jpg');  
figure, imshow(im_obj), title('Imatge Model')
```



```
im_obj = rgb2gray(im_obj);  
figure, imshow(im_obj), title('Imatge Model Mono')
```



```
im_esc = rgb2gray(imread('anunci.jpg'));  
figure, imshow(im_esc), title('Objecte a Reconixer')
```




```

kp_obj = detectSIFTFeatures(im_obj); % Prenem els paràmetres per defecte
kp_esc = detectSIFTFeatures(im_esc);

figure, imshow(im_obj), title('100 Keypoints');
hold on, plot(selectStrongest(kp_obj, 100))

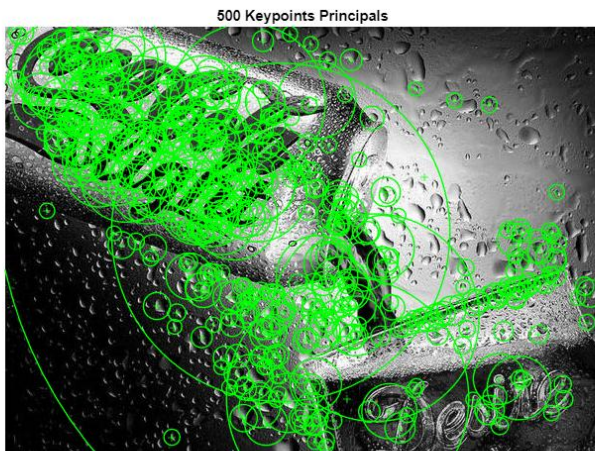
```



```

% Agafem més de 100 perquè l'escena tindrà més kp
figure, imshow(im_esc), title('500 Keypoints Principals');
hold on, plot(selectStrongest(kp_esc, 500))

```



```

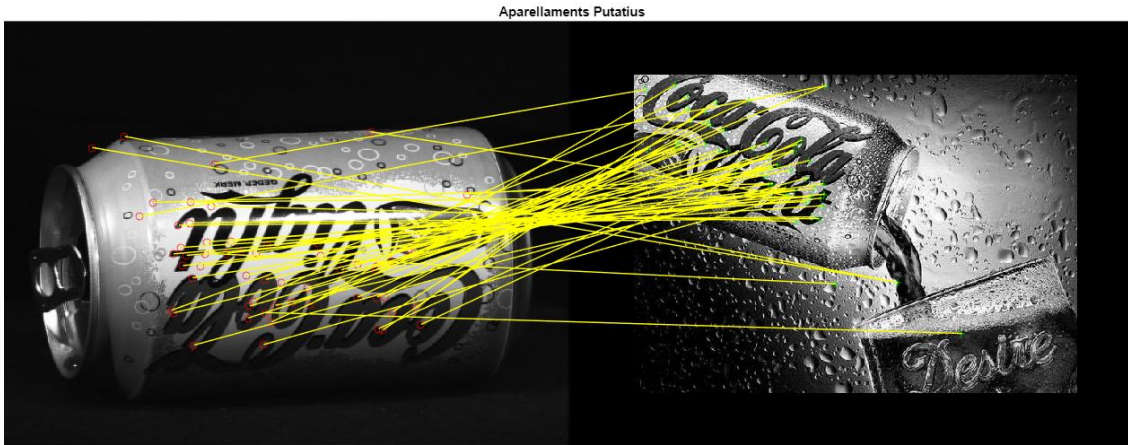
% Descriptors de SIFT
% Veiem a feat_obj.size q hi ha 899 punts d'interès
[feat_obj, kp_obj] = extractFeatures(im_obj, kp_obj);
% Veiem a feat_esc.size q hi ha 1876 punts d'interès
[feat_esc, kp_esc] = extractFeatures(im_esc, kp_esc);

% Aparellament
% Veiem a pairs.size que hi ha 71 aparellaments
pairs = matchFeatures(feat_obj, feat_esc, 'MatchThreshold', 10);

matched_kp_obj = kp_obj(pairs(:,1),:);
matched_kp_esc = kp_esc(pairs(:,2),:);
figure, showMatchedFeatures(im_obj, im_esc, matched_kp_obj, matched_kp_esc, 'montage');

```

```
title('Aparellaments Putatius')
```



```
% Global Matching
```

```
% Obtenim la matriu Tform
```

```
[tform, inliners] = estimateGeometricTransform2D(matched_kp_obj, matched_kp_esc, 'affine');
```

```
% A inliner_kp_obj.size i a inliner_kp_esc.size veiem que hi ha 34 aparellaments  
% putatius correctes (només necessitem 3 per a fer la rotació)
```

```
inliner_kp_obj = matched_kp_obj(inliners,:);
```

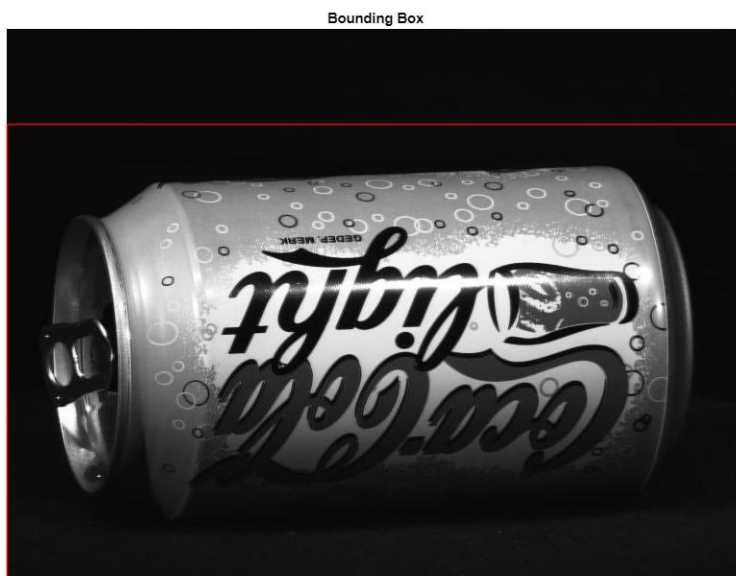
```
inliner_kp_esc = matched_kp_esc(inliners,:);
```

```
[miday, midax] = size(im_obj);
```

```
box_obj = [1,100; midax,100; midax,miday; 1,miday; 1,100];
```

```
figure, imshow(im_obj), title('Bounding Box')
```

```
hold on, line(box_obj(:,1),box_obj(:,2),'color','r');
```



```
box_esc = transformPointsForward(tform, box_obj);
```

```
figure, imshow(im_esc), title('Matching')
```

```
hold on, line(box_esc(:,1),box_esc(:,2),'color','r');
```

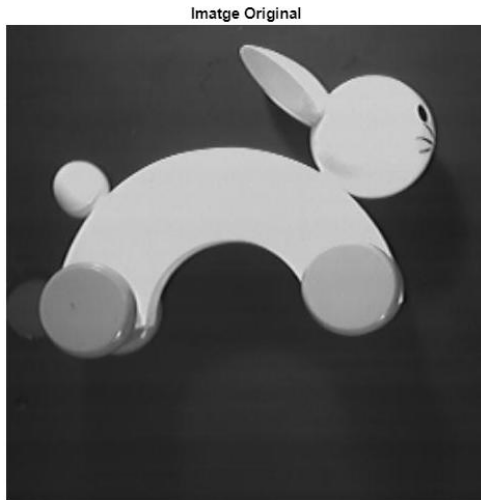
Matching



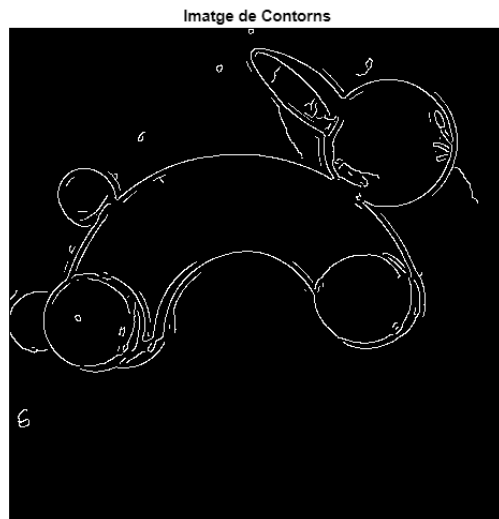
Entrega 21: KeyPoints II

Exercici: trobar circumferències en la imatge implementant la transformada de Hough sabent que l'equació de la circumferència és: $(x - a)^2 - (y - b)^2 = r^2$. Assumim $r = 58$.

```
im = imread('rabbit.jpg');  
figure, imshow(im), title('Imatge Original')
```



```
cont = edge(im, 'canny');  
figure, imshow(cont), title('Imatge de Contorns')
```



```
[rows, cols] = find(cont); % Veiem a rows.size que tenim 4442 candidats  
r = 58; % Com coneixem r, hem de trobar 'a' i 'b'
```

```
% La taula de hough tindrà mida 4442x4442
```

```
T_hough = zeros(size(im));
```

```
for coord=1:4442
```

```
    x = rows(coord);
```

```
    y = cols(coord);
```

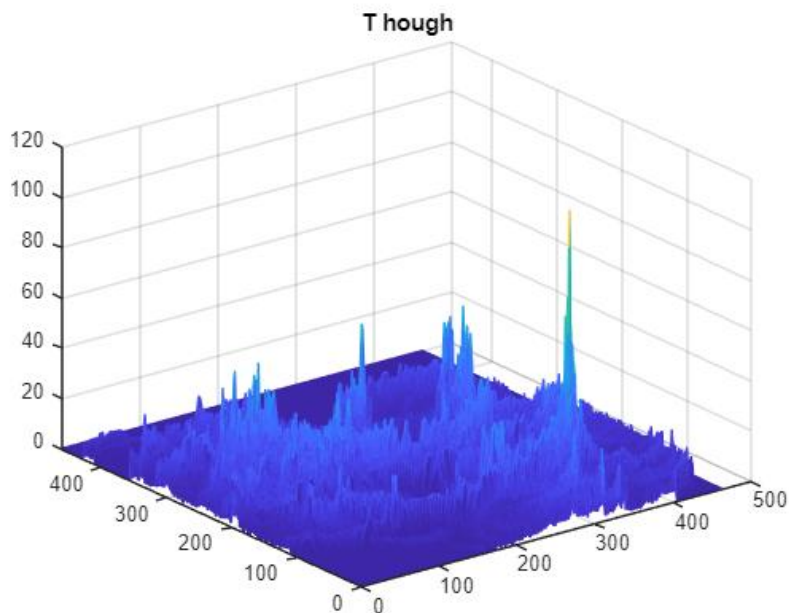
```

% Provar totes les combinacions possibles que satsfaguin l'equació
for i=-r:r
    cy = y + i;
    if cy > 0 && cy <= size(im, 1)
        % Calcular els possibles valors de cx utilitzant l'eq. de la circumferència
        d = r^2 - i^2;
        if d >= 0
            d = sqrt(d);
            cx1 = x + round(d);
            cx2 = x - round(d);

            % Verificar que cx1 y cx2 estan dins dels límits de la imatge
            if cx1 > 0 && cx1 <= size(im, 2)
                T_hough(cx1, cy) = T_hough(cx1, cy) + 1;
            end
            if cx2 > 0 && cx2 <= size(im, 2)
                T_hough(cx2, cy) = T_hough(cx2, cy) + 1;
            end
        end
    end
end
end
end

```

```
figure, mesh(T_hough), title('T_hough')
```



```

figure, imshow(im), title('Imatge Original')
[centrex, centrey] = find(T_hough==max(T_hough(:)));
hold on

```

```
viscircles([centrey, centrex],r,'Edgecolor','b');
```

Image Original

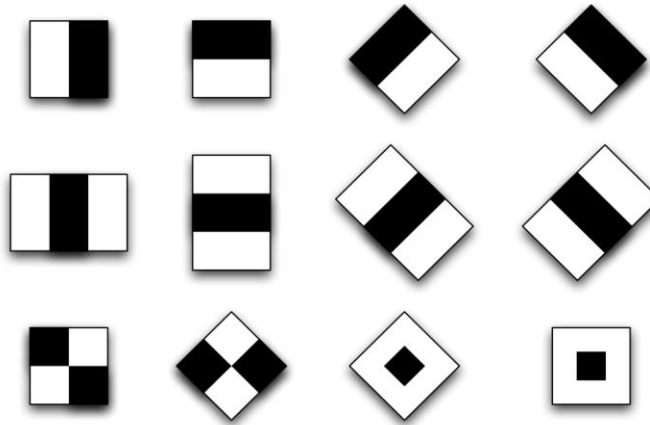


Entrega 22: Descriptors de Haar i l'algorisme de Viola-Jones

Descriptors de Haar

Serveixen per a detectar objectes en imatges. Típicament s'han utilitzat per a la detecció de cares, però també funcionen especialment bé amb gats i senyals de trànsit. De manera genèrica serveixen per a detectar objectes simètrics.

Els **filtres de Haar** són funcions simples que comparen àrees clares i fosques en una imatge, serveixen per a detectar canvis d'intensitat. Es representen com a combinacions de rectangles blancs i negres. Funcionen de manera similar als filtres de convolució però sense multiplicar.



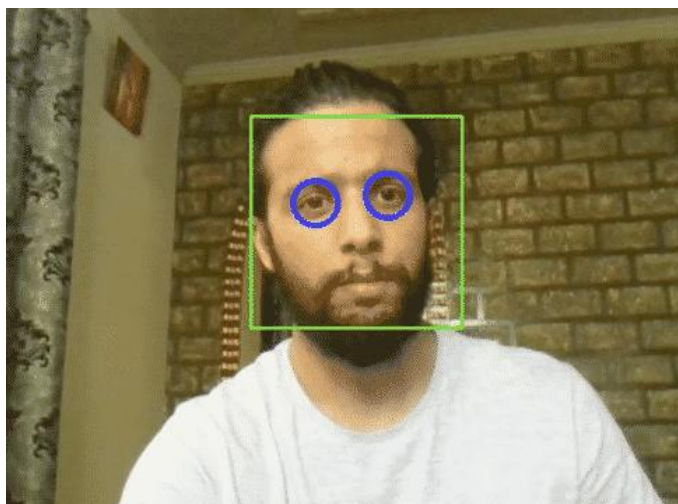
Per a obtenir els descriptors a partir de la imatge i els filtres, s'ha d'anar lliscant el filtre per sobre de la imatge sumant les àrees que quedin solapades pel blanc i restant les àrees solapades pel negre.

Algorisme de Viola-Jones

Va aparèixer l'any 2001 i ràpidament es va incorporar a la majoria de dispositius amb càmera digital ja que és un algorisme robust i que pot executar-se en temps real. L'algorisme permet la detecció de cares (no la seva identificació).

El mètode es basa en tres conceptes principals:

1. L'ús de descriptors Haar.
2. AdaBoost per a seleccionar els millors descriptors.
3. Classificació en cascada per a accelerar el procés.



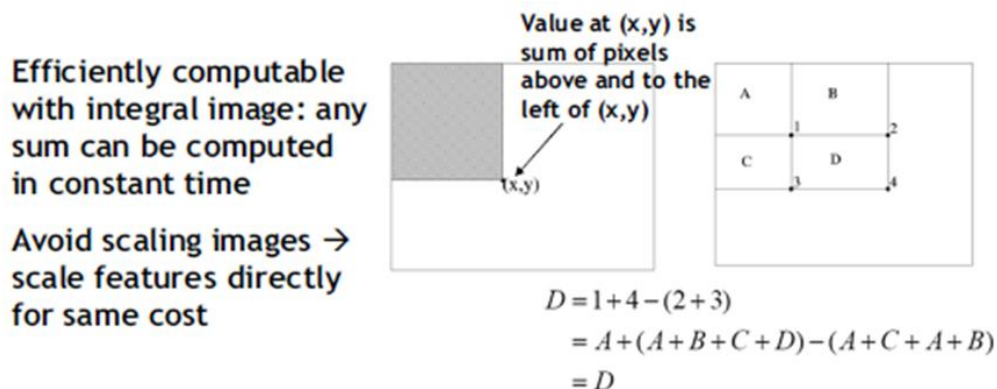
Les fases de l'algorisme sense accelerar són:

1. S'apliquen milers de descriptors Haar en diferents regions de la imatge per a detectar patrons característics d'una cara. Durant aquest procés s'utilitzen **imatges integrals**.
2. Selecció de les característiques més significatives mitjançant **AdaBoost** (no tots els descriptors seran útils).
3. Es repeteix el procés amb la imatge re-escalada (no es re-escala el filtre) per detectar cares de diferents mides.

El principal atractiu de l'algorisme Viola-Jones és el seu reduït temps de còmput, això s'assoleix essencialment gràcies a tres aspectes: les imatges integrals, la classificació en cascada i AdaBoost.

Imatges Integrals

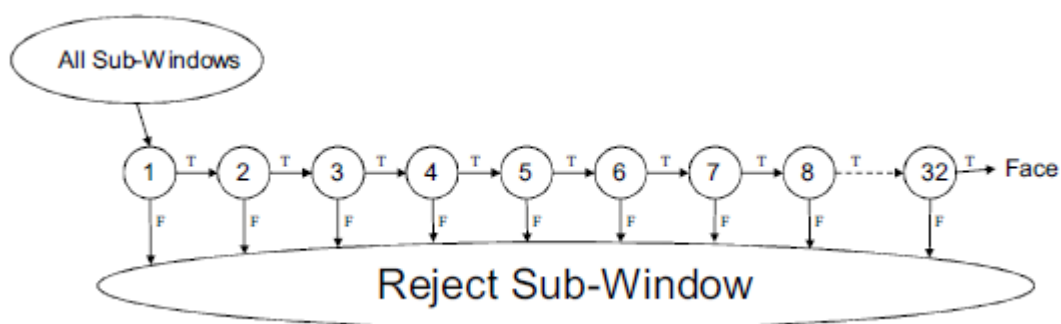
El càlcul de descriptors de Haar pot ser molt costós si es fa píxel a píxel ($O(n^2)$), és per això que s'utilitzen imatges integrals. Una imatge integral és una transformació de la imatge on cada píxel emmagatzema la suma acumulada de tots els píxels anteriors en la imatge, així doncs, calcular el descriptor de Haar a partir de la imatge integral es torna una operació $O(1)$ ja que podem fer-la amb només 4 accessos a la imatge.



Classificació en Cascada

Aplicar a tota la imatge milers de filtres, seria un procés molt costós. Així doncs la classificació en cascada el que fa és aplicar en un primer estadi els filtres més simples i ràpids a cada regió. Si la regió no compleix els criteris, es descarten immediatament i no s'hi apliquen els filtres posteriors. Això elimina al voltant del 90% de regions sense cares amb poc càlcul computacional. Aquest procés es repeteix sobre les imatges que surtin positives. A cada fase s'apliquen classificadors més complexos i precisos. Si una regió de la imatge no passa una etapa, ja no se la considera en les següents fases. Si una regió de la imatge passa totes les etapes, es considera una cara.

Per a aconseguir això, cada classificador s'entrena per a deixar passar un 10% de falsos positius aproximadament i detectar un 100% de les cares.



AdaBoost

És un mètode d'aprenentatge en conjunt. La idea és tenir diversos classificadors no massa bons que en combinar-se donin un resultat fiable. S'assignen pesos a cada classificador segons la seva fiabilitat i després es sumen els resultats ponderats per a obtenir el resultat final.



Ús de xarxes Neuronals a MATLAB

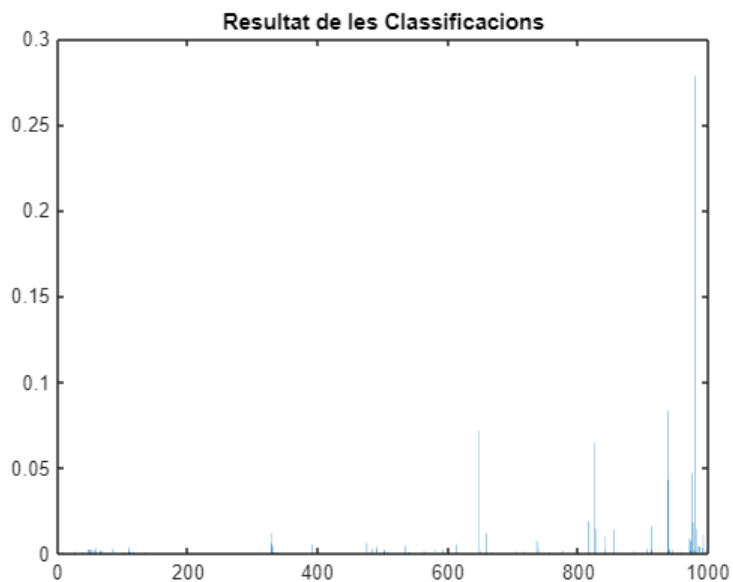
A continuació es mostra un exemple d'ús de xarxa neuronal (ANN) per aprendre'n uns bàsics destinats a facilitar la realització de la pràctica.

```
% Carreguem una xarxa neuronal genèrica pre-entrenada per a animals i objectes comuns
net = googlenet;
categories = net.Layers(end).ClassNames;

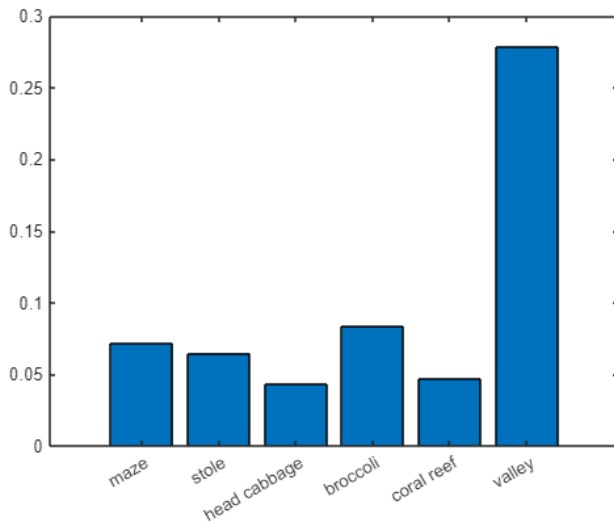
% Carreguem una imatge de test
im = imread('animals\animals\test_animals\test02.jpg');
figure, imshow(im), title('Test Image')
```



```
% Classifiquem la imatge amb la xarxa neuronal
im = imresize(im, [224, 224]);
[pred, scores] = classify(net, im);
figure, bar(scores), title('Resultat de les Classificacions')
```



```
% Filtrem només aquelles classes que tinguin una probabilitat superior al 2%
highscores = scores > 0.02;
figure, bar(scores(highscores));
xticklabels(categories(highscores))
```



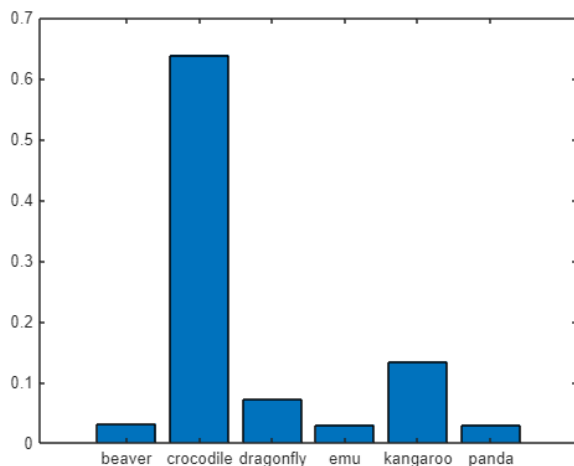
Veiem que hem obtingut uns resultats horribles, ja que hem pres una xarxa genèrica pre-entrenada. Per a obtenir una bona classificació ens caldrà entrenar la nostra pròpia xarxa.

```
% Carreguem una xarxa que hem entrenat específicament per a classificar els
% animals que ens interessin.
net = load('animals\animals\trainedNetwork_animals.mat');
categories = net.trainedNetwork_animals.Layers(end).ClassNames;

% Classifiquem la imatge
[pred, scores] = classify(net.trainedNetwork_animals, im);

% Filtrem els valors no significatius
highscores = scores > 0.02;

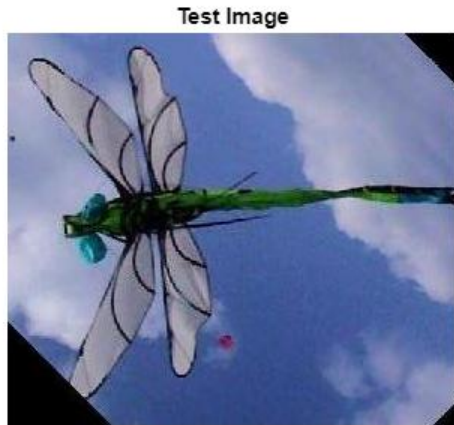
% Observem uns resultats satisfactoris
figure, bar(scores(highscores));
xticklabels(categories(highscores))
```



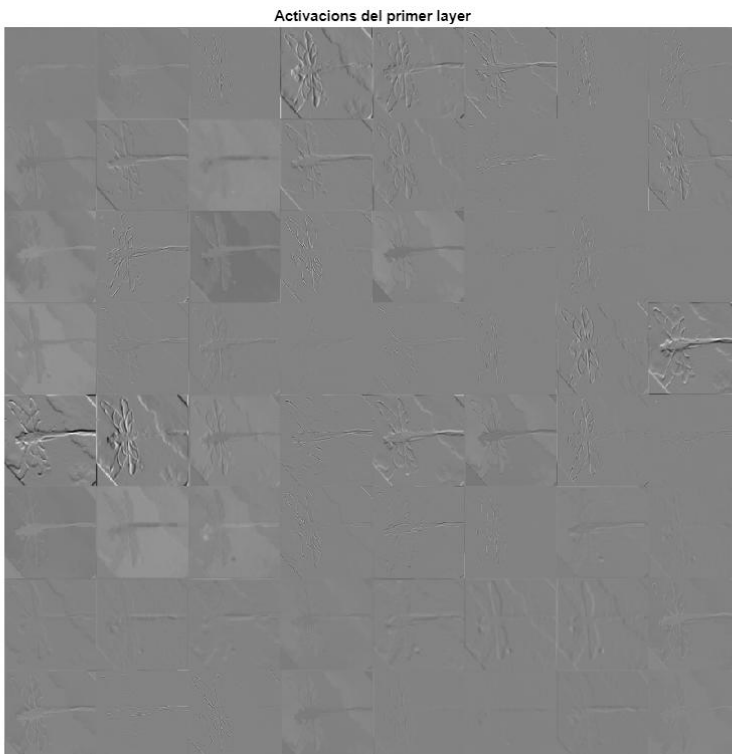
Les xarxes neuronals estan compostes de **capes convolucionals**, cada capa aplica filtres per a detectar diferents característiques de la imatge, com contorns o textures en les capes més superficials o patrons complexos en les capes més profundes. Cada filtre genera una **imatge d'activació**, que és una versió processada de la imatge original on només ressalten les característiques detectades pel filtre.

Amb el següent codi veiem les imatges d'activació de la primera capa.

```
im = imread('animals\animals\test_animals\test01.jpg');  
figure, imshow(im), title('Test Image')
```



```
im = imresize(im, [224, 224]);  
  
act1 = activations(net, im, 'conv1-7x7_s2');  
sz = size(act1);  
  
% Observem els 64 mapes de característiques resultants de les convolucions  
% aplicades a la imatge.  
act = reshape(act1, [sz(1),sz(2),1,sz(3)]);  
figure ,imshow(imtile(mat2gray(act), 'GridSize',[8,8])), title('Activacions del primer layer')
```



```
figure, imagesc(mean(act1, 3)), colormap jet, colorbar;  
title('Activacions promig de la primera capa convolucional');
```

